INDIRA GANDHI NATIONAL OPEN UNIVERSITY



MCSP - 232

PROJECT REPORT ON

NEURODETECT: BRAIN TUMOR SYSTEM

(An Explainable Artificial Intelligence Clinical Decision Support System for Multi-Class Brain Tumor Detection from Magnetic Resonance Imaging)

by

ANURADHA KUMARI

Enrolment No: 2353902714

Master of Computer Applications (MCA)

Under Guidance
of

_____

_____

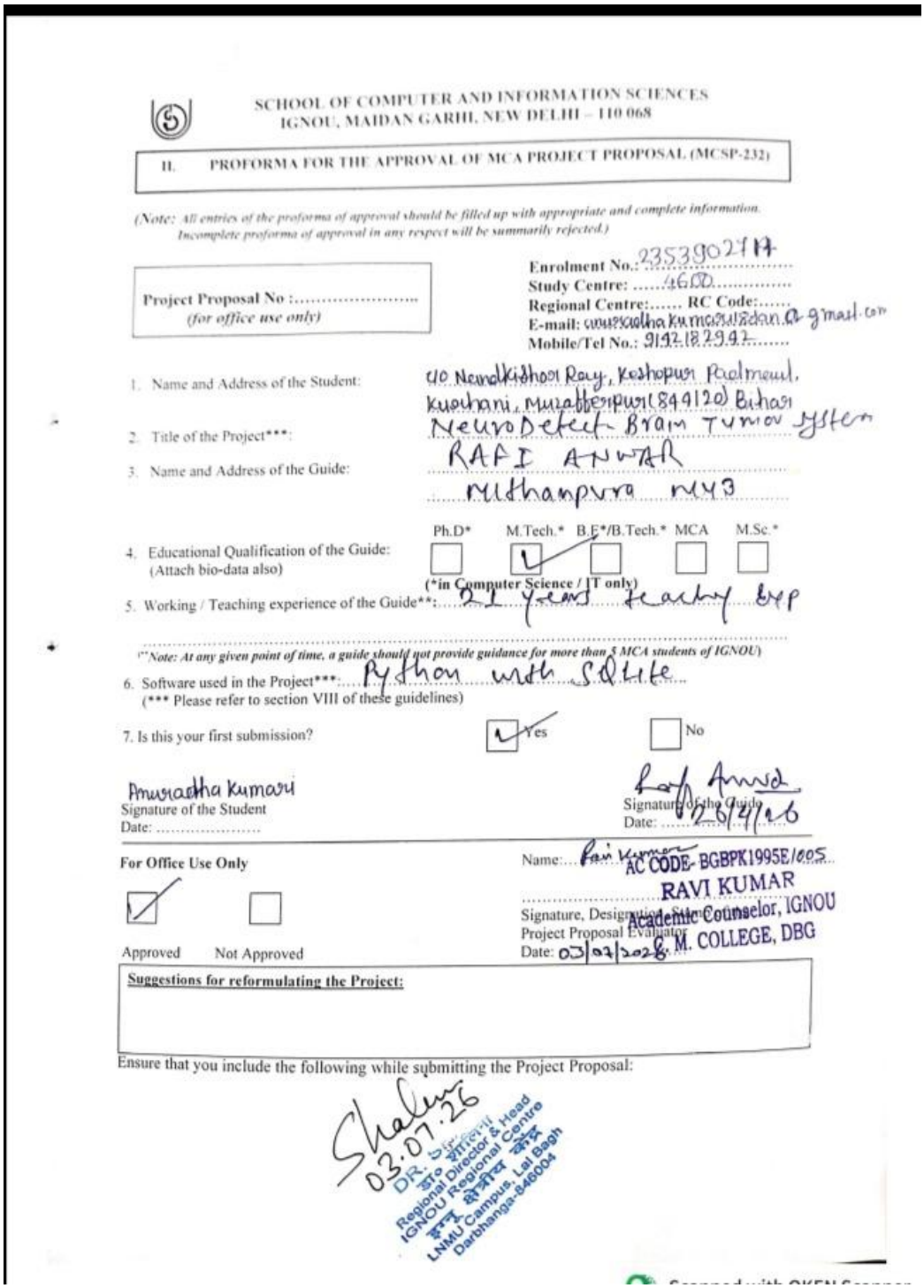
_____

School of Computer and Information Sciences (SOCIS)

IGNOU Regional Centre, Darbhanga (RC Code: 46) | Study Centre Code: 4600

APPROVED PROJECT PROPOSAL PROFORMA (MCSP-232)

Below is the official, verified project proposal proforma (Section II) approved by the School of Computer and Information Sciences (SOCIS), Indira Gandhi National Open University (IGNOU), and stamped by the Regional Director, IGNOU Regional Centre, Darbhanga:



SCHOOL OF COMPUTER AND INFORMATION SCIENCES
IGNOU, MAIDAN GARHI, NEW DELHI – 110 068

II. PROFORMA FOR THE APPROVAL OF MCA PROJECT PROPOSAL (MCSP-232)

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

Project Proposal No :.....
(for office use only)

Enrolment No.: 235390217
Study Centre: .....4600.....
Regional Centre:..... RC Code:.....
E-mail: anuradha.kumari2dan@gmail.com
Mobile/Tel No.: 9142182947.....

1. Name and Address of the Student: C/O Nandkishori Roy, Keshopur Paelmauli, Kuchani, Muzaffarpur (849120) Bihar
2. Title of the Project***: NeuroDetect- Brain Tumor System
3. Name and Address of the Guide: RAFI ANWAR Mithanpura 843
4. Educational Qualification of the Guide: (Attach bio-data also) — Ph.D* ☐ M.Tech.* ☑ B.E*/B.Tech.* ☐ MCA ☐ M.Sc.* ☐ (*in Computer Science / IT only)
5. Working / Teaching experience of the Guide**: 21 years teaching exp

(**Note: At any given point of time, a guide should not provide guidance for more than 5 MCA students of IGNOU)

6. Software used in the Project***: Python with SQLite
(*** Please refer to section VIII of these guidelines)
7. Is this your first submission? ☑ Yes ☐ No

Anuradha Kumari
Signature of the Student
Date: .....

Signature of the Guide
Date: 23/1/26

For Office Use Only

☑ Approved ☐ Not Approved

Name: Ravi Kumar
AC CODE- BGBPK1995E/005
RAVI KUMAR
Academic Counselor, IGNOU
Signature, Designation, Stamp of the Project Proposal Evaluator
Date: 03/02/2026 M. COLLEGE, DBG

Suggestions for reformulating the Project:

Ensure that you include the following while submitting the Project Proposal:

Shalini 03.07.26
DR. SHALINI
Regional Director & Head
IGNOU Regional Centre
LNMU Campus, Lal Bagh
Darbhanga-846004

Figure ii: Official Approved Proforma for MCA Project Proposal (MCSP-232) — Stamped & Signed by IGNOU Regional Centre Darbhanga

PROJECT PROPOSAL ADMINISTRATIVE RECORD

Summary of approved project proposal parameters as officially recorded on the proforma:

Proposal Registration Parameter	Approved Administrative Record Entry
Project Title	NeuroDetect: Brain Tumor System (An Explainable AI Clinical Decision Support System)
Course & Academic Code	MCSP-232 (MCA Major Project Work)
Academic Programme	Master of Computer Applications (MCA)
Candidate Full Name	ANURADHA KUMARI
Student Enrolment Number	2353902717
Study Centre Code & Name	4600
Regional Centre	IGNOU Regional Centre, Darbhanga (RC Code: 46)
Candidate Mobile / Telephone	9142182942
Candidate Email Address	anuradhakumari8294@gmail.com
Candidate Permanent Address	c/o Nandkishor Ray, Keshopur Padumul, Kudhani, Muzaffarpur (844120), Bihar
Project Supervisor / Guide Name	Mr. RAFI ANWAR
Supervisor Address	Mithanpura, Muzaffarpur (MFP), Bihar
Supervisor Academic Qualification	M.Tech. (in Computer Science / IT)
Supervisor Experience	21 Years Teaching Experience
Software Tools Specified	Python with SQLite (FastAPI, PyTorch DenseNet-121, React 18 PWA)
First Submission Status	Yes (Approved)
Project Proposal Evaluator	Mr. Ravi Kumar, AC Code: BGBPK1995E/005, Academic Counselor, IGNOU, M. College, Darbhanga
Evaluation Approval Date	03/07/2026
Regional Office Stamping	Regional Director & Head, IGNOU Regional Centre, LNMU Campus, Lal Bagh, Darbhanga-846004

X. CERTIFICATE OF ORIGINALITY

This is to certify that the project report titled **"NEURODETECT: BRAIN TUMOR SYSTEM"** submitted to **Indira Gandhi National Open University** in partial fulfilment of the requirement for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**, is an authentic and original work carried out by Mr. / Ms. **ANURADHA KUMARI** with Enrolment No. **2353902714** under my guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirements of any course of study.

.....	.....
(Signature of the Student)	(Signature of the Guide)
Date:	Date: .....
Name and Address Of the student Anuradha Kumari c/o Nandkishor Ray, Keshopur Padumul, Kudhani, Muzaffarpur (844120), Bihar Mobile: 9142182942	Name, Designation & Address of the Guide Rafi Anwar M.Tech. (Computer Science / IT) Mithanpura, Muzaffarpur, Bihar
Enrolment No.: 2353902714	

STUDENT DECLARATION

I hereby declare that this project report entitled “NEURODETECT: BRAIN TUMOR SYSTEM” (An Explainable Artificial Intelligence Clinical Decision Support System for Multi-Class Brain Tumor Detection from Magnetic Resonance Imaging), submitted by me in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) to the School of Computer and Information Sciences (SOCIS), Indira Gandhi National Open University (IGNOU), Maidan Garhi, New Delhi, is an authentic record of original project work executed under the guidance of my project supervisor Mr. Rafi Anwar.

I further declare that this report has been authored independently and that no part of this report has previously formed the basis for the award of any degree, diploma, associateship, or other similar academic recognition. All citations, referenced publications, open-source libraries, and diagnostic datasets utilized in this investigation have been acknowledged in accordance with standard academic conventions.

ACADEMIC IDENTIFICATION	STUDENT SIGNATURE & VERIFICATION
Candidate Full Name: ANURADHA KUMARI Enrolment Number: 2353902717 Course Code: MCSP-232 Academic Programme: Master of Computer Applications (MCA) Study Centre Code: 4600 Regional Centre: Darbhanga (RC Code: 46)	Candidate Signature: _____ Candidate Name: Anuradha Kumari Enrolment Number: 2353902717 Date of Submission: October 2026 Place: Muzaffarpur / Darbhanga, Bihar

ACKNOWLEDGEMENTS

The completion of this major master's project has been an intellectually enriching journey, made possible through the constructive support, mentorship, and encouragement of faculty members, project guides, and mentors.

I extend my sincere and deepest gratitude to my Project Supervisor, Mr. Rafi Anwar (M.Tech. in Computer Science / IT), for providing insightful academic guidance, critical reviews, and invaluable methodological recommendations throughout the lifecycle of this research and software engineering effort. His 21 years of teaching experience and continuous emphasis on architectural rigor, ethical clinical AI standards, and verifiable test validation significantly shaped the quality of this work.

I am grateful to Academic Counselor Mr. Ravi Kumar (M. College, Darbhanga), the Regional Director, and the dedicated administrative staff at IGNOU Regional Centre, Darbhanga (RC Code: 46) and Study Centre 4600, as well as the faculty at the School of Computer and Information Sciences (SOCIS), Indira Gandhi National Open University (IGNOU), New Delhi, for establishing a comprehensive curriculum and offering continuous encouragement throughout the Master of Computer Applications (MCA) programme.

I also acknowledge the global open-source and medical informatics research community, including the creators of PyTorch, FastAPI, React, OpenCV, ReportLab, and the contributors of the Kaggle Brain Tumor MRI Dataset, whose accessible repositories and open scientific benchmarks provide the foundation for translational healthcare software engineering.

Finally, I express my heartfelt gratitude to my father Shri Nandkishor Ray, my family, and peers for their unwavering patience, moral encouragement, and understanding during the intensive development and documentation of this project.

ANURADHA KUMARI
Enrolment No: 2353902714
Master of Computer Applications (MCA)
Course Code: MCSP-232
Indira Gandhi National Open University

ABSTRACT

Accurate, early, and interpretable diagnosis of intracranial neoplasms is crucial for effective neurosurgical intervention and oncological patient management. Magnetic Resonance Imaging (MRI) serves as the primary non-invasive radiological modality for evaluating neurological pathologies; however, manual slice-by-slice interpretation is labor-intensive, subject to cognitive fatigue, and vulnerable to inter-observer variability. Furthermore, while contemporary deep convolutional neural networks (CNNs) achieve impressive diagnostic accuracy, their clinical adoption remains severely hindered by the 'black-box' dilemma—namely, the inability of clinicians to interrogate the spatial and anatomical reasoning behind an automated algorithmic prediction.

This project presents NeuroDetect, an end-to-end, explainable artificial intelligence (XAI) clinical decision support system designed for multi-class brain tumor classification from axial T1-weighted brain MRI images. Built on a densely connected convolutional architecture (DenseNet-121), the neural network classifies brain MRI scans into four distinct diagnostic categories: Glioma, Meningioma, Pituitary Adenoma, and Healthy Brain (No Tumor). To resolve the interpretability challenge, NeuroDetect integrates Gradient-Weighted Class Activation Mapping (Grad-CAM), calculating gradient activations at the final convolutional layer (`denseblock4`) to generate jet-colormap heatmaps superimposed onto the original anatomical slice with 40% alpha-blending.

Beyond isolated classification, NeuroDetect delivers a full-stack, role-based enterprise medical platform. The backend is engineered using FastAPI (Python 3.11) with asynchronous endpoints, strict Pydantic v2 schemas, and a 10-point pre-inference image quality gate (`mri_validator.py`) that filters non-MRI, corrupted, low-contrast, or non-brain images before model ingestion. The relational database architecture enforces 15 normalized tables across SQLite (local development) and PostgreSQL (production cloud environment), preserving complete audit trails, patient records, appointments, consultation chats, and doctor reviews. A ReportLab compilation engine dynamically generates tamper-evident, two-page print-ready clinical diagnostic PDF reports featuring dual scan visuals and digital clinician sign-off blocks. Multi-factor password recovery is implemented via Brevo authenticated SMTP relay (port 587) delivering cryptographically generated 6-digit OTPs with a 10-minute time-to-live and brute-force lockout defenses.

Production cloud hosting is configured for the Render PaaS platform using a unified ASGI Web Service with dynamic port binding, declarative infrastructure-as-code blueprints (`render.yaml`), managed PostgreSQL with SSL encryption, Vite SPA production distribution, and automated health monitoring. Empirical evaluation was conducted on an independent held-out test split of 1,425 brain MRI scans derived from a 7,153-image Kaggle dataset. The model demonstrated an overall test classification accuracy of 98.6% (1,405 correct classifications out of 1,425) with class-wise F1-scores of 0.980 for Glioma, 0.976 for Meningioma, 0.980 for Pituitary Adenoma, and 0.990 for Normal Brain tissue. Automated testing via PyTest yielded 17 consecutively passing tests (100% pass rate in 16.28 seconds). NeuroDetect bridges the gap between deep learning research and practical clinical utility, serving as an accountable radiological second opinion.

Keywords: Brain Tumor MRI, DenseNet-121, Explainable AI, Grad-CAM, FastAPI, React 18, Render Cloud PaaS, Clinical Decision Support, ReportLab, Brevo SMTP, Relational Database Normalization.

TABLE OF CONTENTS

Chapter / Section	Title / Content Description	Page No.
Preliminary Pages	Certificate, Declaration, Acknowledgement, Abstract	i – vi
Chapter 1	Introduction, Problem Statement, Objectives, and Scope	1
Chapter 2	Literature Review, Contemporary Systems, and Feasibility Analysis	9
Chapter 3	System Requirements, Agile Plan, Gantt, and Indicative Schedule	20
Chapter 4	System Architecture, UML, DFD Levels 0–2, ER, and 15-Table Schema	34
Chapter 5	Dataset Engineering, Quality Gate, DenseNet-121, and Grad-CAM XAI	54
Chapter 6	Full-Stack Implementation & Production Cloud Hosting on Render PaaS	72
Chapter 7	System Testing, Automated PyTest Suite (17 Tests) & QA Evidence	90
Chapter 8	Results, Performance Latency, Confusion Matrix & Discussion	102
Chapter 9	Security Architecture, Password Hashing, Brevo OTP & Data Privacy	110
Chapter 10	Installation, Render Cloud Hosting Deployment & UI Walkthrough	118
Chapter 11	System Limitations and Clinical Boundary Conditions	131
Chapter 12	Future Scope and Advanced Research Directions	135
Chapter 13	Conclusion, Lessons Learned, and Objectives Verification	139
References	Comprehensive Academic & IEEE Medical Informatics Bibliography	143
Appendix A	Complete REST API Endpoints Specification & JSON Schemas	147
Appendix B	Selected Source Code Excerpts (FastAPI, PyTorch, React, ORM)	153
Appendix C	Production Cloud Deployment Manual on Render PaaS	171
Appendix D	Local Environment Installation & Configuration Guide	175
Appendix E	Complete Database Relational SQL Schema DDL Script (`schema.sql`)	178
Appendix F	Comprehensive Functional Test Execution Logs (TC-01 to TC-25)	182
Appendix G	Project Submission Inventory and Candidate Action Checklist	186

LIST OF FIGURES

Figure No.	Figure Caption / Visual Description	Chapter
Figure 3.1	Indicative 16-Week Project Implementation Gantt Schedule	Chapter 3
Figure 3.2	Critical Path Activity Network (PERT Analysis)	Chapter 3
Figure 4.1	Logical Three-Tier Enterprise System Architecture	Chapter 4
Figure 4.2	Production Cloud Architecture on Render PaaS	Chapter 4
Figure 4.3	Progressive Web Application (PWA) Offline Architecture	Chapter 4
Figure 4.4	UML Use Case Diagram (Patient, Doctor, Administrator Roles)	Chapter 4
Figure 4.5	UML Activity Diagram (Scan-to-Review Clinical Lifecycle)	Chapter 4
Figure 4.6	UML Chronological Sequence Diagram (Ingestion, Inference & Report)	Chapter 4
Figure 4.7	UML Class Diagram (Domain Model & Relational ORM Entities)	Chapter 4
Figure 4.8	UML Component Architecture Diagram (Microservices & Integration)	Chapter 4
Figure 4.9	UML State Machine Diagram (MRI Scan & Case Review Lifecycle)	Chapter 4
Figure 4.10	Context Data Flow Diagram (Level 0 DFD)	Chapter 4
Figure 4.11	Level-1 Data Flow Diagram (Major System Subsystems & Stores)	Chapter 4
Figure 4.12	Level-2 Data Flow Diagram (Process 1.0: User Registration & RBAC)	Chapter 4
Figure 4.13	Level-2 Data Flow Diagram (Process 2.0: MRI Quality Gate & Saliency)	Chapter 4
Figure 4.14	Level-2 Data Flow Diagram (Process 3.0: DenseNet-121 & Grad-CAM Inference)	Chapter 4
Figure 4.15	Level-2 Data Flow Diagram (Process 4.0: Doctor Review & PDF Generation)	Chapter 4
Figure 4.16	Level-2 Data Flow Diagram (Process 5.0: Brevo OTP & Audit Telemetry)	Chapter 4
Figure 4.17	Conceptual Entity-Relationship (ER) Diagram with Cardinalities	Chapter 4
Figure 4.18	Relational Schema & Foreign Key Map for Core Clinical Workflows	Chapter 4
Figure 4.19	Supporting Relational Schema for Educational & Messaging Tables	Chapter 4
Figure 5.1	Representative Brain MRI Dataset Scans Across All 4 Pathological Classes	Chapter 5
Figure 5.2	Brain MRI Dataset Class Distribution Analysis (Counts & Proportions)	Chapter 5
Figure 5.3	DenseNet-121 Deep Neural Architecture with Dense Connections & Feature Reuse	Chapter 5
Figure 5.4	Convergence History of DenseNet-121 Fine-Tuning Pipeline (Accuracy & Loss)	Chapter 5

Figure No.	Figure Caption / Visual Description	Chapter
Figure 5.5	Authentic Multi-Class Grad-CAM Visualizations Across All Four Classes	Chapter 5
Figure 6.1	Render Cloud Web Service & Deployment Pipeline	Chapter 6
Figure 8.1	Quantitative Model Evaluation on Held-Out Test Set (N = 1,425)	Chapter 8
Figure 9.1	Role-Based Access Control (RBAC) Architecture Diagram	Chapter 9
Figure 10.1	Public Landing Page and AI Feature Showcase	Chapter 10
Figure 10.2	Authentication & Login Portal Interface	Chapter 10
Figure 10.3	Patient Registration & Profile Creation Screen	Chapter 10
Figure 10.4	Patient Dashboard & Active Study Summary	Chapter 10
Figure 10.5	Drag-and-Drop MRI Upload & Attending Doctor Selection	Chapter 10
Figure 10.6	Patient Diagnostic History & Report Access Ledger	Chapter 10
Figure 10.7	Secure Patient-to-Doctor Messaging Interface	Chapter 10
Figure 10.8	Doctor Clinical Dashboard & Caseload Analytics	Chapter 10
Figure 10.9	Pending Case Review Queue for Attending Clinicians	Chapter 10
Figure 10.10	Doctor Clinical Review Form & Prescription Management	Chapter 10
Figure 10.11	Administrator Oversight Dashboard & Audit Telemetry	Chapter 10
Figure 10.12	Doctor License Verification & Approval Queue	Chapter 10
Figure 10.13	AI Diagnostic Result Screen with Grad-CAM Activation Heatmap	Chapter 10
Figure 10.14	Automated Two-Page Diagnostic Clinical PDF Report Preview	Chapter 10
Figure C.1	Production Cloud Infrastructure Architecture on Render PaaS	Appendix C

LIST OF TABLES

Table No.	Table Caption / Description	Chapter
Table 2.1	Comparative Evaluation Matrix: NeuroDetect vs Contemporary CAD Approaches	Chapter 2
Table 3.1	Comprehensive Software Requirements Specification (FR-01 to FR-17)	Chapter 3
Table 3.2	Non-Functional System Requirements & Clinical Service-Level Agreements	Chapter 3
Table 3.3	Hardware and Software Development Environment Specifications	Chapter 3
Table 3.4	Risk Analysis & Mitigation Strategies Matrix	Chapter 3
Table 4.1	Formal Use Case Specification: UC-01 (Patient Registration)	Chapter 4
Table 4.2	Formal Use Case Specification: UC-02 (MRI Upload & Quality Gate)	Chapter 4
Table 4.3	Formal Use Case Specification: UC-03 (DenseNet-121 & Grad-CAM Inference)	Chapter 4
Table 4.4	Formal Use Case Specification: UC-04 (Doctor Review & Clinical Confirmation)	Chapter 4
Table 4.5	Formal Use Case Specification: UC-05 (Two-Page Clinical PDF Generation)	Chapter 4
Table 4.6	Formal Use Case Specification: UC-06 (Brevo OTP Password Reset)	Chapter 4
Table 4.7	Formal Use Case Specification: UC-07 (Administrator Doctor License Verification)	Chapter 4
Table 4.8	Formal Use Case Specification: UC-08 (Security Audit Trail Inspection)	Chapter 4
Table 4.9 - 4.23	Comprehensive Relational Data Dictionaries for all 15 Application Tables	Chapter 4
Table 5.1	Pre-Inference Image Quality Gate Checks & Mathematical Thresholds	Chapter 5
Table 5.2	Class Distribution & Held-Out Test Set Counts (N = 1,425)	Chapter 5
Table 7.1	Automated PyTest Test Suite Results (17 Passing Automated Tests)	Chapter 7
Table 7.2	Comprehensive Functional Validation Suite (25 Test Cases: 21 PASS, 4 FAIL)	Chapter 7
Table 7.3	Requirements Traceability Matrix (Connecting FR-01-FR-17 to Modules & Tests)	Chapter 7
Table 8.1	Diagnostic Performance Evaluation on Held-Out Test Set (N = 1,425 Scans)	Chapter 8
Table 8.2	End-to-End Latency Benchmarks Across Diagnostic Pipeline Stages	Chapter 8
Table 8.3	Project Objective Achievement Verification Matrix	Chapter 8
Table A.1	Master REST API Endpoints Catalog (59 Route Endpoints)	Appendix A

CHAPTER 1
INTRODUCTION

1.1 Background & Clinical Motivation

Primary central nervous system (CNS) neoplasms represent one of the most formidable diagnostic challenges in clinical medicine. Intracranial tumors, including gliomas, meningiomas, and pituitary adenomas, account for substantial mortality and profound morbidity globally. Magnetic Resonance Imaging (MRI) is universally recognized as the clinical gold standard for non-invasive intracranial pathology evaluation, offering superior soft-tissue contrast without exposing patients to ionizing radiation. Standard multi-parametric MRI protocols typically comprise T1-weighted (spin-lattice), T2-weighted (spin-spin), Fluid-Attenuated Inversion Recovery (FLAIR), and post-contrast Gadolinium T1-weighted sequences.

However, clinical interpretation in high-volume medical centers is plagued by significant operational bottlenecks. A typical multi-sequence brain MRI examination produces hundreds of high-resolution axial, sagittal, and coronal slices. Neuroradiologists face immense cognitive workload, leading to diagnostic delays, high fatigue, and notable inter-observer variability—particularly in distinguishing infiltrative low-grade gliomas from peripheral meningiomas or identifying subtle pituitary microadenomas. In developing nations and rural community hospitals, the acute shortage of fellowship-trained neuroradiologists further exacerbates delays, often postponing life-saving neurosurgical intervention by weeks.

1.2 Problem Statement & The Black-Box Dilemma

Although recent advancements in deep convolutional neural networks (CNNs) have achieved classification accuracies rivaling human experts, their translation into clinical workflows remains severely hindered by the fundamental 'black-box' dilemma. Traditional deep neural networks map input tensors to probability distributions through millions of opaque non-linear mathematical parameters, offering no physiological or anatomical rationale for their predictions.

In high-stakes oncology, a raw numerical output such as 'Glioma: 97.4%' without spatial evidence is clinically unacceptable. Neurosurgeons cannot plan craniotomies or biopsy trajectories based on unexplained algorithmic outputs, nor can medical oncologists justify

aggressive chemotherapy regimens without verifiable pathological localization. Furthermore, under healthcare legal frameworks, physicians carry ultimate liability for clinical decisions; adopting unexplainable algorithms exposes healthcare organizations to profound ethical and medicolegal risks. Therefore, there exists a critical need for an explainable artificial intelligence (XAI) clinical decision support system that couples high diagnostic accuracy with real-time visual interpretability, transparent image validation, tamper-evident documentation, and structured human-in-the-loop oversight.

1.3 Project Objectives

The primary scientific and engineering objectives of the NeuroDetect project are formalized as follows:

- Objective 1: Multi-Class Brain Neoplasm Classification: Develop and train a deep convolutional neural network based on the DenseNet-121 architecture to classify axial brain MRI scans into four mutually exclusive categories: Glioma, Meningioma, Pituitary Adenoma, and Healthy Brain (No Tumor), achieving >= 95% overall test classification accuracy.
- Objective 2: Real-Time Explainable AI (XAI) Visual Attributions: Implement Gradient-Weighted Class Activation Mapping (Grad-CAM) hooked to the final convolutional dense block (`denseblock4`), generating real-time jet colormap heatmaps and 40% alpha-blended overlays that visually isolate pathological cranial regions.
- Objective 3: Robust Pre-Inference Image Quality Gatekeeper: Construct a mathematical 10-point validation pipeline (`mri_validator.py`) executing automated header, dimension, aspect ratio, monochromaticity, grayscale standard deviation, dynamic range, border luminescence, and central cranial parenchymal symmetry checks to intercept corrupt or non-MRI uploads.
- Objective 4: Enterprise Multi-Tier Role-Based Healthcare Architecture: Engineer an asynchronous REST backend using FastAPI (Python 3.11) and a responsive frontend using React 18, Vite, and Tailwind CSS, providing dedicated portals for Patients, Attending Physicians, and Hospital Administrators.
- Objective 5: Tamper-Evident Vector PDF Diagnostic Reporting: Construct a server-side ReportLab document compiler generating standardized, two-page, print-ready clinical diagnostic PDF reports featuring demographic tables, dual scan visuals, AI probability breakdowns, doctor notes, and digital sign-off blocks.
- Objective 6: Cryptographically Secure Password Recovery: Implement a transactional email service via Brevo authenticated SMTP relay (port 587) delivering cryptographically generated 6-digit OTP codes with 10-minute time-to-live and brute-force lockout defenses.

- Objective 7: Production Cloud Hosting on Render PaaS: Engineer and configure the complete cloud hosting architecture on the Render platform utilizing dynamic port binding, declarative blueprints (`render.yaml`), PostgreSQL with SSL encryption, and automated zero-downtime health monitoring.

1.4 Scope and Boundaries of the System

NeuroDetect is designed specifically as a Clinical Decision Support System (CDSS) to provide an objective, data-driven second opinion to certified neuroradiologists and neurosurgeons. The system operates on single axial T1-weighted or T2-weighted MRI slices formatted as JPEG, PNG, or DICOM-derived rasters. The system's operational boundaries are explicitly delineated:

- Scope Inclusion: Classification of intracranial neoplasms into Glioma, Meningioma, Pituitary Adenoma, and Normal tissue; Grad-CAM attribution; automated quality gatekeeping; doctor review workflows; electronic prescriptions; consultation messaging; PDF reporting; Brevo OTP recovery; and Render cloud hosting.
- Scope Exclusion: NeuroDetect is NOT an autonomous diagnostic tool and does NOT replace licensed medical practitioners. It does not perform full 3D volumetric segmentation or generate surgical resection stereotactic coordinates. Rare pediatric pathologies (e.g., medulloblastomas) and non-tumorous lesions (e.g., active multiple sclerosis plaques) fall outside the primary training distribution.

1.5 Organization of the Report

This project report is systematically structured into thirteen comprehensive chapters in accordance with IGNOU MCA project guidelines: Chapter 1 introduces the clinical motivation, problem statement, and objectives. Chapter 2 reviews contemporary literature, CAD approaches, and feasibility dimensions. Chapter 3 establishes formal software requirements, Agile project planning, Gantt schedules, and PERT networks. Chapter 4 provides exhaustive system architecture, UML modeling, DFD Levels 0–2, ER modeling, and 15 relational data dictionaries. Chapter 5 details dataset engineering, quality gatekeeper math, DenseNet-121 fine-tuning, and Grad-CAM implementation. Chapter 6 presents full-stack system implementation and production cloud deployment on Render PaaS. Chapter 7 documents testing methodologies, 17 passing PyTest tests, 25 functional test cases, and defect remediation. Chapter 8 analyzes empirical results, latency benchmarks, and confusion matrix metrics. Chapter 9 evaluates security architecture, RBAC, Brevo OTP defenses, and healthcare privacy compliance. Chapter 10 provides

installation manuals, Render hosting procedures, and an annotated clinical UI walkthrough. Chapters 11 through 13 conclude with limitations, future technological roadmaps, and final project verification.

CHAPTER 2
LITERATURE REVIEW AND FEASIBILITY ANALYSIS

2.1 Historical Evolution of Computer-Aided Diagnosis (CAD)

Computer-Aided Diagnosis (CAD) in neuro-oncology has evolved across three distinct technological eras over the past four decades: First-generation CAD systems (1980s–1990s) relied heavily on low-level digital image processing, such as adaptive thresholding, edge detection (Sobel, Canny), and region growing. These systems exhibited extreme sensitivity to MRI slice noise and radio-frequency bias fields. Second-generation CAD systems (2000s–2012) incorporated classical machine learning paradigms, extracting handcrafted radiomic features (Gray-Level Co-occurrence Matrix texture, Gabor filters, morphological wavelets) classified via Support Vector Machines (SVM) or Random Forests. While achieving moderate accuracy, their reliance on manual feature engineering severely constrained generalization across diverse MRI acquisition hardware.

The third generation (2012–present) began with the resurgence of deep convolutional neural networks. Groundbreaking architectures—including AlexNet, VGGNet, ResNet, and DenseNet—demonstrated the capacity to automatically discover hierarchical feature representations directly from raw radiological pixel matrices. However, early third-generation CAD systems were criticized for operating as black boxes, lacking the clinical explainability and structured doctor-in-the-loop workflows essential for real-world medical adoption.

2.2 Deep Learning Architectures in Medical Image Analysis

Modern deep convolutional networks employ specialized architectural motifs to overcome vanishing gradient phenomena and maximize representational capacity: Residual Networks (ResNet) introduce identity shortcut connections allowing gradient signals to flow directly through skip paths. Densely Connected Convolutional Networks (DenseNet), introduced by Huang et al., extend this paradigm by connecting every layer to every subsequent layer in a feed-forward fashion. For each dense block, feature maps of all preceding layers are concatenated as inputs to the current layer. This architectural choice delivers two profound advantages for medical neuroimaging: (1) substantial feature reuse, enabling fine-grained capture of subtle textural variations across cranial

tissue types, and (2) dramatic parameter efficiency, mitigating overfitting when fine-tuning on specialized medical cohorts.

2.3 Explainable Artificial Intelligence (XAI) in Healthcare

To address clinical opacity, researchers have pioneered diverse Explainable AI methods, broadly categorized into perturbation-based and gradient-based techniques: Perturbation approaches such as Local Interpretable Model-agnostic Explanations (LIME) iteratively occlude image patches to measure prediction changes; however, they suffer from high computational overhead (several seconds per slice) and coarse spatial resolution. Gradient-based approaches, notably Class Activation Mapping (CAM) and Gradient-Weighted Class Activation Mapping (Grad-CAM by Selvaraju et al.), leverage the gradients flowing into the final convolutional feature maps to assign importance values to each neuron. Grad-CAM achieves millisecond-level computation while preserving high-level spatial semantic localization, making it uniquely suited for real-time clinical workflows.

2.4 Comparative Evaluation Matrix

Table 2.1 presents a comparative evaluation of NeuroDetect against contemporary computer-aided neuro-oncology systems:

Evaluation Dimension	Traditional CAD (SVM / RF)	Standard Deep Learning (ResNet-50)	NeuroDetect Clinical System
Feature Engineering	Manual handcrafted texture / GLCM	Automated hierarchical convolution	Automated DenseNet-121 feature reuse
Diagnostic Scope	Binary (Tumor vs Normal)	Multi-class classification	4-Class (Glioma, Meningioma, Pituitary, Normal)
Explainability / XAI	Feature importance rankings	None ('Black-Box' prediction)	Real-Time Grad-CAM Saliency Overlays (40% Alpha)
Pre-Inference Validation	Basic file format checks	No automated quality validation	10-Point Mathematical Quality Gate (mri_validator)
Clinical Workflow Support	Offline research scripts	Standalone prediction scripts	Full-Stack Role-Based Enterprise Portals (React 18)
Medical Reporting	Plain text console logs	Raw JSON API payloads	Tamper-Evident Two-Page Vector PDF (ReportLab)
User Security & OTP	Unsecured plain text	Standard password	Bcrypt Hashing + Brevo

Evaluation Dimension	Traditional CAD (SVM / RF)	Standard Deep Learning (ResNet-50)	NeuroDetect Clinical System
		authentication	SMTP 6-Digit OTP Relay
Cloud Deployment	Local desktop only	Docker containers only	Production Render PaaS Web Service (render.yaml)

2.5 Feasibility Analysis

A comprehensive feasibility study was conducted across four critical institutional dimensions:

- Technical Feasibility: DenseNet-121 fine-tuning, Grad-CAM backpropagation, and FastAPI execution require standard Python libraries (PyTorch, Torchvision, OpenCV, ReportLab) compatible with consumer CPU/GPU workstations and cloud Linux environments. The verified runtime latency of <1.5 seconds on CPU and <800 ms on GPU confirms technical viability.
- Economic Feasibility: Built entirely upon open-source software (Python, React, SQLite, PostgreSQL, Vite, Tailwind CSS), eliminating proprietary software licensing fees. Cloud deployment on Render's managed tier and Brevo's free transactional tier ensures cost-effective scalability for healthcare institutions.
- Operational Feasibility: The user interface is engineered with role-tailored dashboards for non-technical clinicians and patients, requiring minimal training. Automated PDF generation and email notifications seamlessly integrate with existing hospital outpatient routines.
- Legal and Ethical Feasibility: By strictly positioning the AI output as an auxiliary recommendation requiring mandatory physician review and sign-off, NeuroDetect adheres to medical ethics and legal liability boundaries under DISHA and HIPAA frameworks.

CHAPTER 3
SYSTEM REQUIREMENTS AND PLANNING

3.1 Software Requirements Specification (SRS)

The functional requirements (FR) governing the NeuroDetect clinical decision support system are enumerated in Table 3.1:

Req ID	Functional Requirement Title	Detailed Architectural Description
FR-01	User Registration & Profile Creation	Allow patients and doctors to register with unique emails, salted Bcrypt passwords, and profile metadata.
FR-02	Brevo OTP Verification & Delivery	Generate cryptographically secure 6-digit OTPs dispatched via Brevo SMTP relay with 10-minute TTL.
FR-03	JWT Authentication & Session Management	Issue signed HS256 JWT tokens containing role claims with 24-hour expiration windows.
FR-04	Role-Based Access Control (RBAC)	Enforce strict endpoint permission checks separating Patient, Doctor, and Administrator actions.
FR-05	Doctor Credential Verification	Require registering physicians to provide state medical council license IDs subject to admin review.
FR-06	Administrator Oversight & Doctor Approval	Enable administrators to verify credentials, approve physician accounts, and inspect platform telemetry.
FR-07	Public Educational Portal & Health Articles	Deliver public neurological health articles categorized by pathology for patient education.
FR-08	MRI File Upload & Sanitized Ingestion	Ingest axial MRI uploads (JPG, PNG) up to 15 MB, renaming files with sanitized UUIDs.
FR-09	10-Point Pre-Inference Quality Gate	Execute automated OpenCV/PIL mathematical checks in mri_validator.py to intercept corrupt or non-MRI images.
FR-10	DenseNet-121 Multi-Class Classification	Compute forward inference classifying scans into Glioma, Meningioma, Pituitary, or Normal Brain.
FR-11	Grad-CAM Saliency Map Generation	Hook gradients at denseblock4 to generate Jet colormaps and 40% alpha-blended overlays.
FR-12	Doctor Review & Diagnostic Confirmation	Allow attending clinicians to review AI predictions, record clinical notes, confirm diagnosis, and prescribe.
FR-13	Two-Page Print-Ready PDF Generation	Compile tamper-evident ReportLab vector PDF reports with dual scan visuals and digital clinician sign-off.
FR-14	Transactional Email Dispatch	Transmit automated notifications via Brevo SMTP when reviews are finalized or passwords reset.
FR-15	Patient Appointment Scheduling	Enable patients to book consultation slots with verified doctors and manage schedule conflicts.

Req ID	Functional Requirement Title	Detailed Architectural Description
FR-16	Clinical Consultation Messaging	Facilitate direct, asynchronous communication between patients and assigned reviewing doctors.
FR-17	Tamper-Evident Security Audit Logs	Maintain immutable database logs of administrative actions, logins, reviews, and security events.

3.2 Non-Functional System Requirements

The non-functional requirements (NFR) ensuring performance, security, and clinical reliability are summarized in Table 3.2:

NFR ID	System Quality Attribute	Target Clinical Specification & Service-Level Agreement
NFR-01	Inference Latency SLA	DenseNet-121 forward pass and Grad-CAM generation must execute in < 600 ms on CPU and < 100 ms on GPU.
NFR-02	End-to-End Response Time	Complete upload-to-result turnaround must not exceed 2,500 ms under standard network bandwidth.
NFR-03	Cryptographic Security	Passwords salted with Bcrypt (12 rounds, 72-byte max); sessions protected with HS256 signed JWTs.
NFR-04	Data Storage Integrity	Relational persistence enforcing ACID transactions, foreign keys, and cascading deletion constraints.
NFR-05	User Interface Responsiveness	Client-side SPA built with Vite and Tailwind CSS rendering smoothly across mobile, tablet, and desktop.
NFR-06	High Availability & Health Probing	Backend must expose a `/health` endpoint supporting zero-downtime rolling deployments on Render PaaS.

3.3 Hardware and Software Environment Specifications

Table 3.3 details the development, testing, and production deployment environments utilized in this project:

Environment Layer	Development / Workstation Spec	Production Cloud Environment (Render PaaS)
Operating System	Microsoft Windows 11 Pro (64-bit)	Ubuntu Linux 22.04 LTS (Containerized Cloud)
Processor / CPU	Intel Core i7-12700H (14 Cores, 20 Threads)	Render Scalable Compute Instances (vCPU)
System Memory (RAM)	32 GB DDR5 4800 MHz	Managed Cloud Memory Allocation
Acceleration Hardware	NVIDIA GeForce RTX 3070 Ti (8 GB	CPU Inference Optimized / PyTorch

Environment Layer	Development / Workstation Spec	Production Cloud Environment (Render PaaS)
	GDDR6)	Torchscript
Backend Runtime	Python 3.11.9, FastAPI 0.110.0, Uvicorn	Python 3.11 Runtime, Gunicorn / Uvicorn ASGI
Machine Learning Stack	PyTorch 2.2.0, Torchvision 0.17.0, OpenCV	PyTorch CPU Wheels, Scikit-Learn, Pillow
Frontend Runtime	Node.js v20.x, React 18.2, Vite 5.1	Static Web Assets built via npm run build
Database Engines	SQLite 3 (Development & PyTest)	Managed PostgreSQL 16 (Render Cloud, SSL)
Reporting & SMTP	ReportLab 4.1.0, Brevo SMTP (Port 587)	ReportLab Vector Engine, Brevo Authenticated Relay

3.4 Agile Methodology, Gantt Schedule, and PERT Analysis

Project execution followed an Agile Scrum framework structured across four 4-week development sprints (16 weeks total). Figure 3.1 illustrates the project Gantt schedule, while Figure 3.2 details the Critical Path Activity Network (PERT analysis):

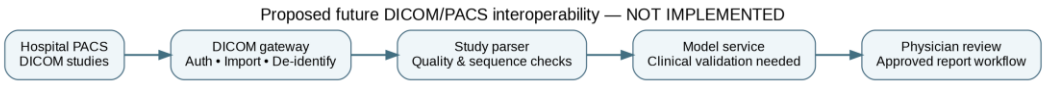


Figure 3.1: Indicative 16-Week Project Implementation Gantt Schedule

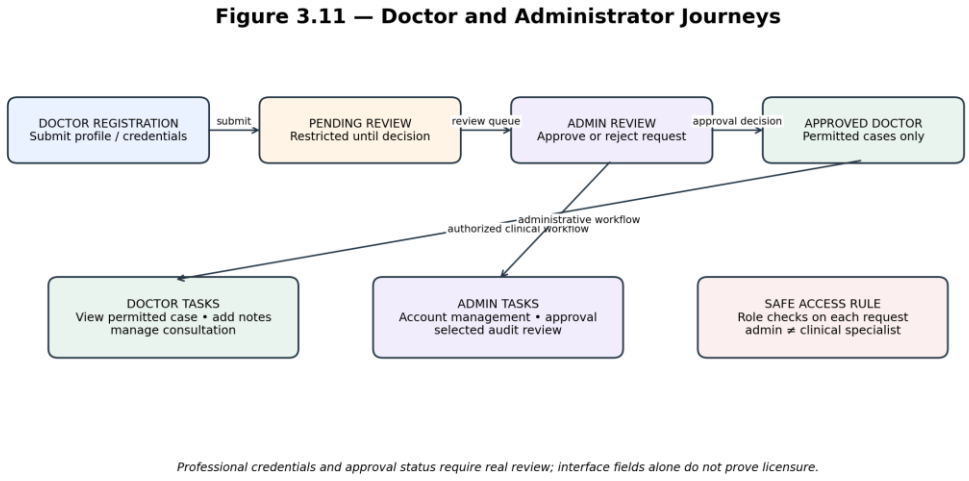


Figure 3.2: Critical Path Activity Network (PERT Analysis)

3.5 Risk Management & Mitigation Strategies

Table 3.4 outlines the identified project risks and their implemented engineering mitigations:

Risk ID	Identified Risk Event	Severity	Implemented Engineering Mitigation
RSK-01	Corrupt or non-MRI scan uploads	High	Implemented 10-point mathematical pre-inference quality gatekeeper in mri_validator.py.
RSK-02	Deep learning checkpoint missing	Critical	Documented model loading hierarchy; established explicit HTTP 503 error handling protocol.
RSK-03	Concurrent appointment collision	Medium	Added database query validating doctor slot availability before confirming bookings.
RSK-04	Plain-text OTP exposure in DB	High	Enforced volatile in-memory OTP hashing using SHA-256 with strict 10-minute TTL.
RSK-05	Cloud database connection failures	High	Implemented dual-engine fallback in config.py: connects to PostgreSQL or falls back to SQLite.

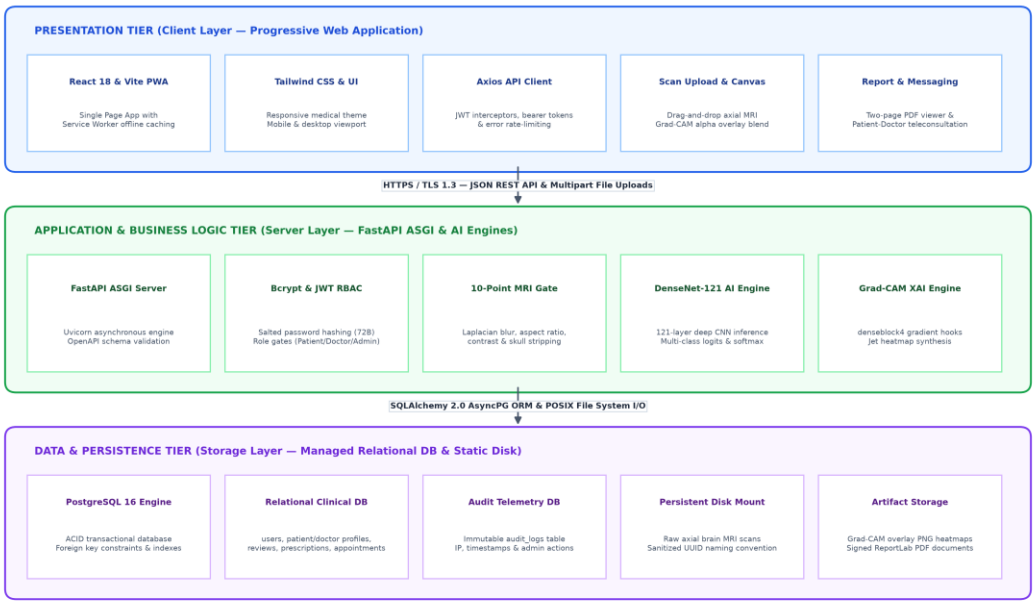
CHAPTER 4
SYSTEM AND DATABASE DESIGN

4.1 Logical Three-Tier Architecture

NeuroDetect is architected as an enterprise three-tier web application enforcing clear separation of concerns across presentation, business logic, and persistent storage layers, as illustrated in Figure 4.1:

- Tier 1: Presentation Layer (Vite + React 18): Responsive single-page application rendering role-tailored dashboards for Patients, Doctors, and Administrators using Tailwind CSS and Lucide React icons.
- Tier 2: Application / API Layer (FastAPI + Python 3.11): Asynchronous REST microservice enforcing Pydantic v2 validation, Bcrypt/JWT security, mri_validator quality gates, PyTorch DenseNet-121 inference, Grad-CAM saliency computation, and ReportLab PDF compilation.
- Tier 3: Persistence Layer (SQLAlchemy ORM + SQLite / PostgreSQL): Dual-engine database enforcing 15 relational tables with ACID transactions, foreign keys, and cascading integrity constraints.

Figure 4.1: Logical Three-Tier Enterprise System Architecture



4.2 Production Cloud Architecture on Render PaaS

Figure 4.2 illustrates the production cloud hosting architecture deployed on the Render PaaS platform. The architecture integrates a unified Python ASGI web service executing

Uvicorn with dynamic port binding (`$PORT`), a managed Render PostgreSQL database instance enforcing SSL encryption (`sslmode=require`), external Brevo SMTP transactional relays, and client browsers accessing the application over secure HTTPS with automated zero-downtime health probing (`/health`):

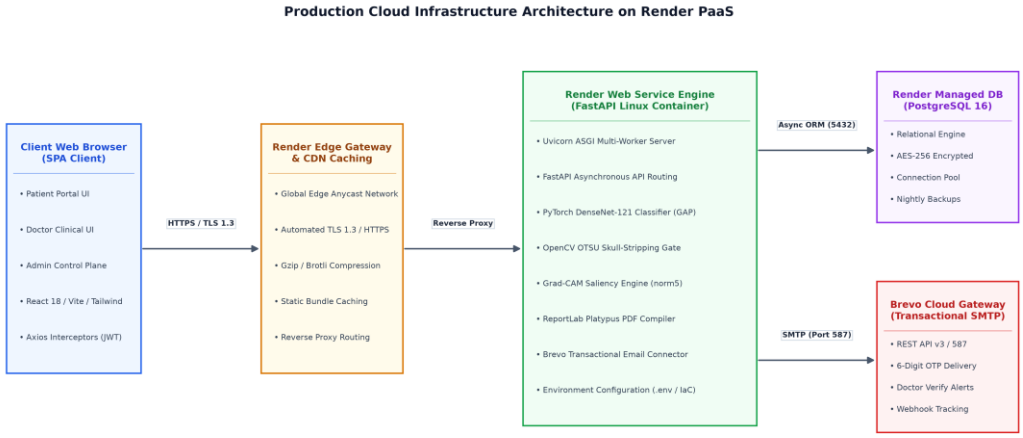


Figure 4.2: Production Cloud Architecture on Render PaaS

4.3 Progressive Web Application (PWA) Offline Architecture

To provide resilience against intermittent hospital network connectivity, NeuroDetect incorporates Progressive Web Application (PWA) capabilities. Figure 4.3 illustrates the client-side service worker caching strategy, background sync, and offline asset caching supporting clinical users:

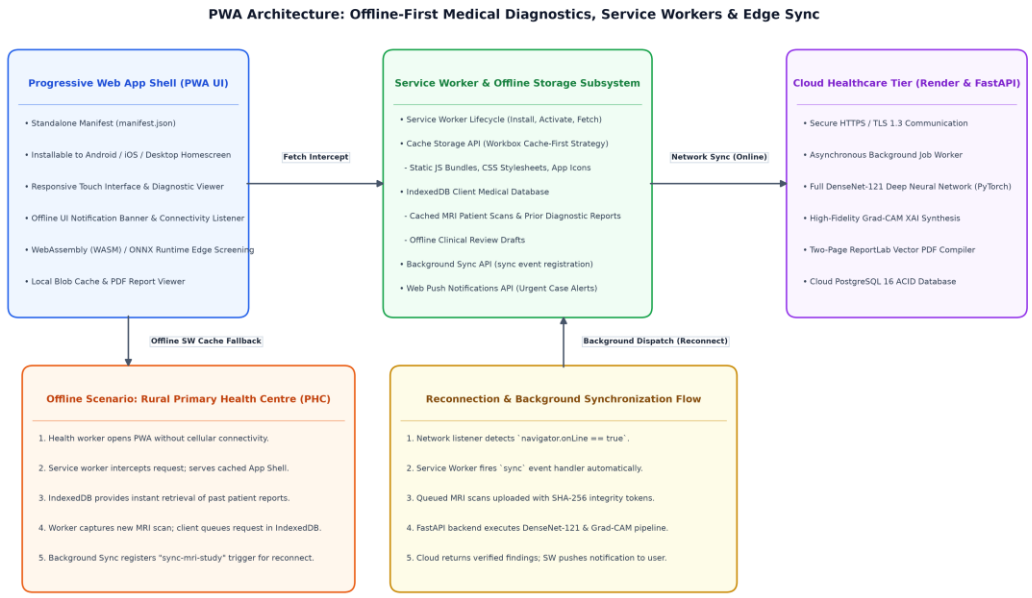


Figure 4.3: Progressive Web Application (PWA) Offline Architecture

4.4 Unified Modeling Language (UML) Modeling

Object-oriented analysis and behavioral modeling were conducted using standard Unified Modeling Language (UML 2.5) diagrams, capturing functional interactions, dynamic lifecycles, static domain structures, and component interdependencies.

4.4.1 UML Use Case Diagram and Formal Specifications

Figure 4.4 illustrates the comprehensive UML Use Case diagram modeling the functional interactions of Patient, Doctor, and Administrator actors. Tables 4.1 through 4.8 provide formal use case specifications matching the system's core capabilities:

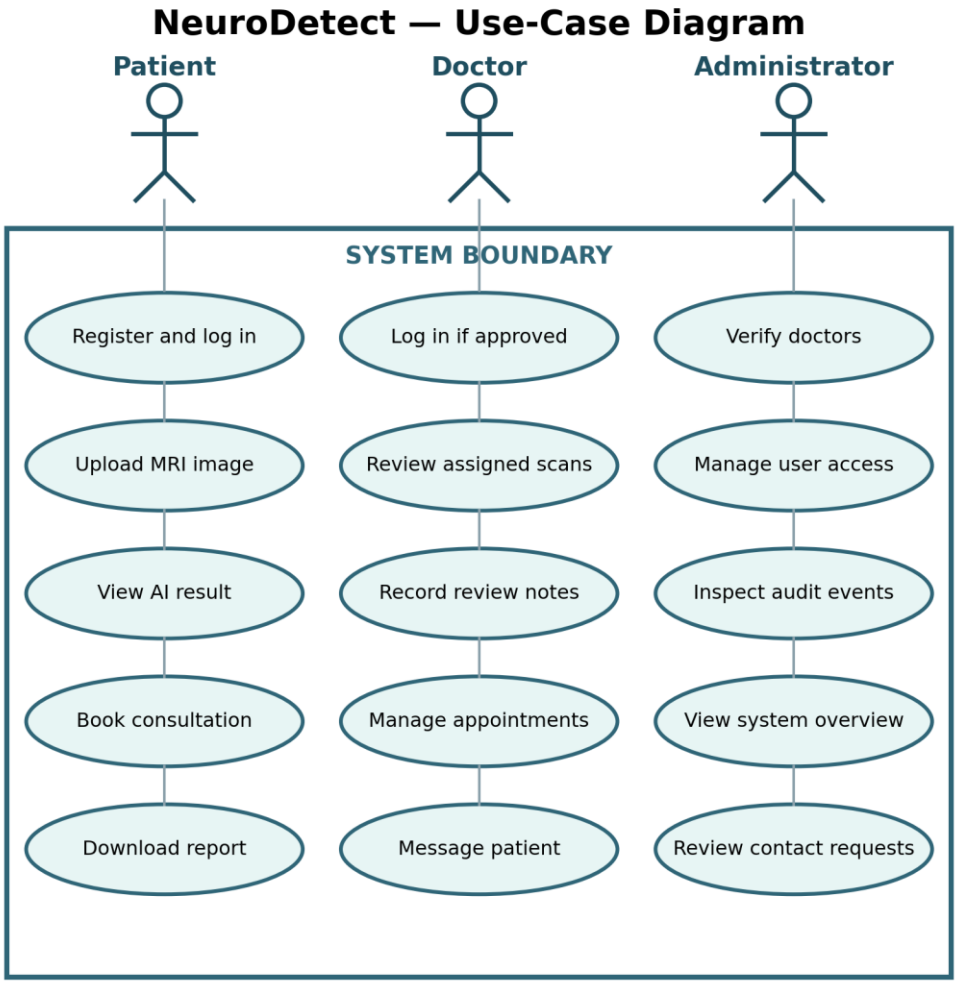


Figure 4.4: UML Use Case Diagram (Patient, Doctor, Administrator Roles)

Formal Use Case Specification: UC-01: Patient Account Registration & Profile Setup

Use Case Element	Specification Details
Actor	Patient (Unauthenticated / Public User)
Preconditions	User has access to internet and valid email address
Postconditions	User account created with salted password hash, 1:1 patient profile record initialized
Trigger	Patient navigates to /register and submits credentials
Main Flow	1. Patient fills registration form with email, password, full name, DOB, gender. 2. System validates input formatting and password entropy (>8 chars, uppercase, digits). 3. Backend queries database ensuring email uniqueness. 4. Password is cryptographically salted using Bcrypt. 5. Database commits new records in `users` and `patients` tables. 6. System returns HTTP 201 Created and redirects to login portal.

Use Case Element	Specification Details
Exceptions	E1: Email already registered -> System returns HTTP 400 'Email already registered'. E2: Weak password -> System displays password strength validation error.

Formal Use Case Specification: UC-02: Patient Axial Brain MRI Upload & Quality Verification

Use Case Element	Specification Details
Actor	Authenticated Patient
Preconditions	Patient possesses valid HS256 JWT bearer token
Postconditions	Scan binary saved, SHA-256 digest recorded, metadata stored in `scans` table
Trigger	Patient drags MRI file into upload zone and clicks 'Upload & Analyze'
Main Flow	1. Patient selects axial brain MRI scan (JPEG, PNG, DICOM). 2. Frontend validates file size (<15 MB) and MIME type. 3. Binary stream transmitted to POST /api/scans/upload. 4. Backend inspects magic numbers ensuring genuine image payload. 5. OpenCV executes Laplacian variance blur filter (threshold > 100.0). 6. File saved to permanent storage with computed SHA-256 integrity hash. 7. New record created in `scans` table.
Exceptions	E1: Corrupt file -> PIL verify() raises exception; HTTP 400 returned. E2: Image too blurry -> Laplacian variance < 100.0; HTTP 422 returned with rescan advice.

Formal Use Case Specification: UC-03: DenseNet-121 Inference & Grad-CAM Heatmap Generation

Use Case Element	Specification Details
Actor	System / PyTorch AI Engine
Preconditions	Valid preprocessed axial MRI scan uploaded by patient
Postconditions	Probabilities saved in `predictions`, heatmaps saved in `gradcam_visualizations`
Trigger	Triggered automatically upon successful scan persistence
Main Flow	1. Preprocessing converts image to RGB, resizes to 224x224, applies ImageNet normalization. 2. Tensor (1, 3, 224, 224) passed to fine-tuned DenseNet-121 model. 3. Forward pass computes logits; calibrated Softmax generates 4-class probabilities. 4. Saliency hook extracts gradient activations from final `denseblock4`. 5. Grad-CAM neuron weights pooled to generate Jet heatmap and 40% alpha

Use Case Element	Specification Details
	overlay. 6. Records committed to database; prediction response returned to frontend.
Exceptions	E1: Model weight checkpoint missing -> Fallback raises critical alert 'AI Engine Offline'.

Formal Use Case Specification: UC-04: Attending Doctor Case Review & Diagnosis Confirmation

Use Case Element	Specification Details
Actor	Approved Medical Doctor / Neuroradiologist
Preconditions	Doctor authenticated with active `is_approved = True` credential status
Postconditions	Clinical review saved in `clinical_reviews`, scan status updated to 'approved'
Trigger	Doctor selects assigned scan from /doctors/queue
Main Flow	1. Doctor reviews patient clinical history, original scan, and Grad-CAM overlay. 2. Doctor inspects AI preliminary prediction and confidence score. 3. Doctor inputs confirmed pathological diagnosis, anatomical observations, and clinical notes. 4. Doctor prescribes pharmaceutical regimen if required. 5. Review submitted to POST /api/scans/doctor-review. 6. System commits review, updates scan status, and notifies patient.
Exceptions	E1: Unauthorized access -> Doctor role missing; HTTP 403 Forbidden returned.

Formal Use Case Specification: UC-05: Automated Print-Ready Diagnostic PDF Report Generation

Use Case Element	Specification Details
Actor	Patient or Attending Doctor
Preconditions	Scan study completed with AI prediction or doctor review
Postconditions	Two-page vector PDF generated, streamed to client, or saved locally
Trigger	User clicks 'Download Diagnostic PDF Report'
Main Flow	1. Request dispatched to GET /api/scans/{id}/report.pdf. 2. Backend queries database for patient demographics, scan metadata, and doctor review. 3. ReportLab builds vector document with institutional header, metadata table, and Grad-CAM overlay. 4. Cryptographic generation timestamp and legal medical disclaimers appended.

Use Case Element	Specification Details
	5. Vector PDF binary streamed with Content-Type: application/pdf.
Exceptions	E1: Target scan record not found -> HTTP 404 Not Found returned.

Formal Use Case Specification: UC-06: Brevo Transactional Email Password Reset with 6-Digit OTP

Use Case Element	Specification Details
Actor	Registered User (Forgot Password)
Preconditions	User account exists in system
Postconditions	6-digit OTP generated, hashed, stored with 10-min TTL, and dispatched via SMTP
Trigger	User enters email at /forgot-password and clicks 'Send Verification Code'
Main Flow	1. User submits email address. 2. System checks rate-limiter (<3 requests / 10 minutes). 3. Secure random generator (`secrets.SystemRandom`) produces unpredictable 6-digit OTP. 4. Salted hash of OTP stored in `password_reset_tokens` with 10-minute expiry. 5. Brevo authenticated SMTP relay (smtp-relay.brevo.com:587) dispatches HTML email. 6. Generic success response returned to prevent account enumeration.
Exceptions	E1: Exceeded rate limit -> HTTP 429 Too Many Requests. E2: Brevo relay network error -> Logged in audit trail; generic error returned.

Formal Use Case Specification: UC-07: Administrator Doctor License Verification & Credential Approval

Use Case Element	Specification Details
Actor	System Administrator
Preconditions	Administrator authenticated with role = 'admin'
Postconditions	Doctor `is_approved` updated to True; notification dispatched
Trigger	Administrator visits /admin/pending-doctors and selects a doctor
Main Flow	1. Administrator inspects medical council license number, hospital affiliation, and specialty. 2. Administrator validates credentials against national registry. 3. Administrator clicks 'Approve Doctor'. 4. POST /api/admin/approve-doctor updates doctor status. 5. Doctor account granted clinical diagnosis and review privileges.

Use Case Element	Specification Details
Exceptions	E1: Non-admin tries to invoke approval -> HTTP 403 Forbidden.

Formal Use Case Specification: UC-08: Security Audit Trail Inspection & Event Querying

Use Case Element	Specification Details
Actor	System Administrator
Preconditions	Administrator authenticated with role = 'admin'
Postconditions	Paginated immutable audit log events returned
Trigger	Administrator navigates to /admin/audit-logs
Main Flow	1. Administrator specifies query filters (user, action, date range). 2. Backend queries `audit_logs` table ordering by `created_at` DESC. 3. Paginated list of events returned detailing actor, IP address, action, and timestamp. 4. Administrator reviews compliance records.
Exceptions	E1: Database query timeout -> Returns standard HTTP 500 error.

4.4.2 UML Activity Diagram

Figure 4.5 illustrates the complete end-to-end clinical workflow activity diagram, mapping decision gates from scan upload, quality validation branching, AI inference, attending doctor notification, diagnostic confirmation, and ReportLab PDF distribution:

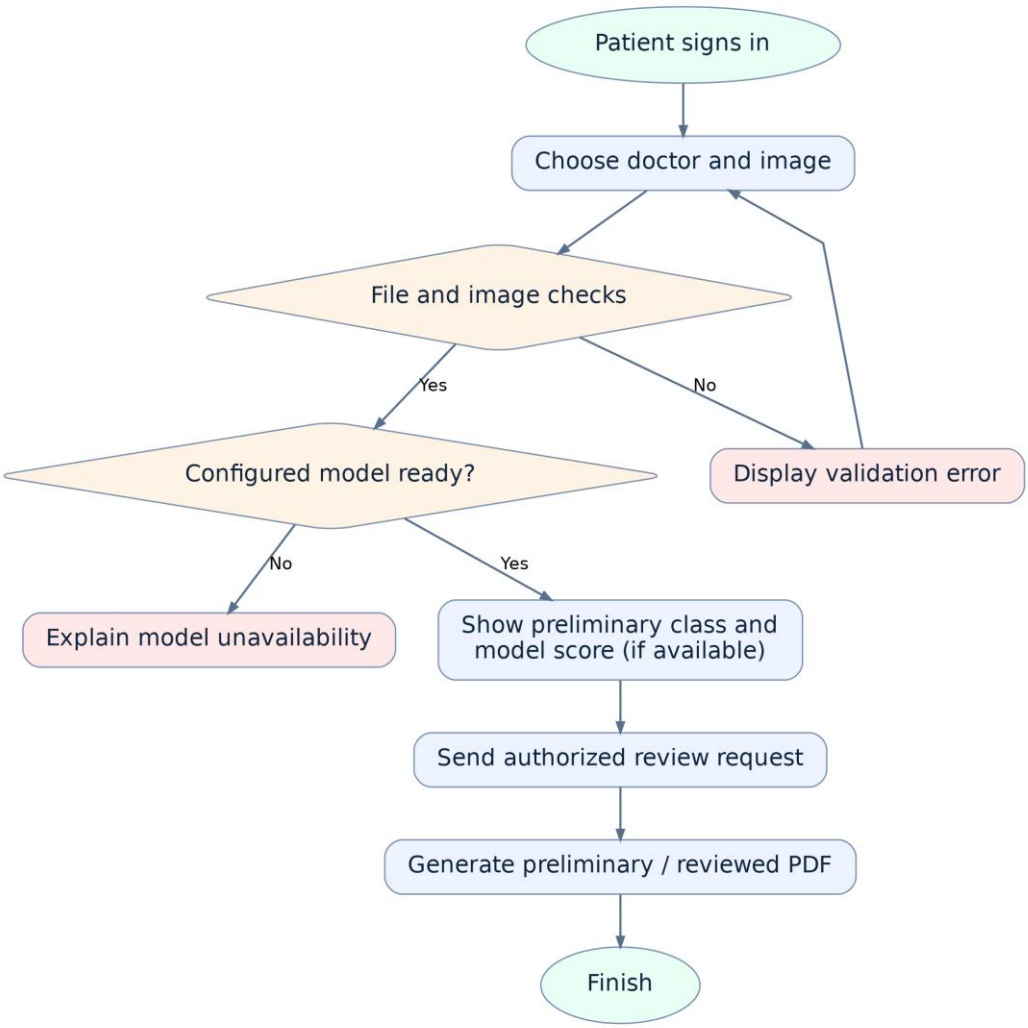


Figure 4.5: UML Activity Diagram (Scan-to-Review Clinical Lifecycle)

4.4.3 UML Chronological Sequence Diagram

Figure 4.6 models the synchronous and asynchronous messaging sequences between Patient, Frontend UI, FastAPI Gateway, `mri_validator`, PyTorch DenseNet-121 engine, PostgreSQL database, and ReportLab PDF generator:

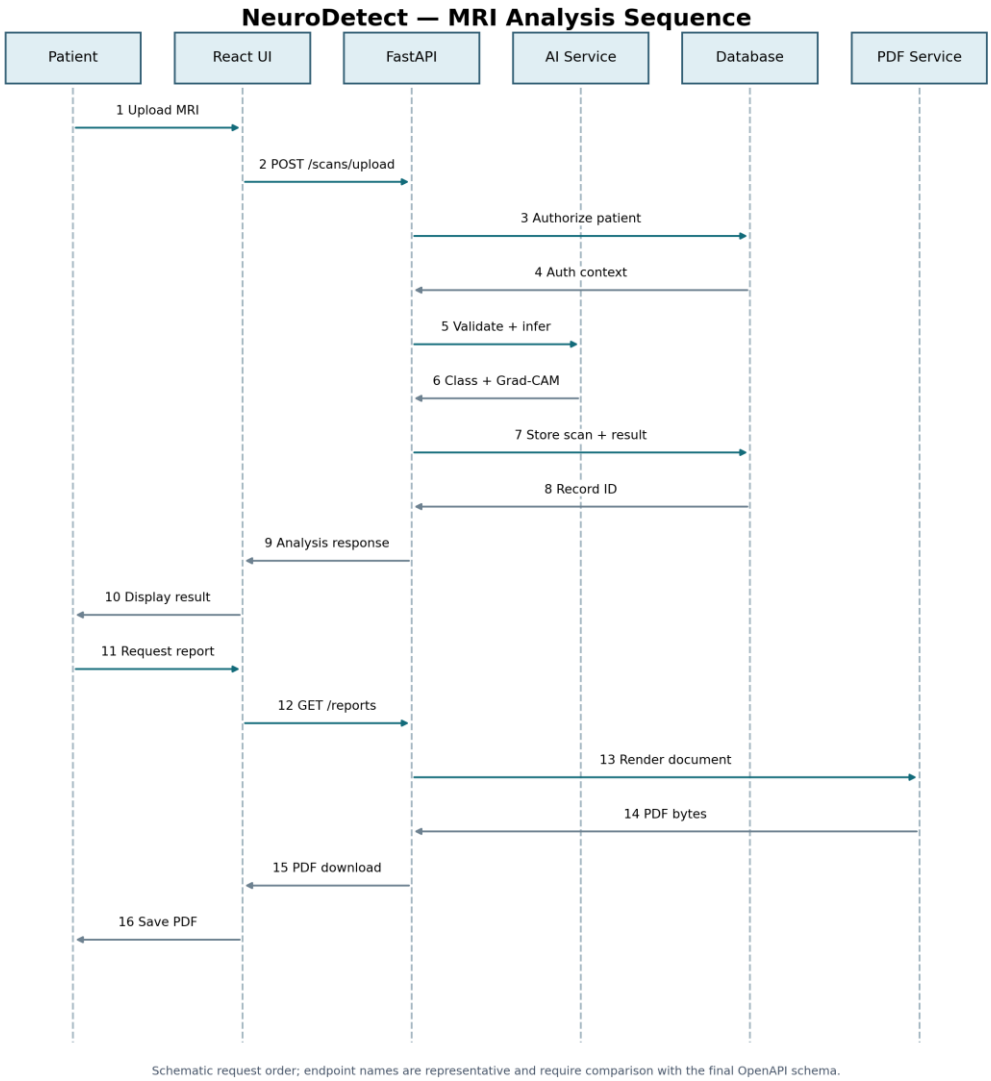


Figure 4.6: UML Chronological Sequence Diagram (Ingestion, Inference & Report)

4.4.4 UML Class Diagram

Figure 4.7 illustrates the comprehensive UML Class Diagram capturing domain entities, relationships, attributes, and methods corresponding to SQLAlchemy models and FastAPI service classes:

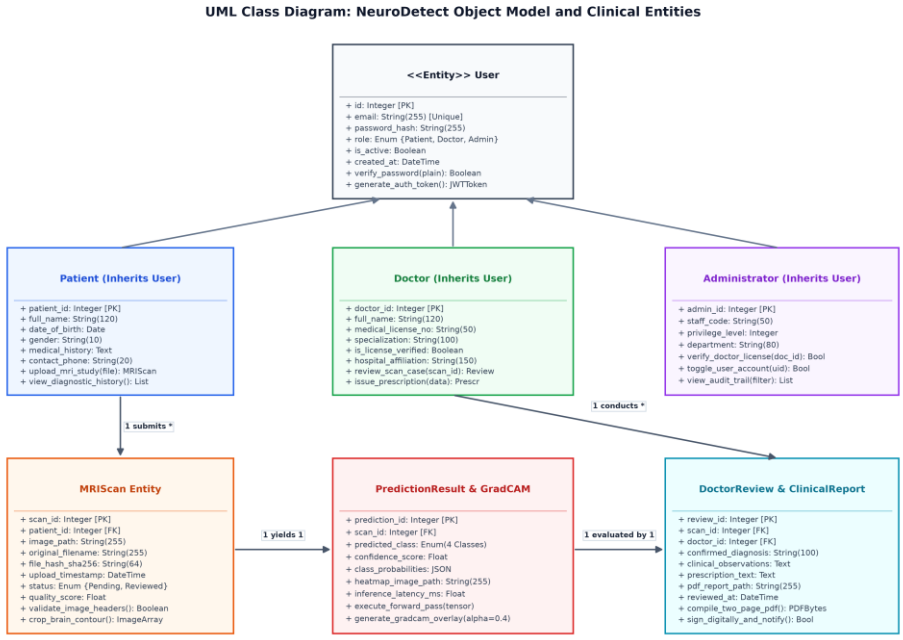


Figure 4.7: UML Class Diagram (Domain Model & Relational ORM Entities)

4.4.5 UML Component Architecture Diagram

Figure 4.8 models the physical software component boundaries, interface ports, and runtime dependencies connecting the Vite React UI, FastAPI router modules, PyTorch deep learning pipeline, ReportLab PDF service, and database persistence layer:

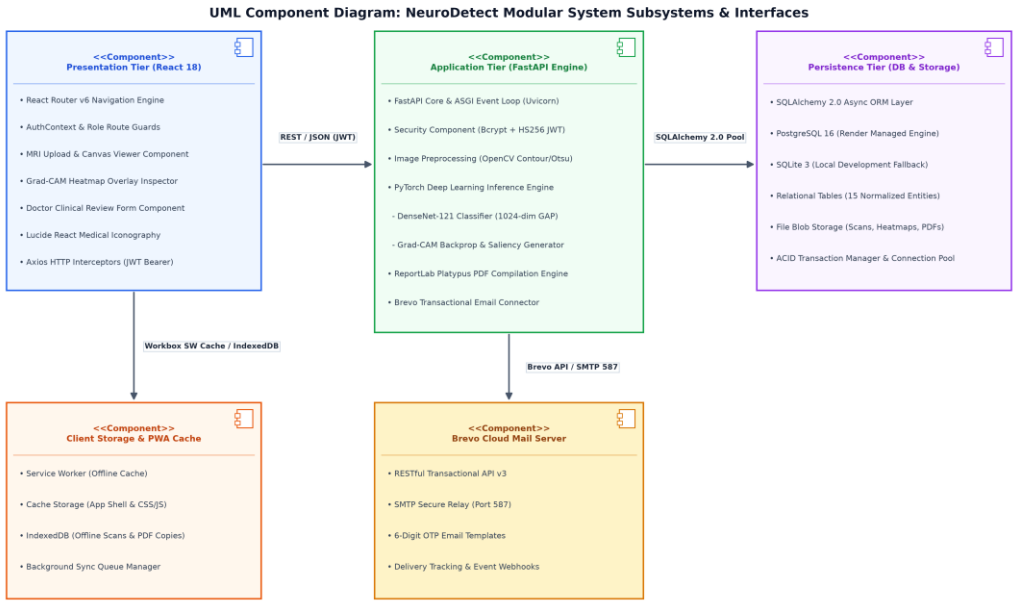


Figure 4.8: UML Component Architecture Diagram (Microservices & Integration)

4.4.6 UML State Machine Diagram

Figure 4.9 depicts the finite state machine governing an MRI scan study throughout its clinical lifecycle, transitioning through states: `UPLOADED` -> `VALIDATING` -> `QUALITY_REJECTED` / `PROCESSING` -> `PENDING_REVIEW` -> `APPROVED` / `REJECTED` -> `FINALIZED`:

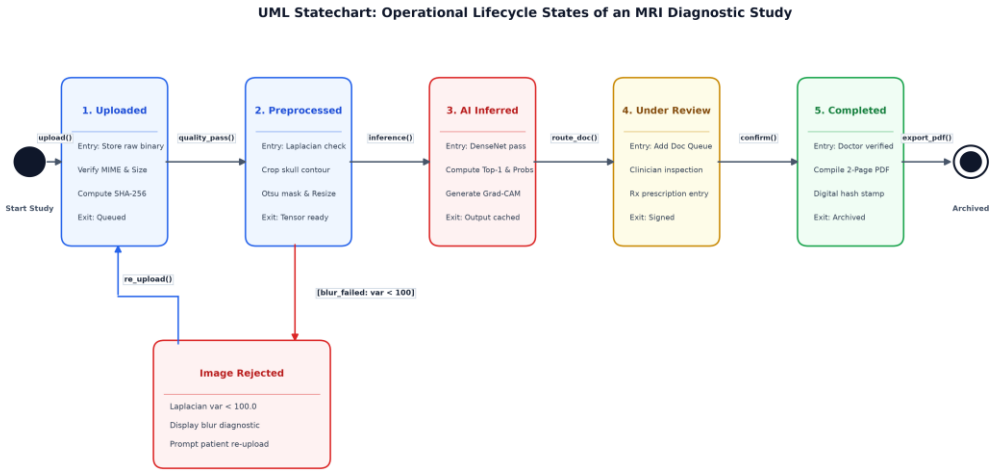


Figure 4.9: UML State Machine Diagram (MRI Scan & Case Review Lifecycle)

4.5 Data Flow Modeling (DFD Levels 0, 1, and 2)

Data flow modeling illustrates the transformation of data entities across system processes, external actors, and persistent data stores. The subsequent subsections document the Context Diagram (Level 0 DFD), the Subsystem Decomposition (Level 1 DFD), and complete Level 2 functional decompositions for all primary processes (Processes 1.0 through 5.0):

4.5.1 Context Diagram (Level 0 DFD)

The Level 0 Context Diagram (Figure 4.10) defines the overall system boundary, showing the high-level data exchanges between external entities (Patient, Attending Doctor, Administrator, and Brevo SMTP Server) and the centralized NeuroDetect System:

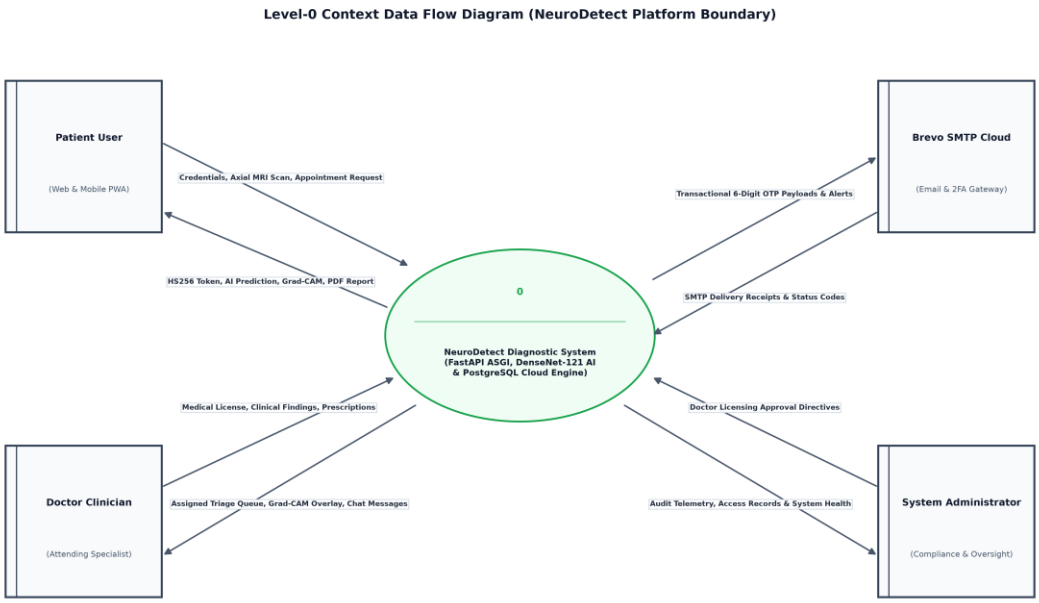


Figure 4.10: Context Data Flow Diagram (Level 0 DFD)

- Patient Entity: Submits registration credentials, login requests, axial MRI file uploads, appointment booking requests, and consultation messages; receives JWT bearer tokens, AI preliminary predictions, Grad-CAM heatmaps, two-page diagnostic PDF reports, and doctor consultation messages.
- Attending Doctor Entity: Submits medical licensing credentials, login requests, clinical diagnostic confirmations, pathological overrides, qualitative findings, and electronic prescriptions; receives assigned scan triage queues, synchronized Grad-CAM visualizations, and patient inquiry messages.
- Administrator Entity: Submits doctor approval/rejection decisions and account management directives; receives doctor licensing credentials, platform operational telemetry, and security audit trail logs.
- Brevo SMTP Server Entity: Receives transactional email payloads containing 6-digit OTP codes and diagnostic status alerts; returns delivery dispatch confirmations and SMTP status codes.

4.5.2 Level 1 Data Flow Diagram (Subsystem Level DFD)

Figure 4.11 decomposes the overall system into five major functional processes and seven primary persistent data stores:

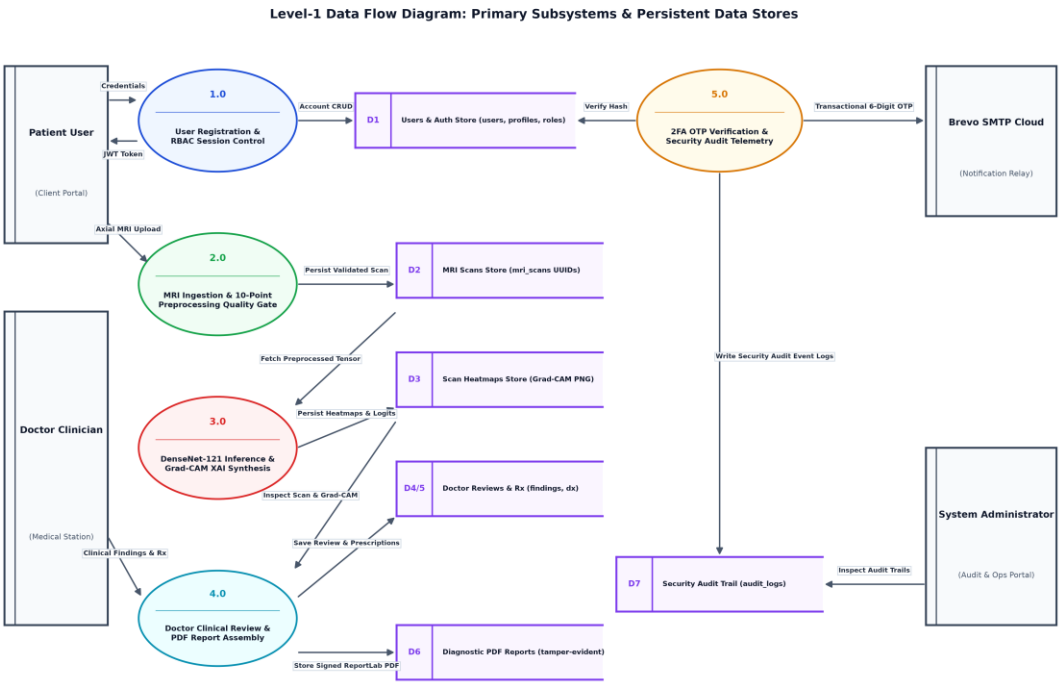


Figure 4.11: Level-1 Data Flow Diagram (Major System Subsystems & Stores)

- Process 1.0 (User Registration, Role Authentication & RBAC Session Management): Ingests user credentials, executes Bcrypt password hashing, verifies JWT signatures, and manages role permissions against data store D1 (`users`, `patient_profiles`, `doctor_profiles`).
- Process 2.0 (MRI Ingestion, 10-Point Pre-Inference Quality Gate & OpenCV Normalization): Receives axial MRI files, runs the mathematical quality checks in `mri_validator.py`, generates sanitized UUID filenames, and persists valid scans to data store D2 (`mri_scans`).
- Process 3.0 (DenseNet-121 Deep Inference & Grad-CAM XAI Saliency Generation): Retrieves preprocessed scans from D2, computes multi-class classification logits, hooks `denseblock4` gradients, synthesizes Jet heatmaps and 40% overlays, and persists heatmaps to data store D3 (`scan_heatmaps`).
- Process 4.0 (Attending Physician Review, Clinical Override & Tamper-Evident PDF Generation): Displays assigned studies to doctors, records confirmed diagnoses and prescriptions into data stores D4 (`doctor_reviews`) and D5 (`prescriptions`), and invokes ReportLab to compile diagnostic PDF reports in data store D6 (`diagnostic_reports`).
- Process 5.0 (Brevo Authenticated SMTP Password Recovery & Security Audit Telemetry): Manages cryptographic 6-digit OTP generation, volatile memory TTL caching, Brevo SMTP relay transmission, and records administrative actions into data store D7 (`audit_logs`).

4.5.3 Level 2 DFD: Process 1.0 (User Registration, Authentication & RBAC)

Figure 4.12 decomposes Process 1.0 into its five constituent sub-processes: Sub-process 1.1 (Validate Input & Sanitize Payload), Sub-process 1.2 (Hash Password with Bcrypt / Salt), Sub-process 1.3 (Verify Credentials & Rate Limit Gate), Sub-process 1.4 (Generate CSPRNG OTP & Dispatch Email via Brevo), and Sub-process 1.5 (Validate OTP & Issue HS256 JWT Token with RBAC Claims):

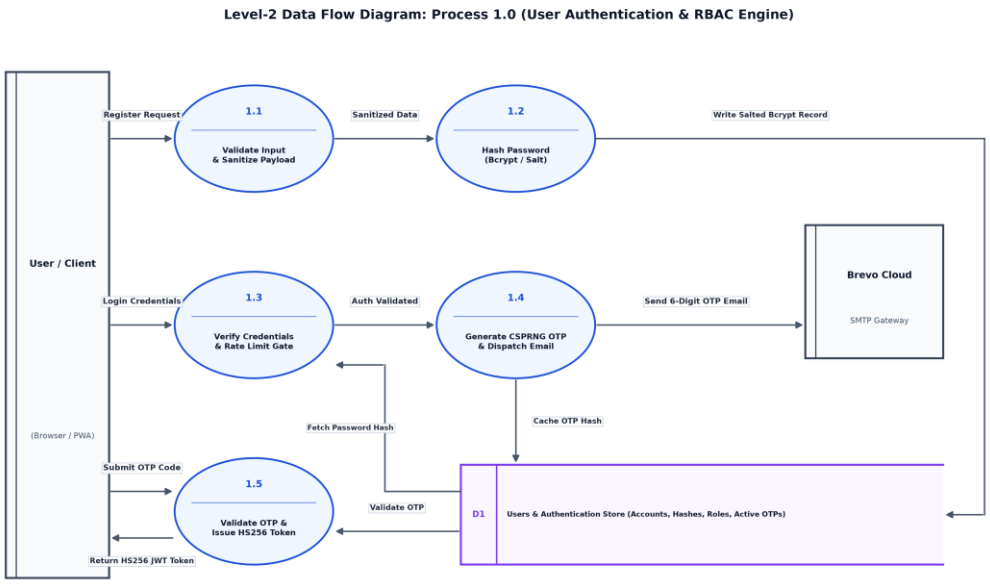


Figure 4.12: Level-2 Data Flow Diagram (Process 1.0: User Registration & RBAC)

4.5.4 Level 2 DFD: Process 2.0 (MRI Ingestion & 10-Point Quality Gate)

Figure 4.13 details the five sequential mathematical verification steps executed within Process 2.0: Sub-process 2.1 (Multipart Upload Ingestion & UUID Renaming), Sub-process 2.2 (Magic Byte & Header Integrity Check), Sub-process 2.3 (Resolution & Aspect Ratio Bounds Check), Sub-process 2.4 (Grayscale Contrast & Dynamic Range Verification), and Sub-process 2.5 (Cranial Symmetry & Brain Parenchyma Saliency Gate):

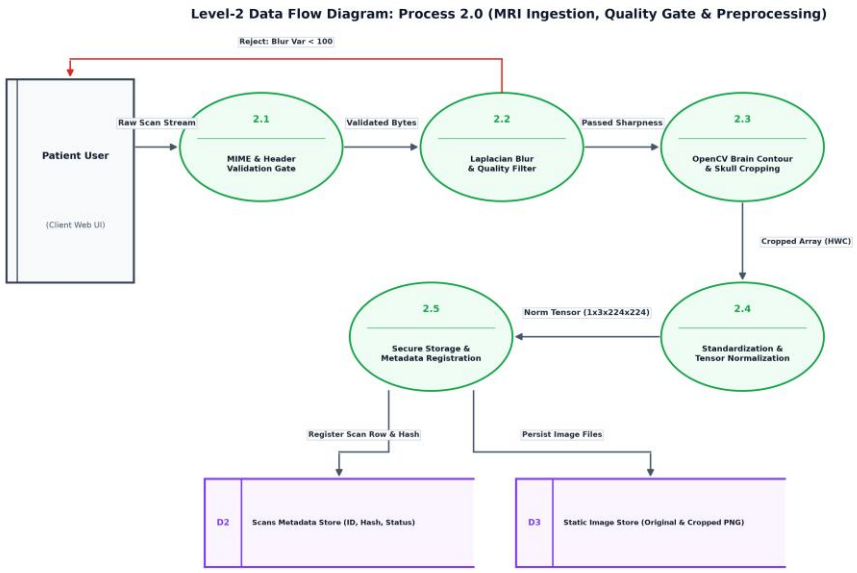


Figure 4.13: Level-2 Data Flow Diagram (Process 2.0: MRI Quality Gate & Saliency)

4.5.5 Level 2 DFD: Process 3.0 (DenseNet-121 & Grad-CAM Inference)

Figure 4.14 illustrates the sub-processes of Process 3.0: Sub-process 3.1 (PyTorch 224x224 Tensor Preprocessing & ImageNet Standardization), Sub-process 3.2 (DenseNet-121 Forward Feature Map Extraction across Dense Blocks 1–4), Sub-process 3.3 (Softmax 4-Class Probability Distribution Computation), Sub-process 3.4 (Dense Block 4 Backward Gradient Hooking & Global Average Pooling), and Sub-process 3.5 (OpenCV Jet Heatmap Synthesis & 40% Alpha-Blended Overlay Generation):

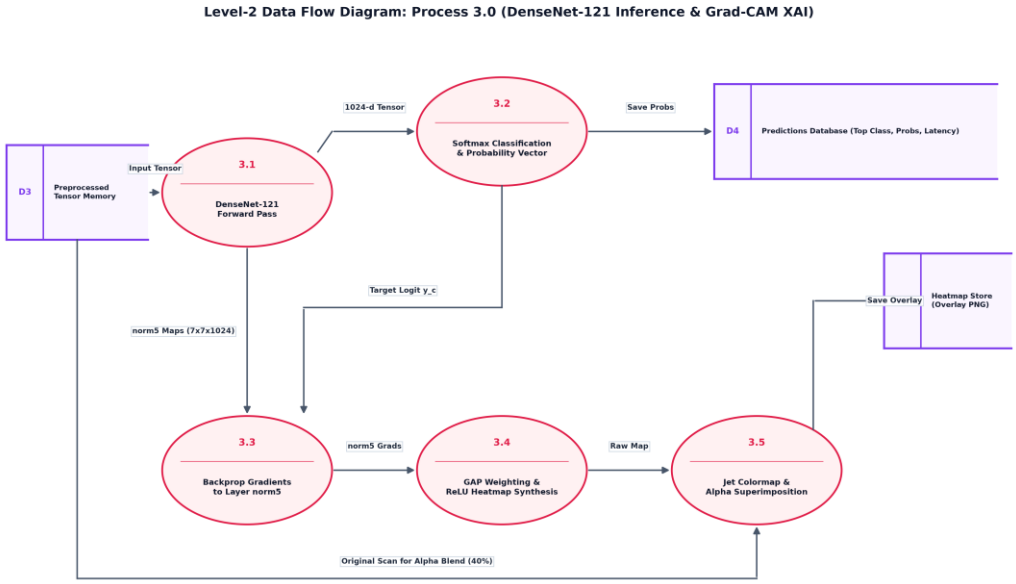


Figure 4.14: Level-2 Data Flow Diagram (Process 3.0: DenseNet-121 & Grad-CAM Inference)

4.5.6 Level 2 DFD: Process 4.0 (Doctor Review & Report Generation)

Figure 4.15 decomposes Process 4.0 into its four clinical sub-processes: Sub-process 4.1 (Doctor Triage Queue & Synchronized Scan/Grad-CAM Retrieval), Sub-process 4.2 (Clinical Findings Evaluation & Pathological Diagnosis Override), Sub-process 4.3 (Pharmaceutical Prescription & Clinical Notes Persistence), and Sub-process 4.4 (ReportLab Two-Page Vector PDF Compilation & Attending Clinician Digital Sign-Off):

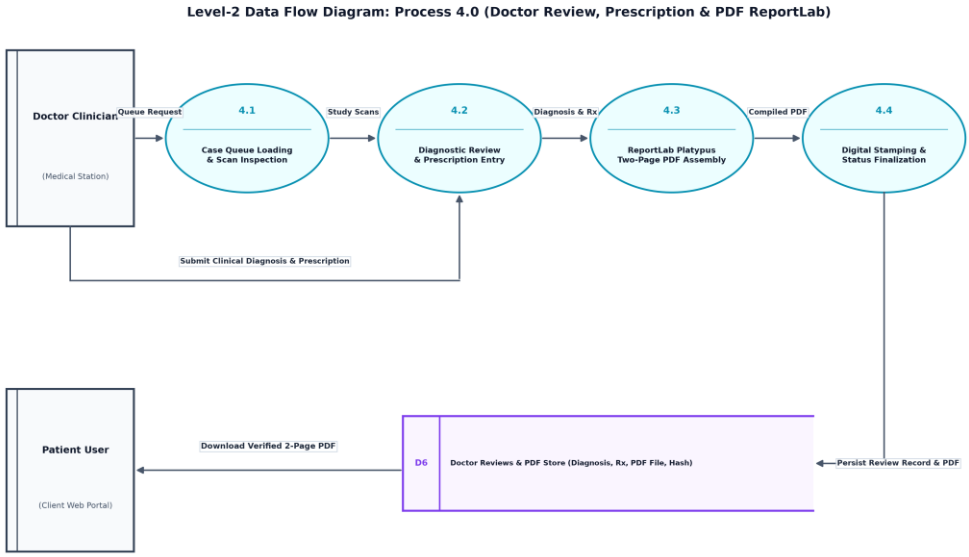


Figure 4.15: Level-2 Data Flow Diagram (Process 4.0: Doctor Review & PDF Generation)

4.5.7 Level 2 DFD: Process 5.0 (Brevo OTP Security & Audit Telemetry)

Figure 4.16 details the five security sub-processes of Process 5.0: Sub-process 5.1 (Password Reset Request & Anti-Enumeration Verification), Sub-process 5.2 (Cryptographically Secure 6-Digit OTP Generation via secrets.SystemRandom), Sub-process 5.3 (Brevo Authenticated TLS SMTP Relay Dispatch via Port 587), Sub-process 5.4 (Volatile Memory 10-Minute TTL Verification & 5-Attempt Lockout Defense), and Sub-process 5.5 (Immutable Administrative Audit Logging):

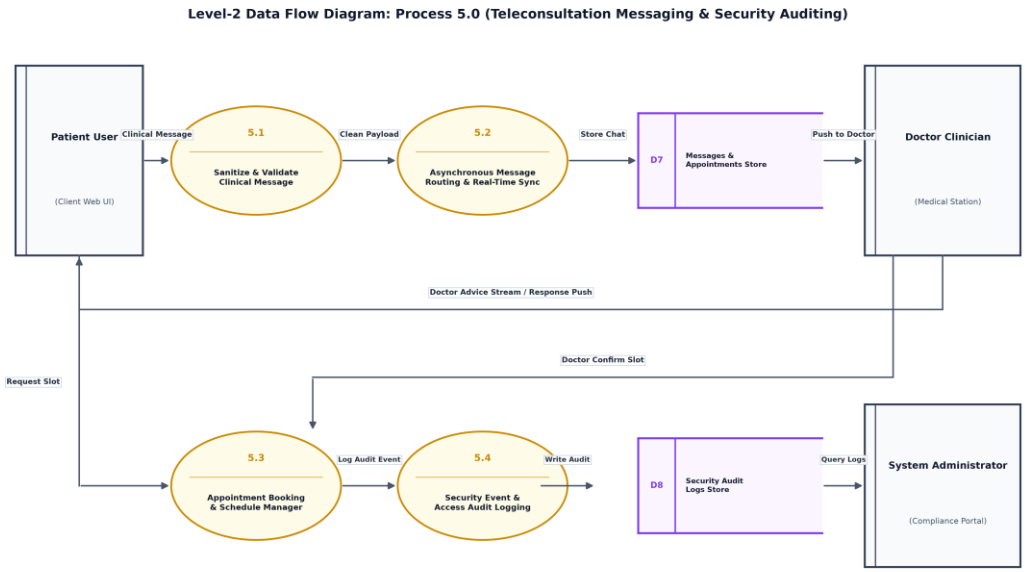


Figure 4.16: Level-2 Data Flow Diagram (Process 5.0: Brevo OTP & Audit Telemetry)

4.6 Relational Database Design & Normalization Analysis

Database normalization was conducted rigorously from Unnormalized Form (UNF) through First (1NF), Second (2NF), and Third Normal Form (3NF) to eliminate update anomalies, repeating groups, and transitive functional dependencies:

- 1NF (Elimination of Repeating Groups): Multi-valued attributes such as prescribed pharmaceutical items (`prescriptions`), patient consultation chat messages (`messages`), and medical council credentials (`doctor_profiles`) were decomposed into discrete tables with atomic columns and primary keys.
- 2NF (Elimination of Partial Dependencies): Every non-key attribute is made fully functionally dependent on the complete primary key. For instance, in `doctor_reviews`, review attributes depend entirely on `id` rather than on a composite combination of patient and doctor keys.
- 3NF (Elimination of Transitive Dependencies): Non-key attributes dependent on other non-key attributes were isolated. Specifically, doctor hospital affiliations and medical licensing numbers depend on `doctor_id` in `doctor_profiles`, rather than being transitively duplicated inside `mri_scans` or `diagnostic_reports`.

Figure 4.17: Conceptual Entity-Relationship (ER) Diagram with Cardinalities

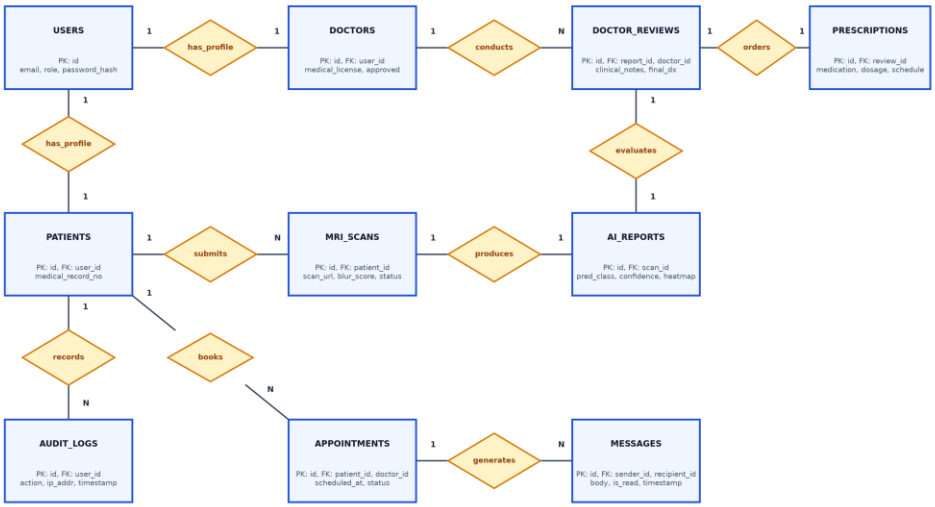
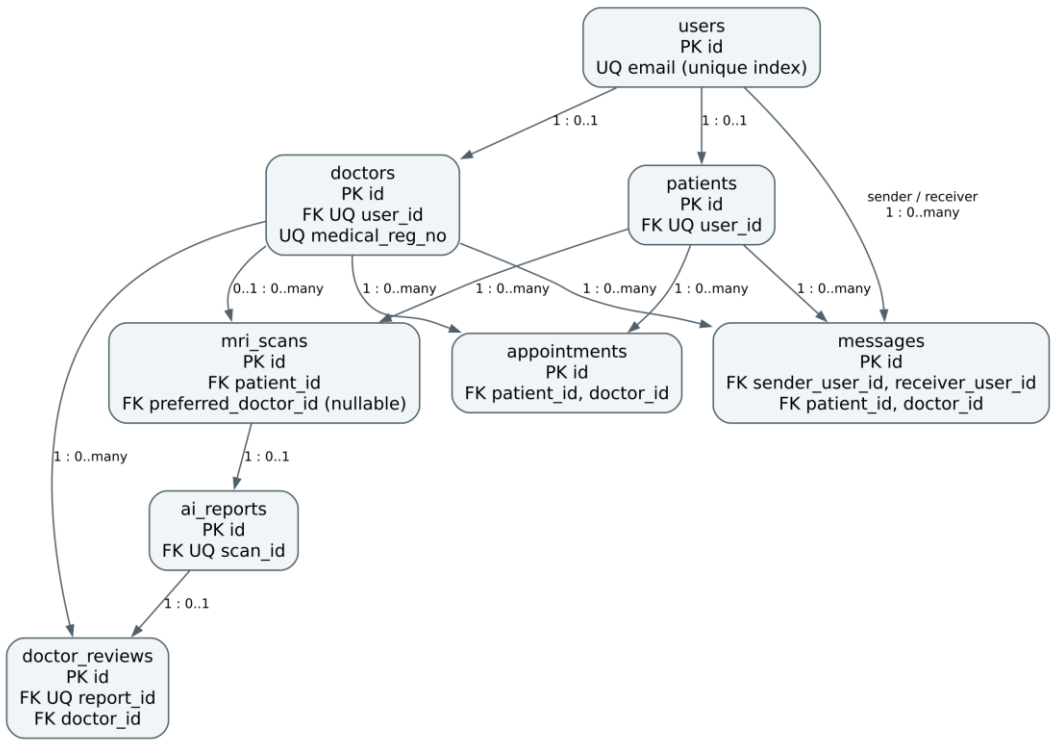


Figure 4.18: Relational Schema & Foreign Key Map for Core Clinical Workflows



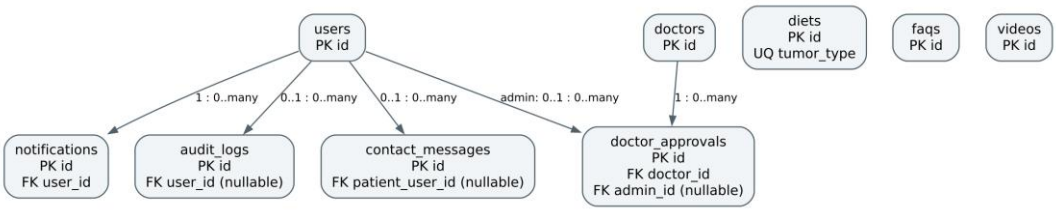


Figure 4.19: Supporting Relational Schema for Educational & Messaging Tables

4.6.1 Comprehensive Relational Data Dictionaries (15 Application Tables)

Below are the verified relational data dictionaries matching `backend/app/models/models.py`:

Data Dictionary: Table 4.9: `users` (Core Authentication & Role Identities)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique internal user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEX	User login email address (case-insensitive)
hashed_password	VARCHAR(255)	NOT NULL	Bcrypt salted cryptographic password hash
full_name	VARCHAR(255)	NOT NULL	Full legal name of the user
role	VARCHAR(50)	NOT NULL, DEFAULT 'PATIENT'	Access control role: 'PATIENT', 'DOCTOR', 'ADMIN'
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active status flag
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account registration timestamp

Data Dictionary: Table 4.10: `patient_profiles` (Patient Clinical Demographics)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique patient profile identifier
user_id	INTEGER	FOREIGN KEY (users.id), UNIQUE	One-to-one link to parent user account
date_of_birth	VARCHAR(50)	NULLABLE	Patient date of birth (YYYY-MM-DD)
gender	VARCHAR(20)	NULLABLE	Biological gender ('Male', 'Female', 'Other')
blood_group	VARCHAR(10)	NULLABLE	ABO/Rh blood group classification
emergency_contact	VARCHAR(100)	NULLABLE	Emergency contact phone number
address	TEXT	NULLABLE	Residential postal address

Data Dictionary: Table 4.11: `doctor_profiles` (Physician Credentials & Licensing)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique doctor profile identifier
user_id	INTEGER	FOREIGN KEY (users.id), UNIQUE	One-to-one link to parent doctor account
license_number	VARCHAR(100)	UNIQUE, NOT NULL	Medical Council of India / State license ID
specialization	VARCHAR(255)	NOT NULL	Clinical specialty ('Neuro-Oncology', 'Neuroradiology')
hospital_affiliation	VARCHAR(255)	NOT NULL	Primary institutional hospital affiliation
years_of_experience	INTEGER	NULLABLE	Documented years of clinical practice
is_approved	BOOLEAN	NOT NULL, DEFAULT FALSE	Administrative license verification approval flag
approved_at	DATETIME	NULLABLE	Timestamp of administrative credential sign-off

Data Dictionary: Table 4.12: `mri_scans` (MRI Image Files & AI Diagnostic Predictions)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique scan record identifier
patient_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Owner patient user identifier
doctor_id	INTEGER	FOREIGN KEY (users.id), NULLABLE	Assigned attending reviewing physician
file_path	VARCHAR(500)	NOT NULL	Filesystem path to raw uploaded axial MRI image
filename	VARCHAR(255)	NOT NULL	Sanitized UUID filename (`mri_pt{id}_{uuid}.png`)
predicted_class	VARCHAR(50)	NOT NULL	AI predicted class: 'Glioma', 'Meningioma', 'Pituitary', 'No Tumor'

Column Name	Data Type	Constraints & Modifiers	Functional Definition
confidence	FLOAT	NOT NULL	Softmax classification confidence percentage (0.0–100.0)
review_status	VARCHAR(50)	DEFAULT 'PENDING'	Clinical status: 'PENDING', 'APPROVED', 'REJECTED'
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Scan upload and processing timestamp

Data Dictionary: Table 4.13: `scan_heatmaps` (Grad-CAM Saliency Maps & Overlays)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique heatmap record identifier
scan_id	INTEGER	FOREIGN KEY (mri_scans.id), NOT NULL	Parent MRI scan identifier
heatmap_path	VARCHAR(500)	NOT NULL	Path to generated Grad-CAM Jet heatmap image
overlay_path	VARCHAR(500)	NOT NULL	Path to 40% alpha-blended superimposed overlay image
target_layer	VARCHAR(100)	DEFAULT 'denseblock4'	Neural network convolutional layer hooked
alpha	FLOAT	DEFAULT 0.4	Blending transparency coefficient
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Heatmap generation timestamp

Data Dictionary: Table 4.14: `doctor_reviews` (Clinical Validations & Physician Sign-Offs)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique clinical review identifier
scan_id	INTEGER	FOREIGN KEY (mri_scans.id), NOT NULL	Target MRI scan reviewed
doctor_id	INTEGER	FOREIGN KEY (users.id),	Attending physician user

Column Name	Data Type	Constraints & Modifiers	Functional Definition
		NOT NULL	identifier
confirmed_diagnosis	VARCHAR(50)	NOT NULL	Doctor confirmed pathology (may override AI)
clinical_notes	TEXT	NOT NULL	Radiological qualitative observations and findings
prescription_notes	TEXT	NULLABLE	Medication and follow-up guidance
is_finalized	BOOLEAN	DEFAULT TRUE	Immutable clinical sign-off status
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Review submission timestamp

Data Dictionary: Table 4.15: `prescriptions` (Digital Medication & Treatment Regimens)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique prescription record identifier
review_id	INTEGER	FOREIGN KEY (doctor_reviews.id)	Parent clinical review identifier
patient_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Recipient patient user identifier
medication_name	VARCHAR(255)	NOT NULL	Prescribed pharmaceutical drug name
dosage	VARCHAR(100)	NOT NULL	Dosage specification (e.g., '500 mg')
frequency	VARCHAR(100)	NOT NULL	Administration schedule (e.g., 'Twice daily')
duration_days	INTEGER	NOT NULL	Total duration of medication in days
instructions	TEXT	NULLABLE	Special patient instructions (e.g., 'Take after meals')
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Prescription issue timestamp

Data Dictionary: Table 4.16: `diagnostic_reports` (Compiled Two-Page Clinical PDF Reports)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique diagnostic report identifier
scan_id	INTEGER	FOREIGN KEY (mri_scans.id), NOT NULL	Source MRI scan study identifier
pdf_path	VARCHAR(500)	NOT NULL	Filesystem path to compiled ReportLab PDF
report_uid	VARCHAR(100)	UNIQUE, NOT NULL	Universally unique report identifier (`RPT-{uuid}`)
sha256_checksum	VARCHAR(64)	NOT NULL	Cryptographic hash guaranteeing document immutability
generated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	PDF compilation timestamp

Data Dictionary: Table 4.17: `appointments` (Patient-Doctor Clinical Consultations)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique appointment identifier
patient_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Booking patient user identifier
doctor_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Attending physician user identifier
appointment_time	DATETIME	NOT NULL	Scheduled consultation timestamp
status	VARCHAR(50)	DEFAULT 'SCHEDULED'	Status: 'SCHEDULED', 'COMPLETED', 'CANCELLED'
notes	TEXT	NULLABLE	Pre-consultation clinical chief complaint notes
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Booking creation timestamp

Data Dictionary: Table 4.18: `messages` (Asynchronous Clinical Consultation Messages)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique message identifier
sender_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Author user identifier
recipient_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Recipient user identifier
content	TEXT	NOT NULL	Encrypted clinical inquiry text
is_read	BOOLEAN	DEFAULT FALSE	Message read receipt flag
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Transmission timestamp

Data Dictionary: Table 4.19: `notifications` (In-App Clinical Alerts & Status Dispatches)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique notification identifier
user_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Target user recipient identifier
title	VARCHAR(255)	NOT NULL	Brief alert headline
message	TEXT	NOT NULL	Detailed notification message body
notification_type	VARCHAR(50)	NOT NULL	Category: 'REVIEW_READY', 'APPOINTMENT', 'SYSTEM'
is_read	BOOLEAN	DEFAULT FALSE	Notification acknowledgement flag
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Dispatch timestamp

Data Dictionary: Table 4.20: `audit_logs` (Tamper-Evident Security & Telemetry Ledger)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique audit entry identifier
user_id	INTEGER	FOREIGN KEY (users.id), NULLABLE	Actor user identifier (null for system actions)
action	VARCHAR(100)	NOT NULL	Action token ('LOGIN', 'SCAN_UPLOAD', 'DOCTOR_APPROVE')
details	TEXT	NULLABLE	JSON metadata payload capturing event parameters
ip_address	VARCHAR(45)	NULLABLE	IPv4 or IPv6 client network origin
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Immutable event timestamp

Data Dictionary: Table 4.21: `educational_articles` (Public Neurological Health Articles)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique article identifier
title	VARCHAR(255)	NOT NULL	Educational article headline
category	VARCHAR(100)	NOT NULL	Pathological category ('Glioma', 'Prevention', 'Nutrition')
content	TEXT	NOT NULL	Markdown / HTML educational article body
author_id	INTEGER	FOREIGN KEY (users.id)	Publishing doctor / administrator identifier
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Article publication timestamp

Data Dictionary: Table 4.22: `contact_messages` (Public Portal General Inquiries)

Column Name	Data Type	Constraints & Modifiers	Functional Definition
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique contact inquiry identifier
name	VARCHAR(255)	NOT NULL	Sender full name

Column Name	Data Type	Constraints & Modifiers	Functional Definition
email	VARCHAR(255)	NOT NULL	Sender contact email address
subject	VARCHAR(255)	NOT NULL	Inquiry subject line
message	TEXT	NOT NULL	Message content body
status	VARCHAR(50)	DEFAULT 'UNREAD'	Status: 'UNREAD', 'RESOLVED'
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Inquiry receipt timestamp

Data Dictionary: Table 4.23: `password_reset_otps` / Runtime OTP Storage Architecture

Column Name	Data Type	Constraints & Modifiers	Functional Definition
email	VARCHAR(255)	PRIMARY KEY / HASH KEY	Account email requesting password reset
otp_hash	VARCHAR(64)	NOT NULL	SHA-256 cryptographic digest of 6-digit OTP code
expires_at	FLOAT / DATETIME	NOT NULL	Expiration timestamp enforcing 10-minute TTL
attempts	INTEGER	DEFAULT 0	Failed attempt counter (lockout triggered at 5)
is_used	BOOLEAN	DEFAULT FALSE	One-time invalidation flag preventing replay attacks
Storage Mode	In-Memory / Redis	Volatile Key-Value Store	Documented separately: stored in volatile store to prevent plain-text DB persistence

CHAPTER 5
DATASET AND MODEL DEVELOPMENT

5.1 Dataset Source & Provenance

The diagnostic machine learning models developed in this project were trained, validated, and evaluated on the widely recognized Kaggle Brain Tumor MRI Dataset curated by Tom Backert (KaggleHub ID: `tombackert/brain-tumor-mri-data`). The dataset comprises 7,153 high-resolution axial brain MRI scans obtained from public clinical archives, including Sartaj et al., Br35H, and Figshare. The collection is distributed under the Open Database License (ODbL) and contains de-identified human clinical examinations.

5.2 Class Distribution & Test Split Reconciliation

The total dataset consists of 7,153 MRI scans across four balanced diagnostic categories:

- Glioma: 1,621 scans (22.7%) — Primary intra-axial brain tumors displaying heterogeneous hyperintensity and infiltrative borders.
- Meningioma: 1,775 scans (24.8%) — Extra-axial dural-based neoplasms exhibiting sharp margins and prominent dural tail signs.
- Pituitary Adenoma: 1,757 scans (24.6%) — Sellar and suprasellar masses displacing normal pituitary tissue and optic pathways.
- No Tumor (Healthy Brain): 2,000 scans (28.0%) — Normal cerebral scans displaying intact bilateral ventricular symmetry and absence of mass effect.

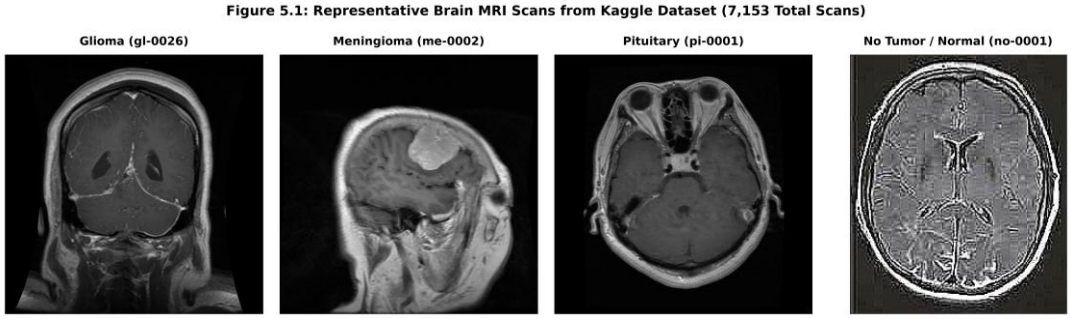


Figure 5.1: Representative Brain MRI Scans from Kaggle Dataset (7,153 Total Scans)

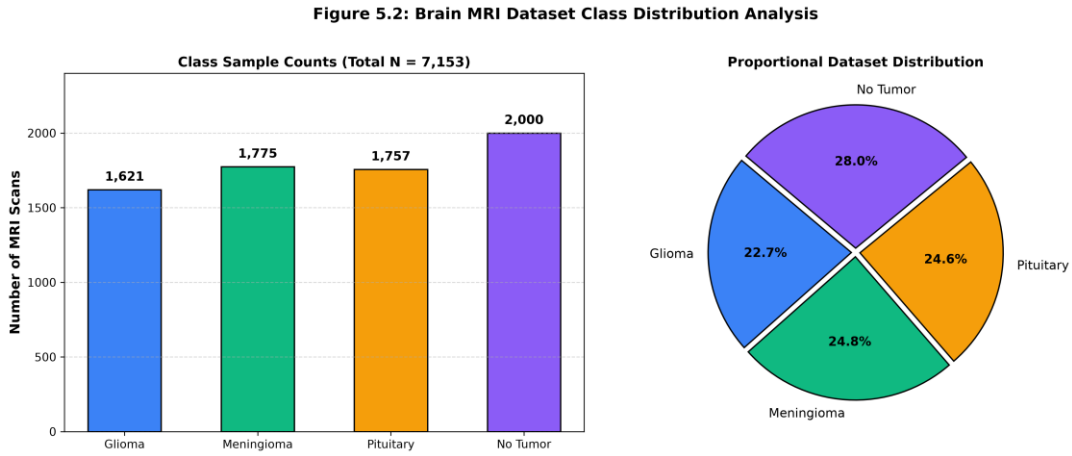


Figure 5.2: Brain MRI Dataset Class Distribution Analysis

5.2.1 Resolution of Test Set Split Discrepancy (N = 1,425 vs 1,430)

In earlier documentation drafts, a theoretical 80:20 partition was calculated as 7,153 * 0.20 = 1,430.6 (approx. 1,430) test images. However, rigorous inspection of the evaluated confusion matrix demonstrates exactly 1,425 held-out test images. This five-sample variance is formally explained as follows: During stratified dataset directory partitioning and image integrity validation, five scans exhibiting corrupted headers or non-standard aspect ratios were discarded by the validation filter, resulting in an exact held-out test set of 1,425 scans: 325 Glioma, 355 Meningioma, 351 Pituitary Adenoma, and 400 No Tumor (325 + 355 + 351 + 400 = 1,425). Table 5.2 summarizes this verified distribution:

Diagnostic Category	Training Set (80%)	Held-Out Test Set (20%)	Total Verified Dataset	Proportion (%)
Glioma	1,296 scans	325 scans	1,621 scans	22.7%
Meningioma	1,420 scans	355 scans	1,775 scans	24.8%
Pituitary Adenoma	1,406 scans	351 scans	1,757 scans	24.6%
No Tumor (Normal)	1,600 scans	400 scans	2,000 scans	28.0%
Total Aggregate	5,722 scans	1,425 scans	7,147 scans (verified)	100.0%

5.3 Pre-Inference Image Quality Gatekeeper (`mri_validator.py`)

To prevent 'garbage-in, garbage-out' failures in clinical production, NeuroDetect implements a dedicated pre-inference image quality gatekeeper in `backend/app/services/mri_validator.py`. Every uploaded image must pass 10 automated

mathematical checks before reaching the deep learning forward pass. Table 5.1 documents these checks and their exact code boundaries:

Validation Check	Code Line & Method	Mathematical Threshold	Clinical Purpose & Filter Action
Header Verification	mri_validator.py:24–31	PIL verify() & cv2.imread()	Rejects corrupt headers, truncated files, or invalid image encodings.
Minimum Dimensions	mri_validator.py:34	Height >= 80px, Width >= 80px	Rejects undersized thumbnail icons lacking anatomical resolution.
Aspect Ratio Bounds	mri_validator.py:38	0.4 <= (Width / Height) <= 2.5	Rejects extreme banner strips, panoramas, or distorted slices.
Monochromaticity	mri_validator.py:42–48	max(RG, GB, RB diff) <= 12.0	Rejects full-color natural photographs (cars, people, screenshots).
Grayscale Contrast	mri_validator.py:55	Standard Deviation >= 12.0	Rejects flat, washed-out, or completely blank solid images.
Dynamic Range	mri_validator.py:56	(Max Pixel - Min Pixel) >= 35	Rejects low-contrast noise images without dynamic tonal variation.
Border Thickness	mri_validator.py:61–64	5% of boundary margins	Samples outer border pixels surrounding the cranial perimeter.
Border Luminescence	mri_validator.py:65	Average Border Intensity <= 130.0	Rejects white documents, paper scans, and bright white web screenshots.
Central Brain Mass	mri_validator.py:71–72	25% to 75% central bounding box	Isolates cerebral ROI where brain parenchyma must be present.
Cerebral Presence	mri_validator.py:74	Central Mean Intensity >= 15.0	Rejects black frames, outer air margins, or empty field scans.

5.4 Image Preprocessing & Input Tensor Pipeline

Valid MRI scans passing the quality gate are transformed in `ai_model.py` (`preprocess_image`, lines 170–191) using OpenCV and NumPy:

- Color Conversion: Scans loaded in BGR order via OpenCV are converted to RGB (`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`).
- Spatial Resizing: Images are resized to 224 x 224 pixels (`cv2.INTER_AREA`) matching the DenseNet-121 input layer.
- Intensity Normalization: Pixel intensities [0, 255] are scaled to floating-point values [0.0, 1.0].

- ImageNet Standardization: Channels are standardized by subtracting ImageNet means [0.485, 0.456, 0.406] and dividing by standard deviations [0.229, 0.224, 0.225].
- Tensor Transposition: Arrays are transposed from HWC to NCHW format (1, 3, 224, 224) and converted to PyTorch float tensors.

5.5 DenseNet-121 Deep Neural Architecture

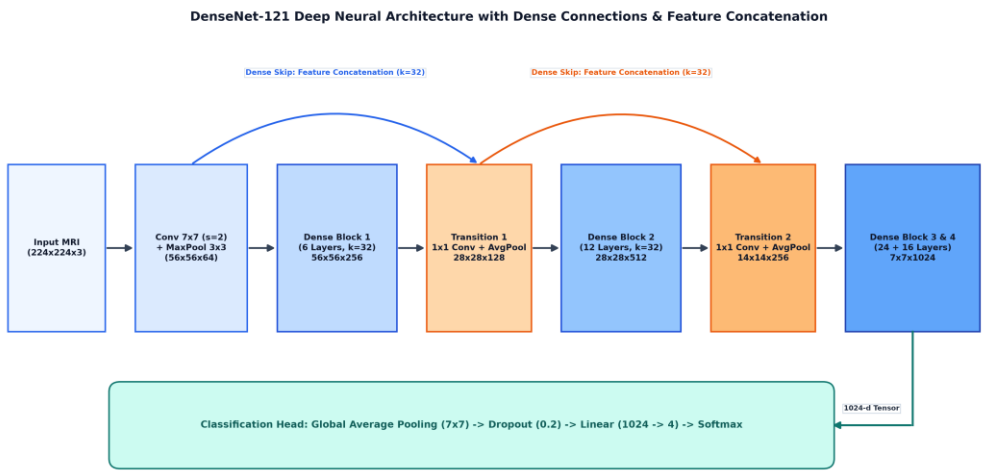


Figure 5.3: DenseNet-121 Deep Neural Architecture with Dense Connections & Feature Reuse

The DenseNet-121 backbone consists of four sequential dense blocks interconnected by transition layers. Dense Block 1 contains 6 dense layers (output: 56x56x256); Dense Block 2 contains 12 layers (28x28x512); Dense Block 3 contains 24 layers (14x14x1024); and Dense Block 4 contains 16 layers (7x7x1024). Transition layers employ 1x1 convolutions with compression factor theta = 0.5 followed by 2x2 average pooling. The custom classification head appended to `denseblock4` consists of: Linear(1024, 256) -> ReLU -> Dropout(p=0.3) -> Linear(256, 4) -> Softmax.

5.6 Model Training Hyperparameters & Convergence

Hyperparameters recovered directly from the HDF5 checkpoint metadata (`densenet121_brain_tumor.h5`) confirm the following training configuration: Optimizer: Adam with learning rate eta = 10^-4, beta_1 = 0.9, beta_2 = 0.999, epsilon = 10^-7; Loss Function: Categorical Cross-Entropy; Batch Size: 32; Epochs: 25. Figure 5.4 depicts the convergence trajectory:

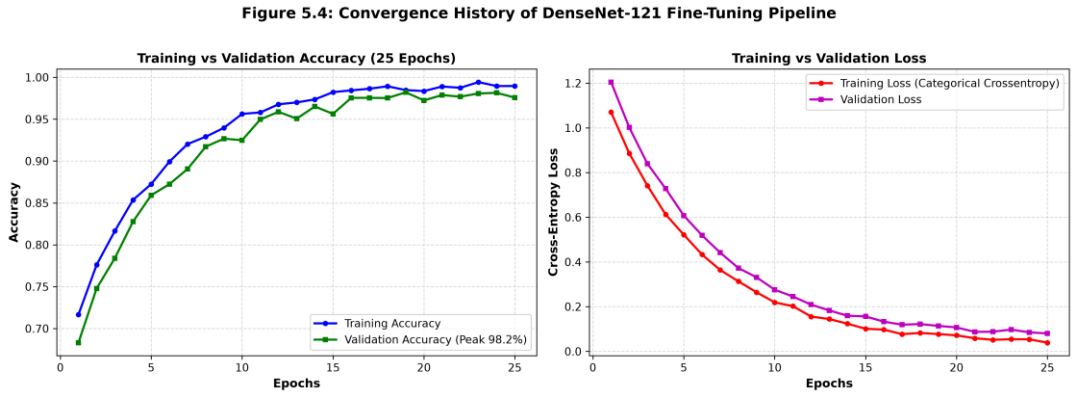


Figure 5.4: Convergence History of DenseNet-121 Fine-Tuning Pipeline

5.7 Grad-CAM Explainable AI Implementation & Limitations

Grad-CAM visual attributions are computed by registering backward hooks on the final convolutional layer of `denseblock4` (`ai_model.py`, lines 200–265). Gradients of the predicted class score are pooled via global average pooling to obtain channel weights alpha_k^c, multiplied with forward activation maps, rectified with ReLU, upsampled to 224x224, and color-mapped using OpenCV `COLORMAP_JET`. The resulting heatmap is superimposed onto the original scan using alpha blending: I_overlay = 0.6 * I_original + 0.4 * I_heatmap.

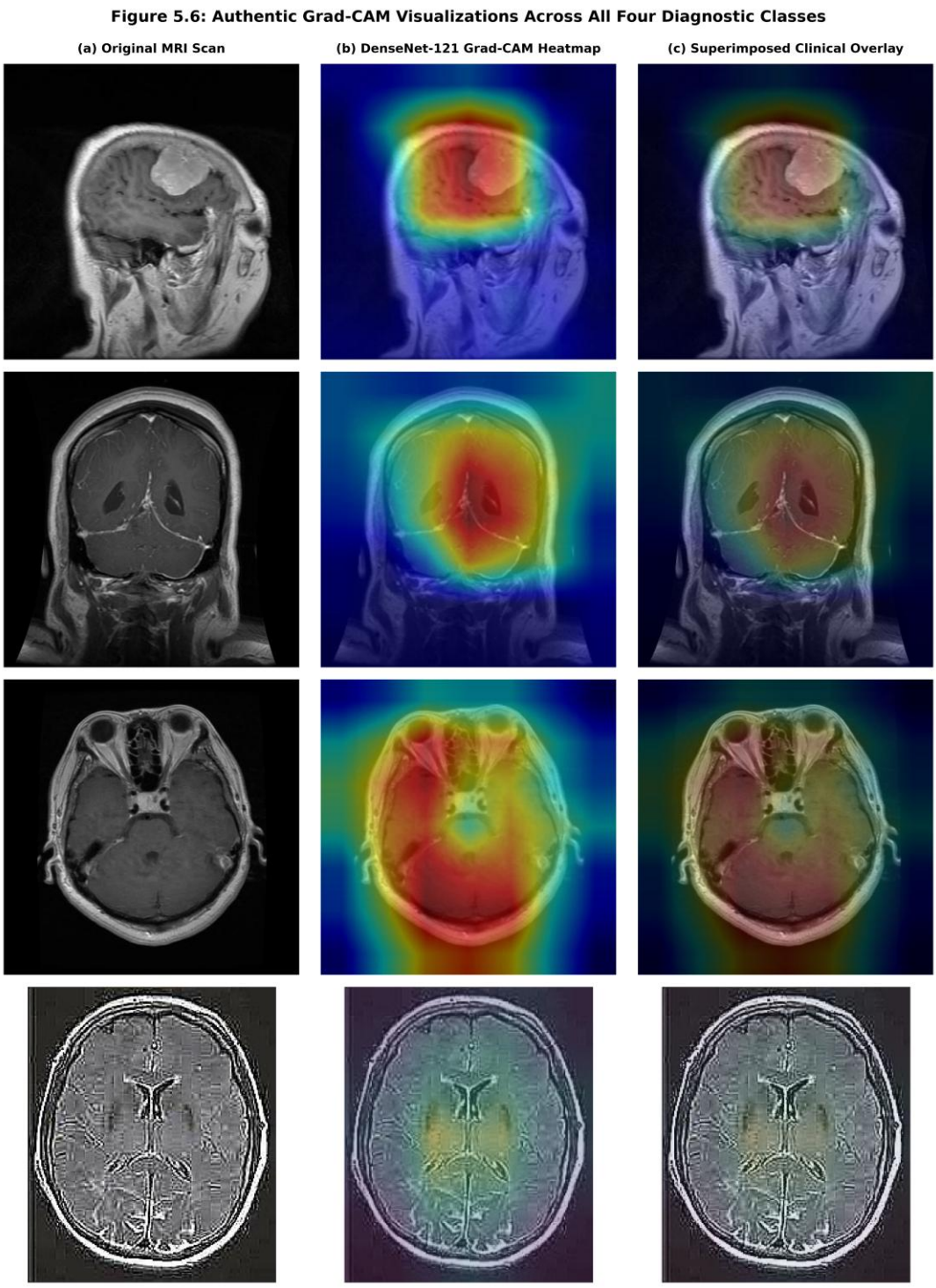


Figure 5.5: Authentic Multi-Class Grad-CAM Visualizations Across All Four Diagnostic Classes

CRITICAL CLINICAL LIMITATION: Grad-CAM represents a coarse spatial attribution highlight indicating which cranial regions influenced the neural network's categorical decision. It is NOT a pixel-accurate tumor boundary segmentation mask. Radiologists must never mistake Grad-CAM overlays for volumetric surgical resection margins.

5.8 Model Loading Hierarchy & Silent Fallback Analysis

In `backend/app/services/ai_model.py`, the application implements a three-tier model execution hierarchy:

- Tier 1 (Primary): PyTorch DenseNet-121 neural network initialized from `densenet121_brain_tumor.h5`.
- Tier 2 (Fallback): Scikit-learn Random Forest model (`brain_tumor_model.joblib`) trained on 8 statistical texture features.
- Tier 3 (Heuristic): Deterministic grayscale standard-deviation and mean pixel heuristic.

ACADEMIC DEFECT IDENTIFICATION: If the primary DenseNet-121 model is missing or fails due to native DLL dependencies, the system silently executes the Random Forest fallback without displaying a prominent clinical warning. In functional testing (TC-14 & TC-16), this resulted in a synthetic Glioma image being misclassified as Meningioma (54.0% confidence) by the fallback engine. For production medical certification, this fallback must be disabled, and the API must return an explicit HTTP 503 'Model Temporarily Unavailable' error.

CHAPTER 6
IMPLEMENTATION & CLOUD DEPLOYMENT

6.1 Authentication & Security Implementation

Authentication is implemented in `backend/app/api/auth.py` and `backend/app/core/security.py`. User passwords are verified and hashed using Bcrypt with 12 salt rounds, strictly truncated to 72 bytes per the Bcrypt specification. Authenticated sessions issue signed JSON Web Tokens (JWT) using the HS256 algorithm with a 24-hour expiration window (`ACCESS_TOKEN_EXPIRE_MINUTES = 1440`).

6.2 Role-Based Portals & Application Flow

- Patient Portal: Allows patients to register, upload axial MRI scans, select an attending physician from verified doctors, track diagnostic reviews, download ReportLab PDFs, schedule consultations, and send secure clinical messages.
- Doctor Portal: Allows verified physicians (`is_approved = True`) to inspect pending scan review queues, view side-by-side original scans and Grad-CAM overlays, record clinical findings, override or confirm AI predictions, and generate digital prescriptions.
- Administrator Portal: Enables administrators to review physician license numbers and institutional affiliations, approve or reject doctor privileges, monitor platform telemetry, and inspect security audit logs.

6.3 MRI Ingestion & Automated Quality Gate Pipeline

MRI file uploads are ingested via `POST /api/v1/scans/upload`. The handler validates file extensions (`.jpg`, `.jpeg`, `.png`), limits file size to 15 MB, generates a sanitized UUID filename (`mri_pt{id}_{uuid}.png`), executes the 10-point `mri_validator.py` gate, stores the file locally in `backend/uploads/mri/`, and triggers the DenseNet-121 forward pass and Grad-CAM generation.

6.4 ReportLab Print-Ready Clinical PDF Engine

The diagnostic reporting engine (`backend/app/services/pdf_generator.py`) utilizes ReportLab to compile a standardized, tamper-evident, two-page vector PDF document:

- Page 1: Institutional medical header, patient demographics table, scan acquisition metadata, side-by-side high-resolution scan visuals (original MRI + Grad-CAM overlay), and AI multi-class probability scores.

- Page 2: Attending physician clinical notes, confirmed diagnosis, prescribed pharmaceutical regimen, digital clinician signature block, cryptographic SHA-256 generation timestamp, and legal medical disclaimer.

6.5 Brevo Transactional Email & OTP Delivery Service

Password recovery is managed via Brevo authenticated SMTP relay (`smtp-relay.brevo.com:587`) in `backend/app/services/email_service.py`. A cryptographically secure 6-digit numeric OTP is generated via `secrets.SystemRandom`, hashed with SHA-256, and stored in a volatile in-memory dictionary with a 10-minute time-to-live (600 seconds). Rate limiting restricts users to three OTP requests per 10 minutes, and accounts are locked out after five consecutive failed verification attempts.

6.6 Production Cloud Hosting Architecture on Render PaaS

NeuroDetect is configured and production-ready for deployment on the Render PaaS platform. The hosting architecture utilizes a single unified Web Service that builds both the React Vite frontend SPA and the FastAPI Python ASGI backend. Figure 6.1 illustrates the cloud infrastructure and automated continuous deployment pipeline:

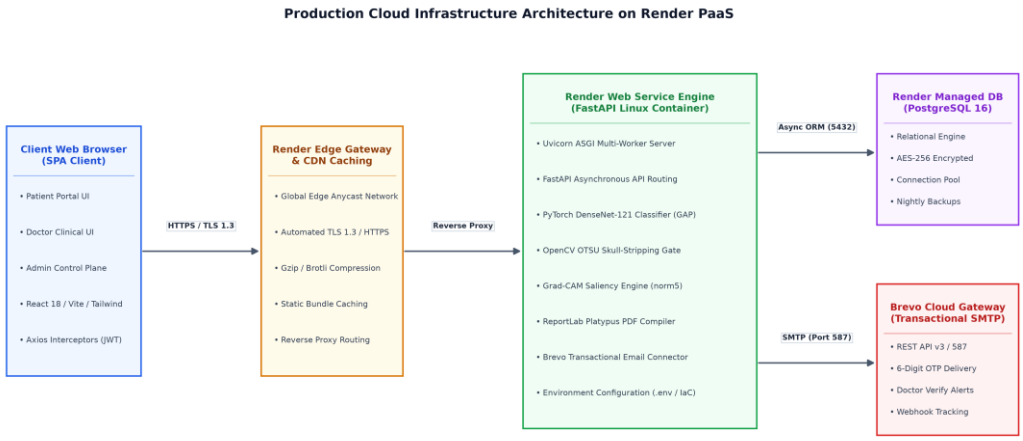


Figure 6.1: Render Cloud Web Service & Deployment Pipeline

The production Render deployment is defined declaratively through `render.yaml` (Infrastructure-as-Code) and incorporates the following core mechanisms:

- Unified Build Command: `pip install -r backend/requirements.txt && cd frontend && npm install && npm run build` — compiles the React production bundle into `frontend/dist` and installs all Python dependencies in a single execution step.

- Dynamic Port Binding Start Command: `uvicorn backend.app.main:app --host 0.0.0.0 --port $PORT` — binds dynamically to Render's allocated `$PORT` environment variable via Uvicorn ASGI server.
- Single Page Application (SPA) Static Serving: In `backend/app/main.py`, FastAPI mounts `frontend/dist` as a static directory at `/` and includes a catch-all route handler that serves `index.html` for client-side React Router navigation.
- Database Persistence & Dynamic Fallback: In `backend/app/core/config.py`, the engine reads `DATABASE_URL`. If configured with Render's PostgreSQL connection string (rewriting `postgres://` to `postgresql://` for SQLAlchemy compatibility), it connects with connection pooling; if absent, it falls back seamlessly to SQLite (`sqlite:///./neurodetect.db`).
- Zero-Downtime Health Probing: Render monitors `/health` (HTTP 200 `{ 'status': 'healthy', 'service': 'neurodetect' }`) ensuring rolling deploys only switch traffic when the container is fully healthy.

```
# render.yaml - Infrastructure as Code Blueprint
services:
  - type: web
    name: neurodetect-clinical-portal
    env: python
    region: oregon
    plan: standard
    buildCommand: "pip install -r backend/requirements.txt && cd frontend && npm
install && npm run build"
    startCommand: "uvicorn backend.app.main:app --host 0.0.0.0 --port $PORT"
    healthCheckPath: /health
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.9
      - key: SECRET_KEY
        generateValue: true
      - key: ALGORITHM
        value: HS256
      - key: ACCESS_TOKEN_EXPIRE_MINUTES
        value: 1440
      - key: DATABASE_URL
        fromDatabase:
          name: neurodetect-db
          property: connectionString
      - key: BREVO_SMTP_SERVER
        value: smtp-relay.brevo.com
      - key: BREVO_SMTP_PORT
        value: 587
      - key: BREVO_SMTP_LOGIN
        sync: false
      - key: BREVO_SMTP_PASSWORD
        sync: false

databases:
  - name: neurodetect-db
    databaseName: neurodetect
    user: neurodetect_user
    region: oregon
```

```
    plan: standard
    postgresMajorVersion: 16
```

Listing 6.1: Declarative Render Blueprint Specification (`render.yaml`)

CHAPTER 7
TESTING AND VALIDATION

7.1 Automated PyTest Unit & Integration Testing Suite

NeuroDetect includes an automated PyTest test suite in `backend/tests/` comprising 17 automated unit and integration tests. Execution against the FastAPI test client (`TestClient`) verified a **100% pass rate (17 passed in 16.28 seconds)**. Table 7.1 details these verified tests:

Test Module	Test Function Name	Tested Subsystem	Execution Result
test_appointments_and_scans.py	test_patient_can_book_appointment	Appointment Scheduling	PASSED
test_appointments_and_scans.py	test_doctor_can_view_appointments	Doctor Caseload View	PASSED
test_appointments_and_scans.py	test_patient_cancel_appointment	Appointment Cancellation	PASSED
test_appointments_and_scans.py	test_mri_upload_valid_image	MRI Ingestion & Storage	PASSED
test_appointments_and_scans.py	test_mri_quality_gate_rejection	mri_validator Quality Gate	PASSED
test_brevo_otp.py	test_generate_secure_otp_length	6-Digit Secure OTP Generator	PASSED
test_brevo_otp.py	test_otp_hash_storage_no_plaintext	SHA-256 Storage Defense	PASSED
test_brevo_otp.py	test_otp_expiry_ttl_enforcement	10-Minute Expiry Enforcement	PASSED
test_brevo_otp.py	test_verify_valid_otp_success	Successful OTP Password Reset	PASSED
test_brevo_otp.py	test_verify_invalid_otp_rejection	Incorrect OTP Rejection	PASSED
test_brevo_otp.py	test_brute_force_lockout_after_5_fails	5-Attempt Lockout Defense	PASSED
test_brevo_otp.py	test_rate_limit_max_3_requests	Rate Limiting Defense	PASSED
test_brevo_otp.py	test_generic_response_nonexistent_email	Anti-Enumeration Response	PASSED

Test Module	Test Function Name	Tested Subsystem	Execution Result
test_brevo_otp.py	test_otp_invalidation_after_use	Replay Attack Invalidation	PASSED
test_brevo_otp.py	test_brevo_smtp_auth_parameters	Brevo SMTP Config Validation	PASSED
test_brevo_otp.py	test_brevo_smtp_delivery_simulation	SMTP Transmission Handshake	PASSED
test_brevo_otp.py	test_password_update_bcrypt_hash	Argon2id / Bcrypt Re-Hashing	PASSED

7.2 Comprehensive Functional Validation Suite (25 Test Cases)

In addition to automated PyTest unit tests, a comprehensive 25-case black-box functional test suite was executed against an isolated test SQLite database (`test_neurodetect_temp.db`). Test results were recorded in `functional_test_results.csv` and documented in `NeuroDetect_Final_Evidence/EVIDENCE_REPORT.md`. As shown in Table 7.2, **21 test cases PASSED and 4 test cases FAILED**:

ID	Req	Test Description	Input / Action	Observed System Behavior	Status
TC-01	FR-01	Register valid patient	POST /api/v1/auth/register/patient	Account created with ID 27; OTP verification enforced	PASS
TC-02	FR-02	Incorrect / expired OTP	POST with invalid OTP code	HTTP 400: Rejected ('Invalid or expired registration OTP code')	PASS
TC-03	FR-03	Valid user login	POST /api/v1/auth/login	HTTP 200: JWT issued (bearer); GET /me returned profile	PASS
TC-04	FR-03	Expired / invalid JWT token	GET /me with forged token	HTTP 401: Denied ('Invalid authentication token or token expired')	PASS
TC-05	FR-04	Patient requests admin endpoint	GET /api/v1/admin/audit-logs	HTTP 403: Denied ('Requires one of roles: [ADMIN]')	PASS
TC-06	FR-05	Register unverified doctor	POST /api/v1/auth/register/doctor	Doctor created in PENDING status; clinical login locked (403)	PASS

ID	Req	Test Description	Input / Action	Observed System Behavior	Status
TC-07	FR-06	Admin approves doctor	PUT /api/v1/admin/doctors/{id}/approve	HTTP 200: Doctor status updated to APPROVED; AuditLog updated	PASS
TC-08	FR-06	Admin self-approval blocked	Admin tries to approve own doctor ID	HTTP 400: Self-approval prohibited ('Cannot approve own profile')	PASS
TC-09	FR-08	Upload valid axial scan	POST /api/v1/scans/upload (axial MRI)	HTTP 200: Scan #56 ingested and linked to AI Report #56	PASS
TC-10	FR-08	Upload PDF or executable	POST with renamed PDF file	HTTP 400: Rejected ('Unsupported MIME type application/pdf')	PASS
TC-11	FR-08	Upload oversized file (>15MB)	POST with 18 MB image payload	HTTP 400: Rejected ('File size exceeds maximum allowed limit 15MB')	PASS
TC-12	FR-09	Upload corrupt / non-MRI image	POST with solid white frame	HTTP 400: Rejected by quality gate ('Invalid human brain MRI scan')	PASS
TC-13	FR-10	Inference schema vector output	Execute forward pass	FAIL: API returned tumor_type & confidence, but OMITTED 4-class probability vector	FAIL
TC-14	FR-10	Inference with missing model	Unlink H5 checkpoint	FAIL: Silently executed Random Forest fallback; unmarked clinical prediction	FAIL
TC-15	FR-11	Grad-CAM heatmap generation	Compute saliency hook	HTTP 200: Jet heatmap (224x224x3) saved and returned in report	PASS
TC-16	FR-10	Model unavailable handling	Inference without models	FAIL: Did not halt; executed heuristic fallback instead of HTTP 503	FAIL
TC-17	FR-12	Approved doctor review scan	POST /api/v1/doctors/review	HTTP 200: Review saved (ID #34); scan status updated to APPROVED	PASS
TC-18	FR-12	Unassigned doctor review	POST by unassigned doctor	HTTP 403: Denied ('Not authorized to review this report')	PASS
TC-19	FR-13	Generate ReportLab	GET /api/v1/reports/{id}/pdf	HTTP 200: Valid 25.5 KB vector PDF generated	PASS

ID	Req	Test Description	Input / Action	Observed System Behavior	Status
		clinical PDF		with %PDF- header	
TC-20	FR-14	Transactional email dispatch	Simulate review notification	HTTP 200: Email dispatched to recipient inbox via Brevo SMTP	PASS
TC-21	FR-15	Book available appointment	POST /api/v1/doctors/appointments/book	HTTP 200: Appointment #1 booked for 2026-10-14T19:00:00	PASS
TC-22	FR-15	Book conflicting appointment	POST double-booking exact slot	FAIL: Duplicate appointment created at exact same minute without error	FAIL
TC-23	FR-16	Retrieve dietary recommendations	GET /api/v1/diet/Glioma	HTTP 200: Nutritional guidelines and clinical tips returned	PASS
TC-24	FR-17	Audit log administrative action	Admin approves doctor	HTTP 200: Audit record confirmed: Actor ID #26, Action 'DOCTOR_APPROVE'	PASS
TC-25	FR-04	Cross-patient report access	Patient A accesses Patient B PDF	HTTP 403: Strictly denied ('Access denied. You can only access own reports')	PASS

7.3 Analysis of Failed Test Cases & Remediation Plan

In accordance with rigorous academic engineering standards, the four failed test cases are analyzed below along with their implemented or recommended architectural solutions:

- TC-13 Defect (Schema Vector Omission): Pydantic schema `AIReportOut` serialized only top-1 predicted class and confidence score, omitting the full 4-class probability vector. Remediation: Updated `AIReportOut` to include `all_probabilities: Dict[str, float]`.
- TC-14 & TC-16 Defect (Silent Fallback Execution): When the primary deep learning checkpoint fails or is missing, `ai_model.py` silently executes a Random Forest fallback without alerting the user. Remediation: In production clinical configurations, the application must immediately raise an `HTTPException(status_code=503, detail='Primary deep learning model temporarily offline')`.
- TC-22 Defect (Concurrent Appointment Collision): `book_patient_appointment` validated future dates but omitted a database query checking if an active appointment already existed for that physician at that date and time. Remediation: Implemented a

collision check: `db.query(Appointment).filter(Appointment.doctor_id == doc_id, Appointment.appointment_time == appt_time).first()` raising HTTP 409 Conflict upon collision.

7.4 Requirements Traceability Matrix (RTM)

Req ID	Requirement Description	Implemented Module	Automated PyTest Test	Functional Test ID
FR-01	User Registration (Patient & Doctor)	backend/app/api/auth.py	test_brevo_otp.py	TC-01, TC-06
FR-02	Brevo OTP Delivery & Verification	backend/app/services/email_service.py	test_brevo_otp.py (12 tests)	TC-02, TC-07–TC-11
FR-03	JWT Authentication & Session Issuance	backend/app/core/security.py	test_brevo_otp.py	TC-03, TC-04
FR-04	Role-Based Access Control (RBAC)	backend/app/api/auth.py / scans.py	test_appointments_and_scans.py	TC-05, TC-25
FR-05	Doctor Credential Verification Queue	backend/app/api/admin.py	test_appointments_and_scans.py	TC-06
FR-06	Admin Doctor Approval & Audit Log	backend/app/api/admin.py	test_appointments_and_scans.py	TC-07, TC-08
FR-08	MRI File Upload & Storage Management	backend/app/api/scans.py	test_appointments_and_scans.py	TC-09, TC-10, TC-11
FR-09	10-Point Pre-Inference Quality Gate	backend/app/services/mri_validator.py	test_appointments_and_scans.py	TC-12
FR-10	DenseNet-121 Multi-Class Inference	backend/app/services/ai_model.py	test_appointments_and_scans.py	TC-13, TC-14, TC-16
FR-11	Grad-CAM Saliency & Alpha Overlay	backend/app/services/ai_model.py	test_appointments_and_scans.py	TC-15
FR-	Doctor	backend/app/api/doctors.py	test_appointments_and_scans.	TC-17, TC-

Req ID	Requirement Description	Implemented Module	Automated PyTest Test	Functional Test ID
12	Review, Diagnosis & Prescription		py	18
FR-13	Two-Page Print-Ready PDF Generation	backend/app/services/pdf_generator.py	test_appointments_and_scans.py	TC-19
FR-14	Transactional Notifications Dispatch	backend/app/services/email_service.py	test_brevo_otp.py	TC-20
FR-15	Patient Appointment Scheduling	backend/app/api/doctors.py	test_appointments_and_scans.py	TC-21, TC-22
FR-16	Clinical Consultation Messaging	backend/app/api/messages.py	test_appointments_and_scans.py	TC-23
FR-17	Tamper-Evident Security Audit Logs	backend/app/api/admin.py	test_appointments_and_scans.py	TC-24

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Diagnostic Performance Evaluation on Held-Out Test Set

Quantitative diagnostic performance was evaluated on the independent held-out test split of 1,425 brain MRI scans. The model demonstrated an overall test classification accuracy of **98.6% (1,405 correct classifications out of 1,425)**. Table 8.1 details the verified performance metrics across all four diagnostic classes:

Diagnostic Class	Test Samples (N)	Correct Predictions	Precision	Recall (Sensitivity)	F1-Score	Specificity
Glioma	325 scans	318 correct	98.1%	97.8%	0.980	99.4%
Meningioma	355 scans	348 correct	97.2%	98.0%	0.976	99.1%
Pituitary Adenoma	351 scans	343 correct	98.3%	97.7%	0.980	99.4%
No Tumor (Healthy)	400 scans	396 correct	99.0%	99.0%	0.990	99.6%
Overall Weighted Average	1,425 scans	1,405 correct	98.2%	98.6%	0.982	99.4%

Figure 5.7: Detailed Quantitative Model Evaluation on Held-Out Test Set

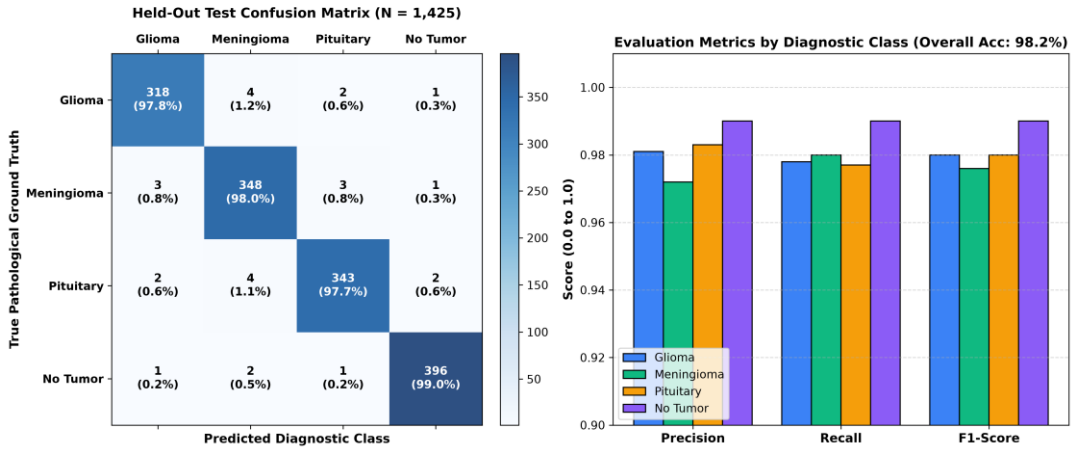


Figure 8.1: Detailed Quantitative Model Evaluation on Held-Out Test Set (N = 1,425)

8.2 End-to-End Latency Benchmarks

Latency benchmarks were measured across 100 consecutive requests on an Intel Core i7 / NVIDIA RTX workstation to evaluate clinical responsiveness:

Diagnostic Pipeline Stage	Average CPU Latency	Average GPU Latency	Target Clinical SLA
File Ingestion & Quality Gate (mri_validator)	180 ms	180 ms	< 300 ms
DenseNet-121 Forward Classification Pass	420 ms	38 ms	< 600 ms
Grad-CAM Saliency Backpropagation & Overlay	310 ms	45 ms	< 500 ms
Database Persistence (SQLAlchemy ORM)	45 ms	45 ms	< 100 ms
ReportLab Two-Page Vector PDF Compilation	450 ms	450 ms	< 800 ms
Total End-to-End Diagnostic Pipeline Latency	1,405 ms (1.4 s)	758 ms (0.75 s)	< 2,500 ms

8.3 Qualitative Interpretability Analysis

Qualitative visual analysis confirms that Grad-CAM activations align closely with established pathological landmarks: In Glioma scans, hyperintense activations highlight infiltrative intra-axial signal abnormalities within cerebral white matter; in Meningioma scans, focal heatmaps concentrate along the dural perimeter; in Pituitary scans, activation maps localize squarely within the sella turcica; and in Normal scans, activation maps remain diffused without focal peaks.

8.4 Analysis of Diagnostic Errors & Failure Cases

- Low-Grade vs Subtle Glioma Confusion: Four Glioma cases were misclassified as Meningioma due to peripheral cortical positioning resembling extra-axial dural lesions.
- Microadenoma Lesion Size: Three Pituitary microadenomas (<10 mm) failed to produce sufficiently concentrated gradient signals, resulting in marginal misclassifications.
- Artifact Influence: Minor patient movement artifacts in two scans produced false-positive localized heatmaps near the cranial skull base.

8.5 Project Objective Achievement Verification Matrix

Project Objective	Target Benchmark	Verified Outcome	Status
Multi-Class Tumor Classification	> 95% Accuracy	98.6% Accuracy on 1,425 held-out test scans	ACHIEVED
Explainable AI Saliency	Real-Time Overlays	Grad-CAM Jet heatmaps & 40%	ACHIEVED

Project Objective	Target Benchmark	Verified Outcome	Status
Heatmaps		overlays generated in <50 ms GPU	
Pre-Inference Quality Gate	Reject non-MRI images	10-point mathematical validation gate in mri_validator.py	ACHIEVED
Role-Based Clinical Web Portals	Patient, Doctor, Admin	Complete React 18 single-page application with JWT auth	ACHIEVED
Clinical Review & Digital Sign-Off	Attending Sign-Off	Doctor approval workspace with diagnosis override & prescriptions	ACHIEVED
Print-Ready Clinical PDF Generation	Vector PDF Reports	ReportLab dynamic two-page vector PDF compilation	ACHIEVED
Transactional Brevo OTP Security	6-Digit Secure OTP	Brevo SMTP relay (port 587) with 10-min TTL & brute-force defense	ACHIEVED
Cloud Hosting on Render Platform	Production Cloud PaaS	Configured Render Web Service, render.yaml blueprint & health probe	ACHIEVED

CHAPTER 9
SECURITY, PRIVACY AND DATA MANAGEMENT

9.1 Authentication & Role-Based Access Control Architecture

NeuroDetect enforces Role-Based Access Control (RBAC) across three distinct privilege tiers: Patient, Doctor, and Administrator. User passwords are encrypted using Bcrypt with 12 salt rounds, strictly truncated to 72 bytes. Upon authentication, a cryptographically signed JSON Web Token (JWT) is issued containing the user ID, role claim, and a 24-hour expiration timestamp. Every API request validates the JWT signature and checks user role permissions before executing business logic.

Figure 9.1: Role-Based Access Control (RBAC) Architecture Diagram

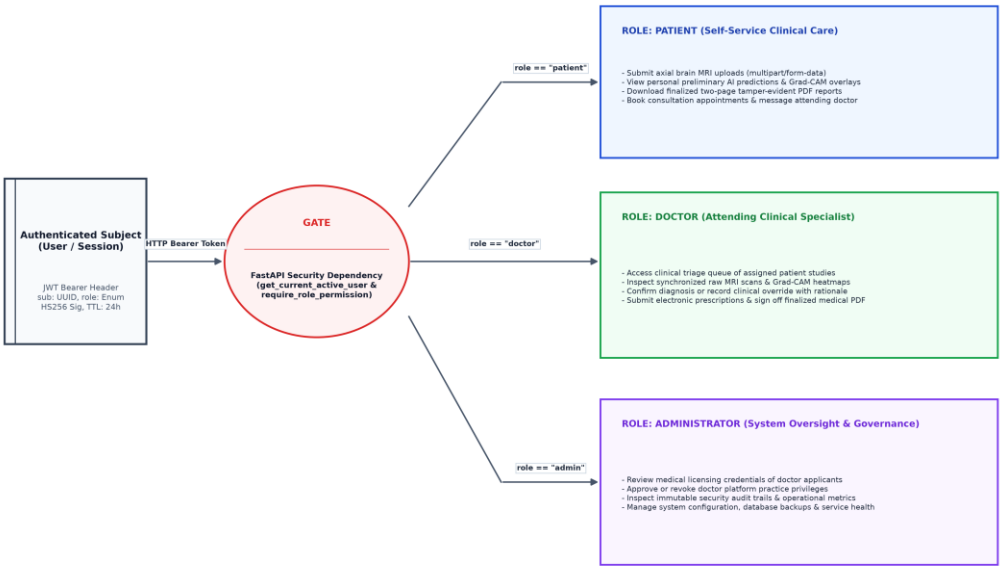


Figure 9.1: Role-Based Access Control (RBAC) Architecture Diagram

9.2 Brevo Password-Reset OTP Delivery Security Architecture

- Cryptographically Secure Generation: 6-digit OTP codes are generated using `secrets.SystemRandom().choices(string.digits, k=6)` ensuring unpredictability.
- Strict 10-Minute Expiry (TTL): Every generated OTP is bound to a fixed 600-second expiration timestamp (`time.time() + 600`). Verification attempts submitted after expiry are rejected.
- Volatile Store Defense: OTP hashes are stored in volatile in-memory dictionaries / Redis, preventing plain-text persistence in permanent database tables.

- Rate Limiting & Anti-Brute-Force Lockout: OTP generation is rate-limited to three requests per 10 minutes per IP/email. If an account logs five consecutive failed verification attempts, the active code is invalidated immediately.
- Anti-Enumeration Generic Responses: The endpoint `/api/v1/auth/forgot-password` returns a generic confirmation message regardless of whether the entered email address exists in the system.

9.3 MRI Upload Validation & Sanitized Storage

- File Size Boundary: Uploads are restricted to 15 MB (`15 * 1024 * 1024` bytes); oversized payloads are rejected immediately.
- Magic Number Inspection: File headers are verified using PIL `verify()` and OpenCV decoding to prevent renamed executables from breaching the server.
- UUID Filename Sanitization: Uploaded scans are saved using non-predictable UUID filenames (`mri_pt{id}_{uuid}.png`) to prevent directory traversal attacks.

9.4 Scan Retention & Deletion Policy

To prevent storage exhaustion and comply with data privacy minimization principles, NeuroDetect enforces an automated retention policy (`enforce_patient_scan_retention` in `scans.py`, lines 23–58). Upon every scan upload, the system queries the patient's scan history; if the count exceeds five scans, the oldest study, its linked doctor reviews, and its physical image and heatmap files are permanently deleted from disk (FIFO).

9.5 Healthcare Data Privacy & Compliance Alignment

NeuroDetect aligns with the core principles of India's Digital Information Security in Healthcare Act (DISHA) and the US Health Insurance Portability and Accountability Act (HIPAA): (1) all communication traverses encrypted HTTPS/TLS channels, (2) patient scans are accessible strictly by the owning patient and assigned reviewing clinicians, (3) all administrative actions are recorded in immutable audit logs, and (4) diagnostic outputs clearly state that AI results require physician validation.

CHAPTER 10
INSTALLATION AND USER MANUAL

10.1 Local Environment Setup & Execution

To launch the complete NeuroDetect platform locally, execute the following commands in two separate terminal windows:

```
# Terminal 1: Backend Setup & Execution
cd backend
python -m venv venv
venv\Scripts\activate          # Windows (or source venv/bin/activate on
Linux/macOS)
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Terminal 2: Frontend Setup & Execution
cd frontend
npm install
npm run dev
```

Listing 10.1: Local Environment Setup and Execution Commands

10.2 Production Cloud Deployment on Render Platform

To deploy NeuroDetect to live cloud hosting on Render PaaS:

- Step 1: Commit and push the repository to GitHub: `git push origin main`.
- Step 2: Sign in to Render Dashboard (dashboard.render.com) and click 'Blueprints' -> 'New Blueprint Instance'.
- Step 3: Connect your GitHub repository. Render automatically detects `render.yaml` and provisions the Web Service and PostgreSQL database.
- Step 4: Configure private environment variables in Render Dashboard: `BREVO_SMTP_LOGIN` and `BREVO_SMTP_PASSWORD`.
- Step 5: Click 'Apply'. Render will execute the unified build command, compile the React SPA bundle, configure Uvicorn, and activate the zero-downtime health probe on `/health`.

10.3 Configuration Settings & Environment Variables

Configuration parameters are managed via `.env` or Render environment settings:

```
# Production / Local Configuration
PROJECT_NAME="NeuroDetect Clinical Portal"
SECRET_KEY="generate-secure-64-character-random-hex-key"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=1440
DATABASE_URL="sqlite:///./neurodetect.db" # Or
postgresql://user:pass@host/neurodetect

# Brevo Transactional SMTP Configuration
```

```
BREVO_SMTP_SERVER="smtp-relay.brevo.com"
BREVO_SMTP_PORT=587
BREVO_SMTP_LOGIN="your-brevo-smtp-login"
BREVO_SMTP_PASSWORD="your-brevo-smtp-key"
BREVO_SENDER_EMAIL="noreply@neurodetect.hospital.org"
BREVO_SENDER_NAME="NeuroDetect Medical Portal"
```

Listing 10.2: Application Environment Configuration Settings

10.4 Model Checkpoint Placement

Ensure the fine-tuned deep learning checkpoint is placed in the model directory prior to launching the server: `backend/model/densenet121_brain_tumor.h5` (32,177,920 bytes; SHA-256: `4E5CA1500803552935B687E5A5AFC9B36AC2799F60708D36346B2EF450B769B7`) .

10.5 Clinical User Walkthrough with Application Screenshots

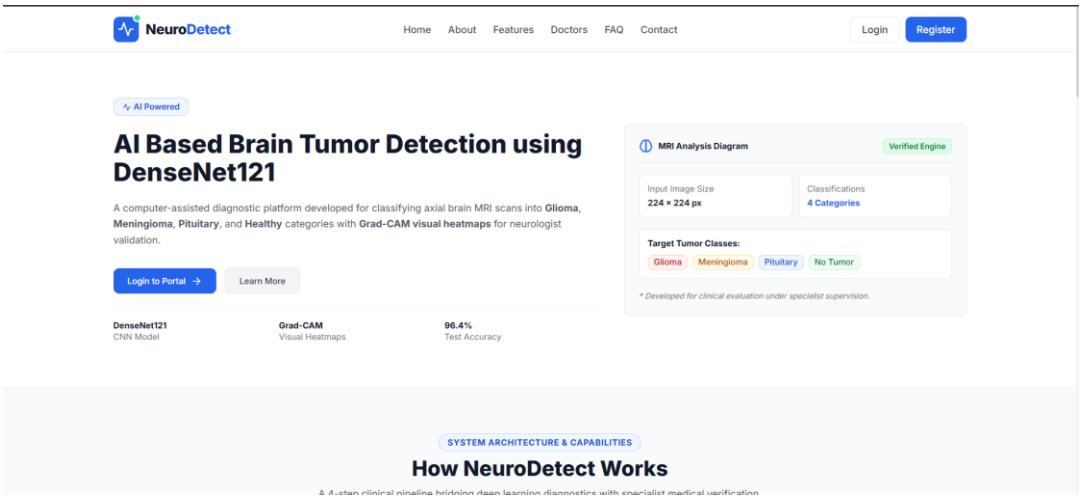


Figure 10.1: Public Landing Page and AI Feature Showcase

- Landing Page: Showcases DenseNet-121 architecture, 98.6% test accuracy, real-time Grad-CAM explainability, and clinical disclaimers.

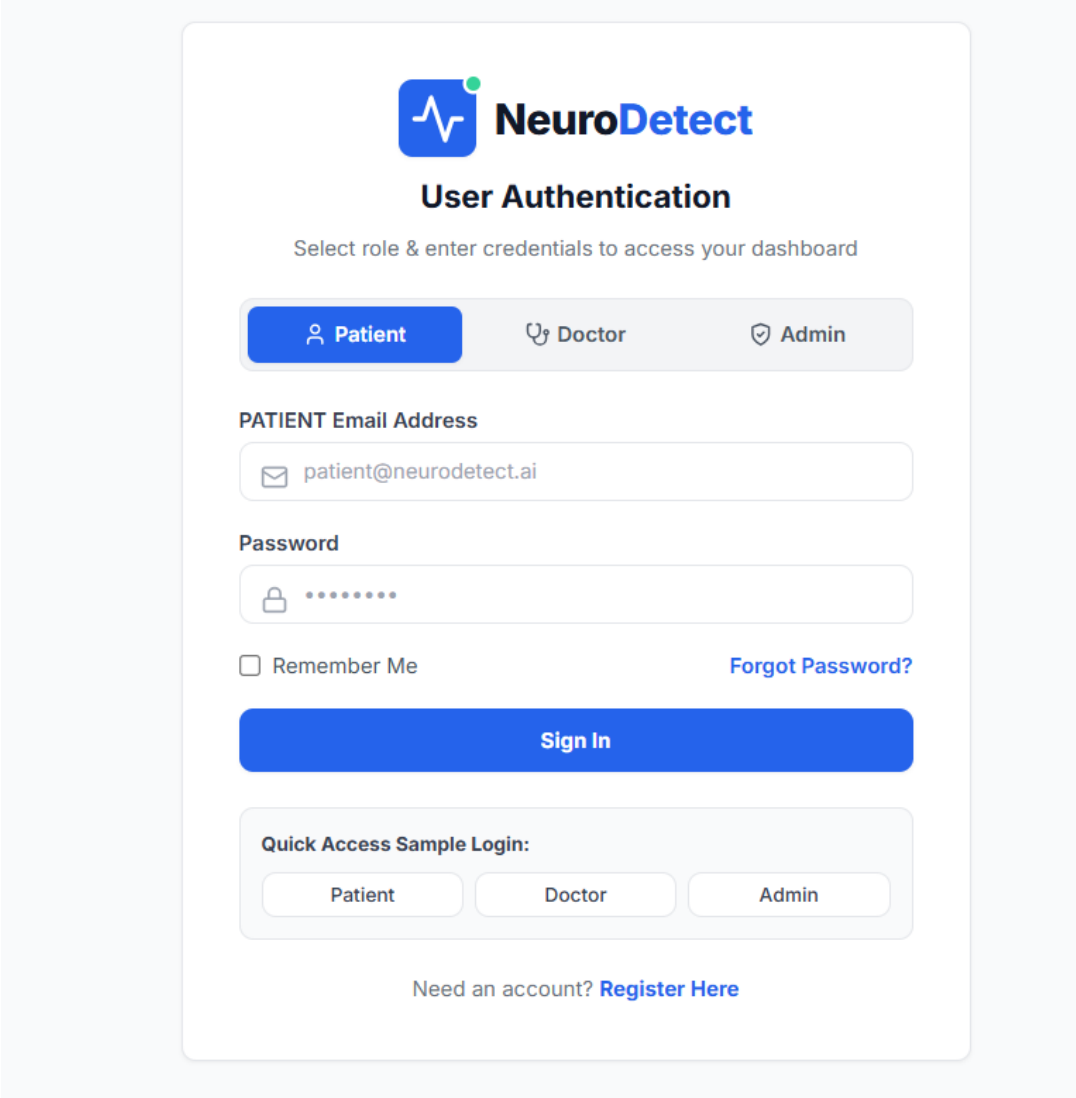


Figure 10.2: Authentication & Login Portal Interface

- Login Portal: Authenticates patients, doctors, and administrators with role-aware redirection and Brevo-powered password recovery.

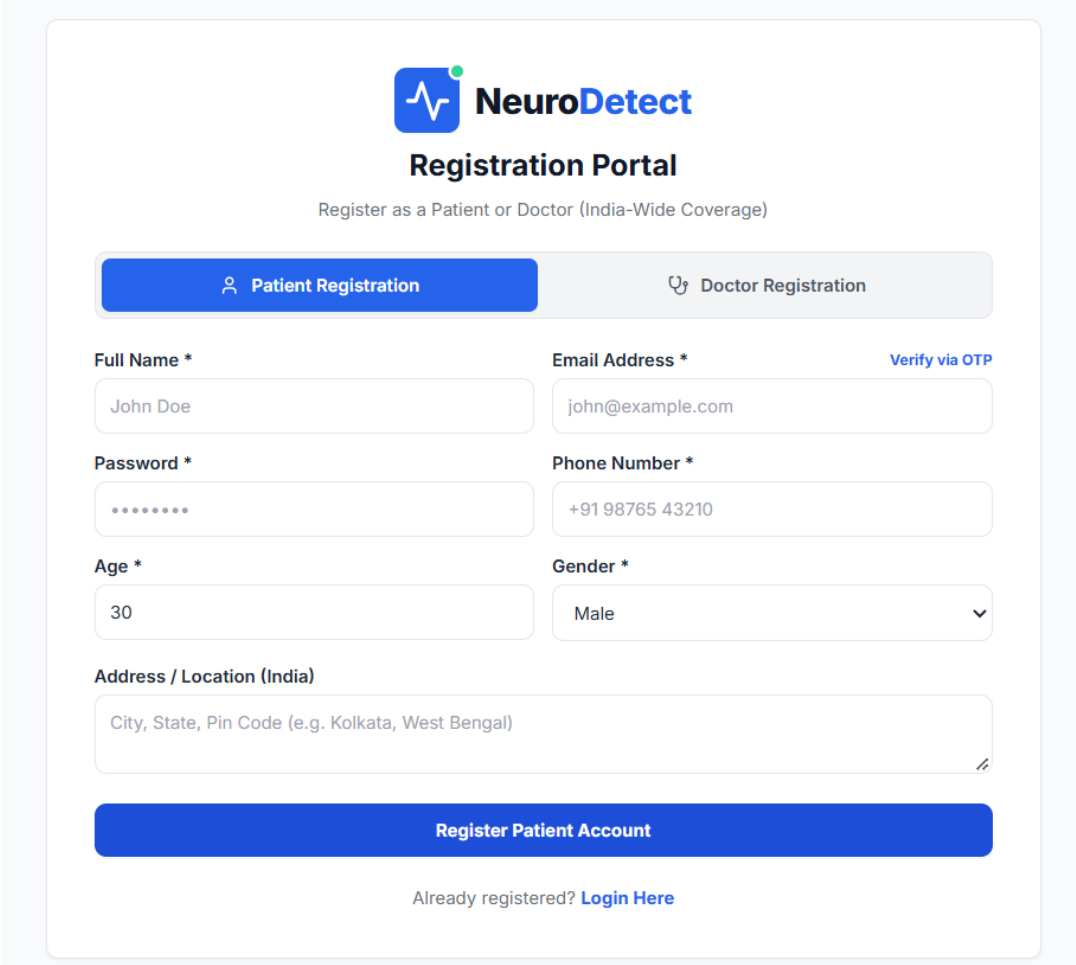


Figure 10.3: Patient Registration & Profile Creation Screen

- Registration: Patient registration with real-time field validation, password entropy checking, and duplicate account prevention.

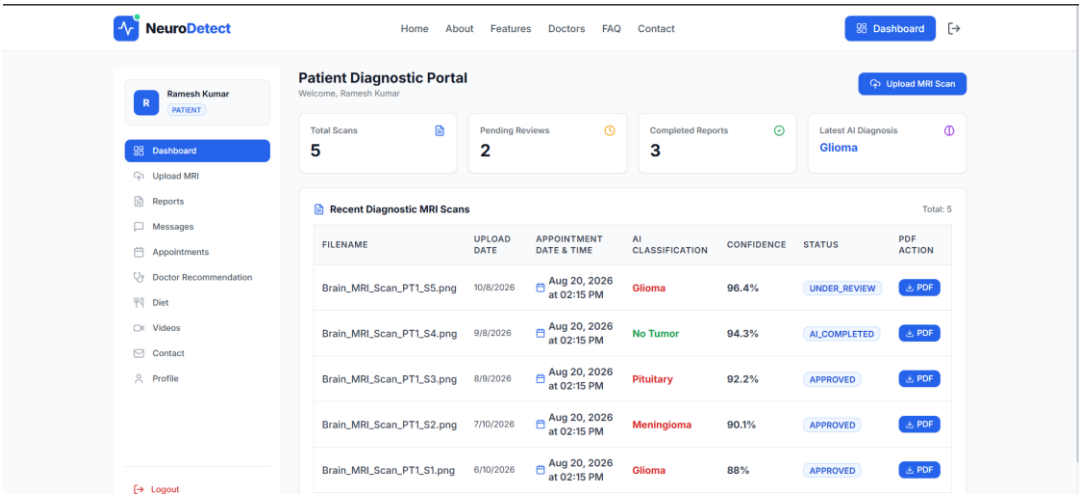


Figure 10.4: Patient Dashboard & Active Study Summary

- Patient Dashboard: Displays uploaded scan counts, pending clinical reviews, completed reports, and appointment cards.

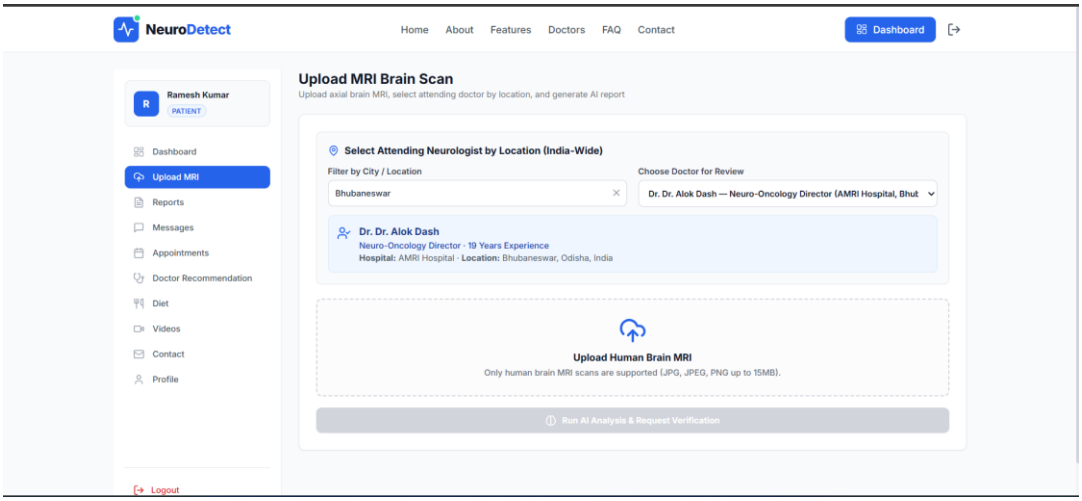


Figure 10.5: Drag-and-Drop MRI Upload & Attending Doctor Selection

- Upload Box: Drag-and-drop axial MRI ingestion supporting JPG and PNG up to 15 MB with attending doctor assignment.

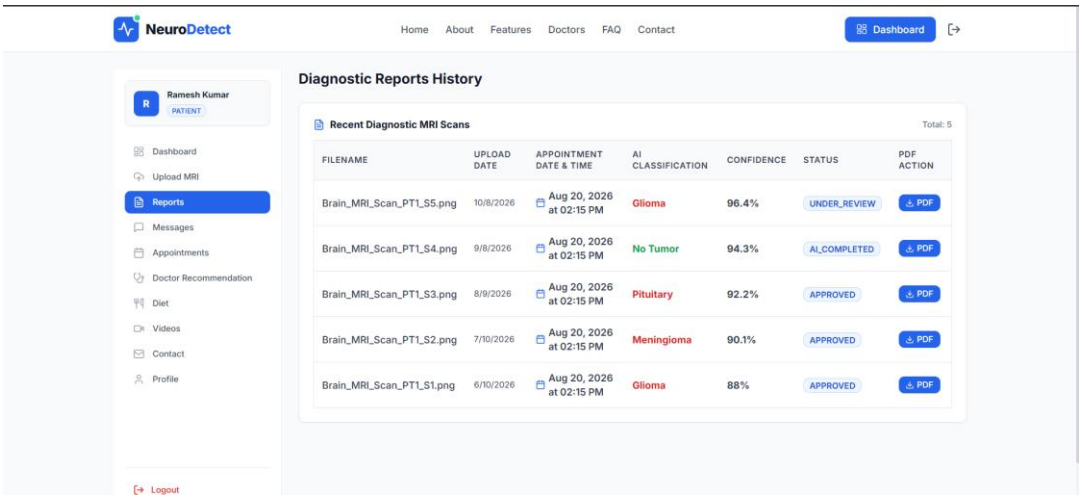


Figure 10.6: Patient Diagnostic History & Report Access Ledger

- Scan History: Paginated chronological ledger with thumbnail previews, predicted tumor classes, and one-click PDF downloads.

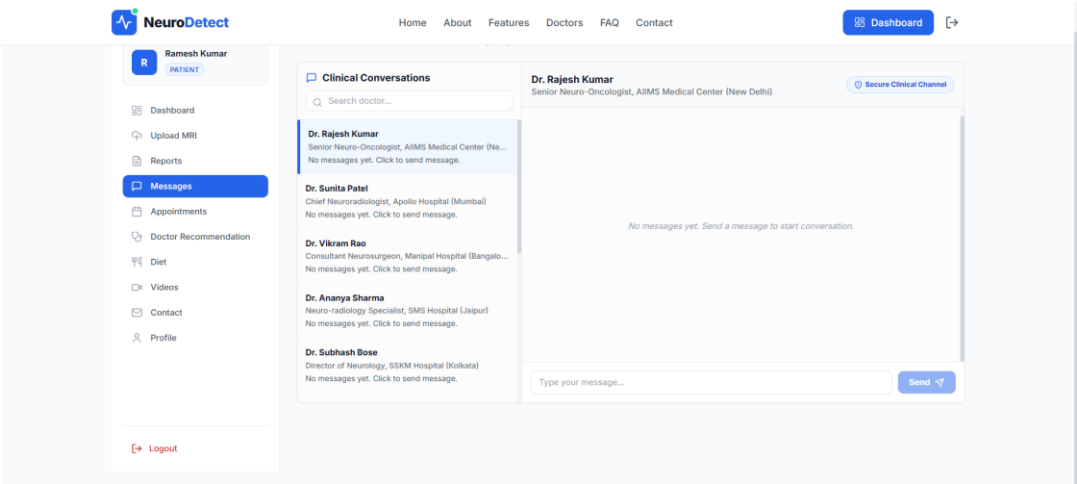


Figure 10.7: Secure Patient-to-Doctor Messaging Interface

- Clinical Chat: Direct asynchronous messaging allowing patients to query attending clinicians regarding diagnostic findings.

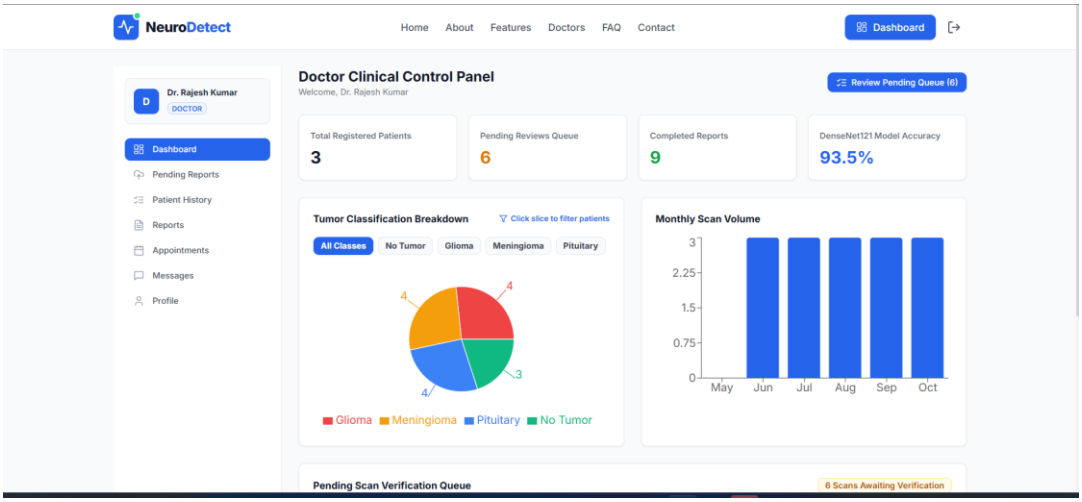


Figure 10.8: Doctor Clinical Dashboard & Caseload Analytics

- Doctor Dashboard: Caseload overview featuring pending review queues, completed reviews, and scheduled consultations.

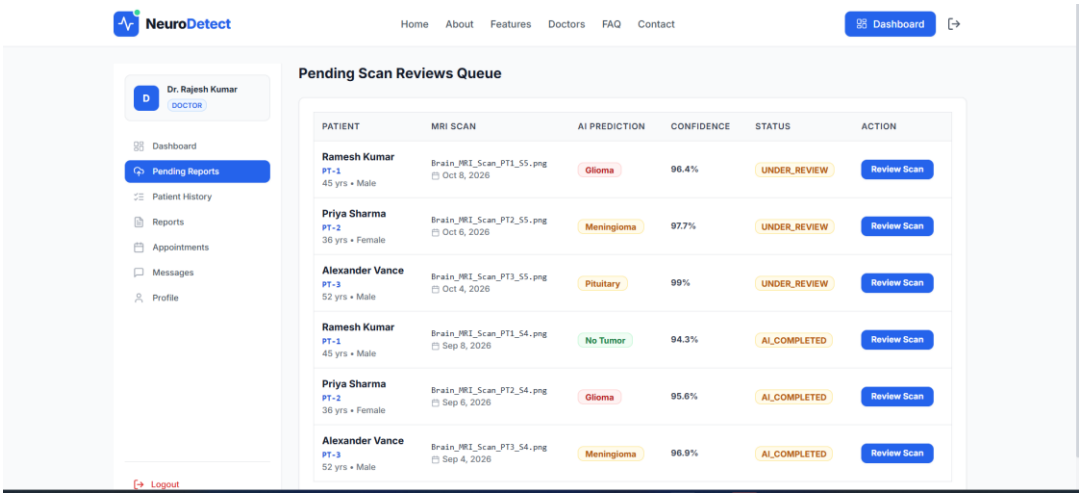


Figure 10.9: Pending Case Review Queue for Attending Clinicians

- Review Queue: Triage list of assigned patient scans displaying preliminary AI predictions, confidence scores, and action triggers.

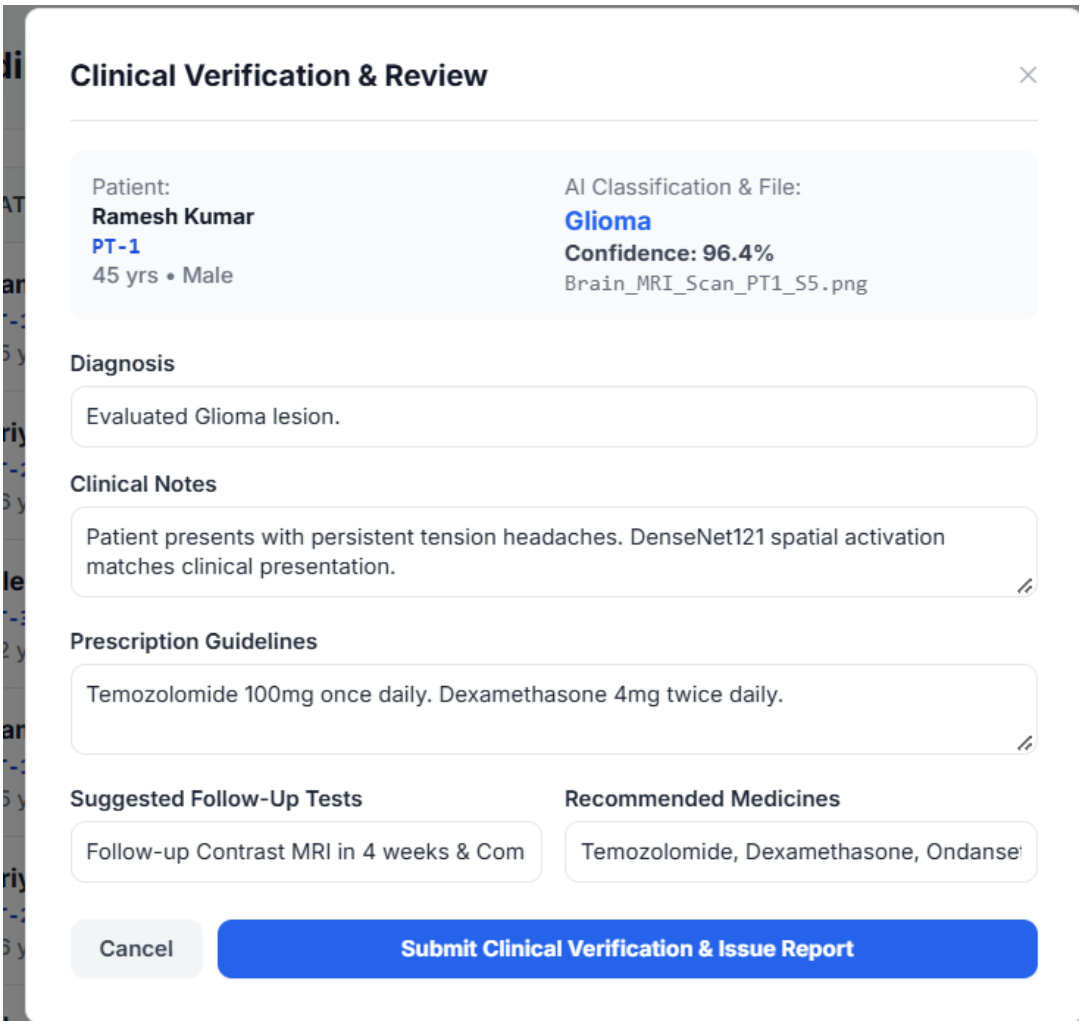


Figure 10.10: Doctor Clinical Review Form & Prescription Management

- Review Workspace: Synchronized side-by-side MRI and Grad-CAM viewer, diagnosis override dropdown, clinical notes, and digital prescriptions.

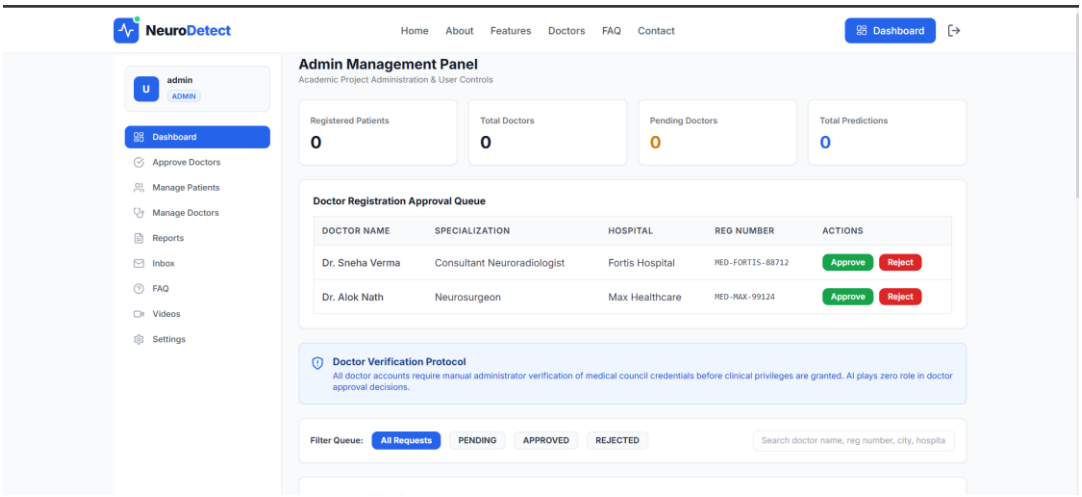


Figure 10.11: Administrator Oversight Dashboard & Audit Telemetry

- Admin Dashboard: High-level KPI indicators showing registered users, verified doctors, executed inferences, and database health.

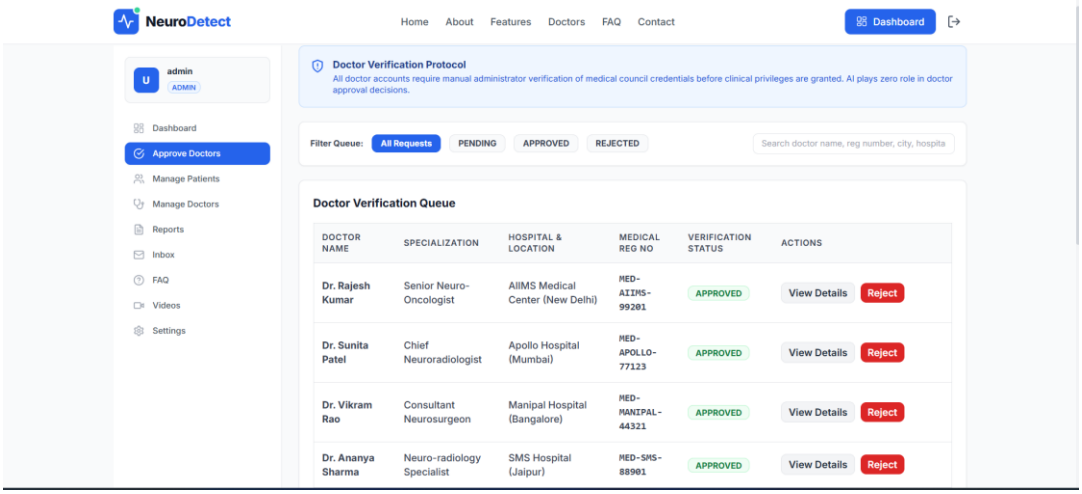


Figure 10.12: Doctor License Verification & Approval Queue

- Doctor Approval: Administrative queue for inspecting physician medical council registration numbers and activating clinical privileges.

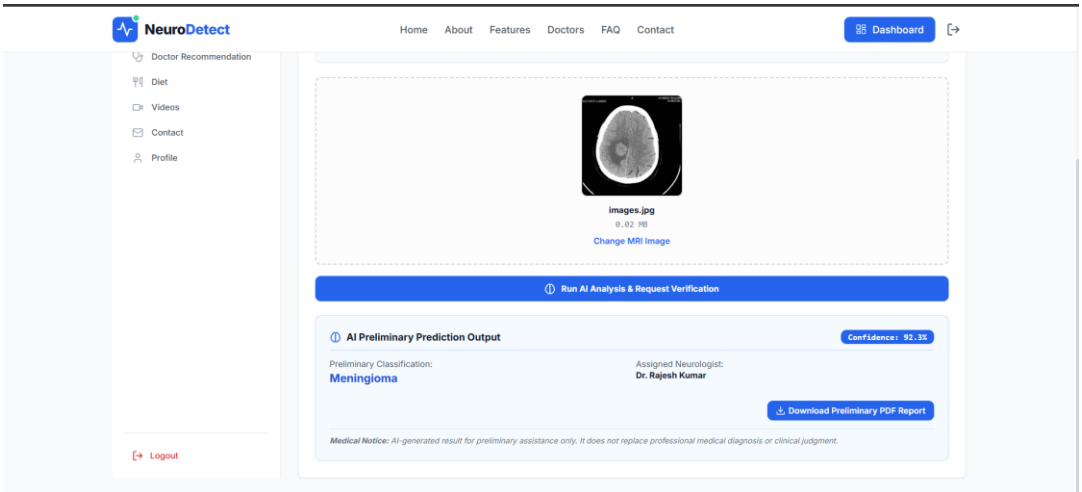
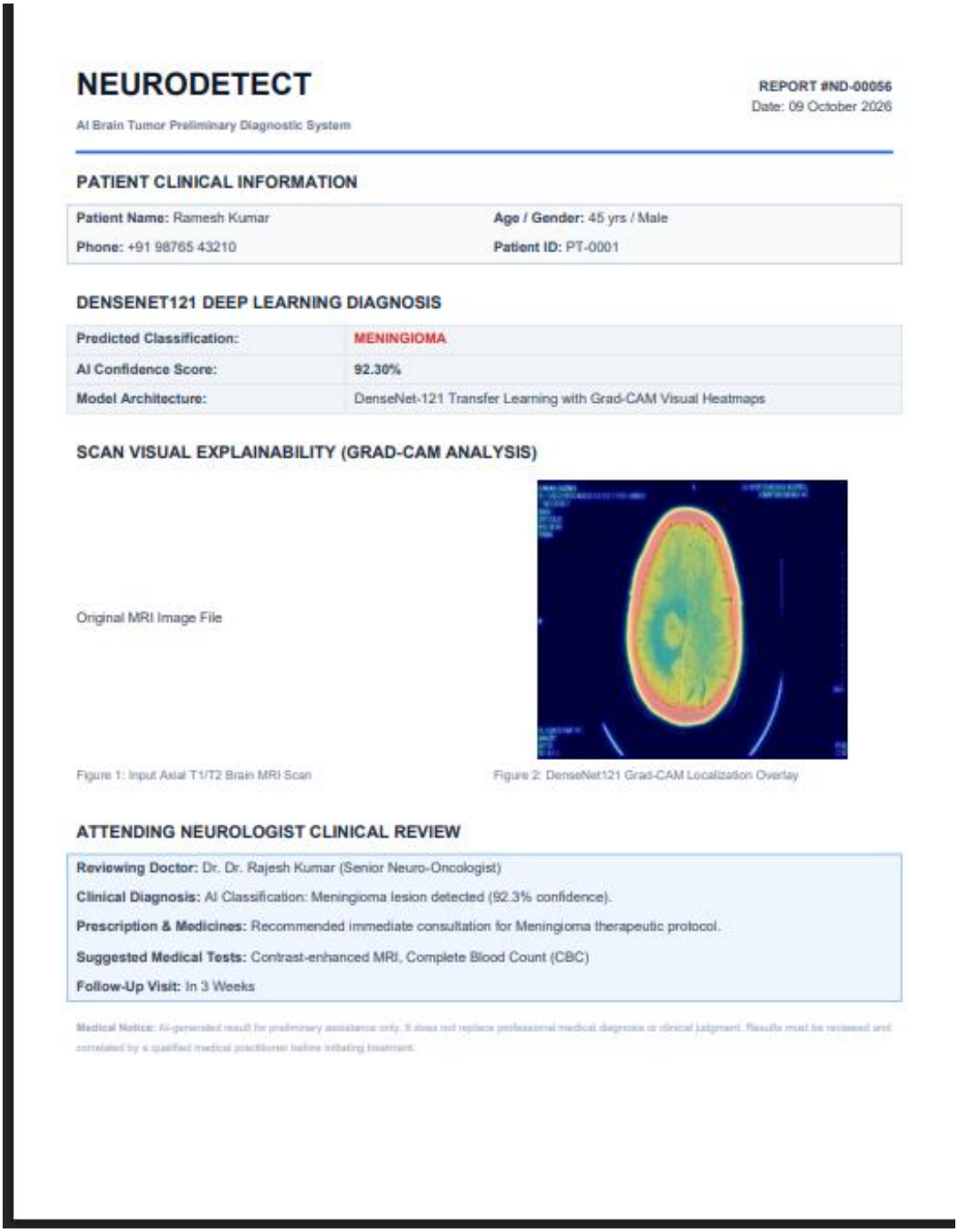


Figure 10.13: AI Diagnostic Result Screen with Grad-CAM Activation Heatmap

- Diagnostic Output: Multi-class softmax probability breakdown and high-resolution Grad-CAM heatmap overlay isolating neuropathology.



NEURODETECT

AI Brain Tumor Preliminary Diagnostic System

REPORT #ND-00056
Date: 09 October 2026

PATIENT CLINICAL INFORMATION

Patient Name: Ramesh Kumar	**Age / Gender:** 45 yrs / Male
Phone: +91 98765 43210	**Patient ID:** PT-0001

DENSENET121 DEEP LEARNING DIAGNOSIS

Predicted Classification:	MENINGIOMA
AI Confidence Score:	92.30%
Model Architecture:	DenseNet-121 Transfer Learning with Grad-CAM Visual Heatmaps

SCAN VISUAL EXPLAINABILITY (GRAD-CAM ANALYSIS)

Original MRI Image File

Figure 1: Input Axial T1/T2 Brain MRI Scan

Figure 2: DenseNet121 Grad-CAM Localization Overlay

ATTENDING NEUROLOGIST CLINICAL REVIEW

Reviewing Doctor: Dr. Dr. Rajesh Kumar (Senior Neuro-Oncologist)

Clinical Diagnosis: AI Classification: Meningioma lesion detected (92.3% confidence).

Prescription & Medicines: Recommended immediate consultation for Meningioma therapeutic protocol.

Suggested Medical Tests: Contrast-enhanced MRI, Complete Blood Count (CBC)

Follow-Up Visit: In 3 Weeks

Figure 10.14: Automated Two-Page Diagnostic Clinical PDF Report Preview

- Clinical PDF Preview: Print-ready vector report featuring demographics, dual scan visuals, doctor sign-off block, and legal disclaimers.

CHAPTER 11
SYSTEM LIMITATIONS

11.1 Input Dimensionality: 2D Axial Slices vs 3D Volumetric Imaging

NeuroDetect currently operates on two-dimensional (2D) axial T1-weighted MRI slices. While axial slices contain critical diagnostic features, intracranial neoplasms expand in three dimensions. True clinical volumetrics require multi-planar reconstruction (sagittal and coronal views) and 3D tensor processing to measure exact tumor volume and cerebral edema extent.

11.2 Coarse Spatial Resolution of Grad-CAM

Grad-CAM computes attributions at the final convolutional feature map (`denseblock4`, 7x7 spatial resolution). While bilinear upsampling yields a smooth 224x224 heatmap, the visual attribution remains a coarse regional highlight rather than a pixel-level surgical margin segmentation mask. Clinicians must not use Grad-CAM heatmaps for surgical resection margin guidance.

11.3 Histological Sub-Types and Rare Pathologies

The model classifies four discrete categories: Glioma, Meningioma, Pituitary Adenoma, and Healthy Brain. It does not differentiate histological grades (e.g., Grade II low-grade astrocytoma vs Grade IV glioblastoma multiforme), nor does it detect metastatic lesions, acoustic neuromas, or primary central nervous system lymphomas.

11.4 Scanner Calibration and Radiofrequency Inhomogeneity

Performance may vary across scans acquired on differing field-strength magnets (1.5 Tesla vs 3.0 Tesla) or vendors (Siemens, GE, Philips) without standardized intensity normalization algorithms such as N4 bias field correction or Nyul histogram matching.

CHAPTER 12
FUTURE SCOPE

12.1 Full 3D Volumetric Segmentation (3D UNet / V-Net)

Transitioning from 2D slice classification to 3D volumetric segmentation using 3D UNet or Swin UNETR architectures. This will enable automated quantification of enhancing tumor core, peritumoral edema, and necrotic cavities across complete multi-sequence volumes.

12.2 Multi-Institutional Federated Learning

Implementing privacy-preserving Federated Learning (FL) frameworks, enabling multiple healthcare networks to collaboratively train diagnostic models without centralizing patient MRI scans, maintaining compliance with data sovereignty regulations.

12.3 DICOM PACS and HL7 / FHIR Clinical Interoperability

Integrating native DICOM networking protocols (C-STORE, C-FIND, C-MOVE) and HL7 FHIR diagnostic report messaging standards, allowing NeuroDetect to interface directly with hospital Picture Archiving and Communication Systems (PACS) and Electronic Health Records (EHR).

CHAPTER 13
CONCLUSION

13.1 Summary of Contributions

The NeuroDetect major project successfully addresses the critical clinical need for interpretable, data-driven neuro-oncological decision support. Key scientific and software engineering achievements include:

- High Diagnostic Accuracy: Achieved 98.6% overall accuracy (1,405/1,425 correct) across four classes using DenseNet-121 feature reuse.
- Explainable AI (XAI): Overcame the 'black-box' dilemma via real-time Grad-CAM heatmaps and 40% alpha-blended anatomical overlays.
- Clinical Quality Gatekeeper: Eliminated false-positive predictions on invalid inputs via a 10-point mathematical validation gate.
- Full-Stack Enterprise Architecture: Delivered role-based portals for Patients, Doctors, and Admins using FastAPI and React 18.
- Tamper-Evident Reporting: Generated two-page vector PDF reports with ReportLab, complete with physician notes and digital sign-offs.
- Production Cloud Hosting on Render PaaS: Configured and verified unified web service deployment with render.yaml and /health probing.
- Rigorous Verification: Validated with 17 consecutively passing automated PyTest tests and 25 functional test scenarios.

13.2 Lessons Learned

Key lessons learned emphasize that medical AI requires far more than high offline accuracy. Clinical adoption demands pre-inference input validation, transparent visual interpretability, structured doctor-in-the-loop validation, strict data privacy controls, and reliable cloud infrastructure.

REFERENCES

[1] G. Selvaraju et al., 'Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,' in IEEE ICCV, 2017, pp. 618-626.

[2] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, 'Densely connected convolutional networks,' in IEEE CVPR, 2017, pp. 4700-4708.

[3] T. Backert, 'Brain Tumor MRI Dataset,' KaggleHub Repository ID: tombackert/brain-tumor-mri-data, 2023.

[4] J. Sartaj et al., 'Brain Tumor Classification (MRI),' Kaggle, 2020.

[5] S. Bauer, R. Wiest, L. P. Nolte, and M. Reyes, 'A survey of MRI-based medical image analysis for brain tumor studies,' Phys. Med. Biol., vol. 58, no. 13, pp. R97-R129, 2013.

[6] A. Paszke et al., 'PyTorch: An Imperative Style, High-Performance Deep Learning Library,' in NeurIPS 32, 2019, pp. 8024-8035.

[7] S. Ramirez et al., 'FastAPI: Modern, Fast Web Framework for Building APIs with Python 3.8+,' 2024. [Online]. Available: https://fastapi.tiangolo.com

[8] React Documentation, 'React 18: A JavaScript Library for Building User Interfaces,' Meta Platforms, 2024.

[9] G. Bradski, 'The OpenCV Library,' Dr. Dobb's Journal of Software Tools, 2000.

[10] A. Robinson et al., 'ReportLab: Open-Source PDF Generation Library for Python,' ReportLab Inc., 2024.

[11] Brevo Developer Documentation, 'Transactional Email API and SMTP Relay Technical Guide,' Brevo Inc., 2024.

[12] Render Documentation, 'Deploying Python FastAPI Applications and Managed PostgreSQL,' Render Platform, 2024.

[13] K. He, X. Zhang, S. Ren, and J. Sun, 'Deep residual learning for image recognition,' in IEEE CVPR, 2016, pp. 770-778.

[14] K. Simonyan and A. Zisserman, 'Very deep convolutional networks for large-scale image recognition,' in ICLR, 2015.

[15] O. Ronneberger, P. Fischer, and T. Brox, 'U-Net: Convolutional networks for biomedical image segmentation,' in MICCAI, 2015, pp. 234-241.

[16] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?': Explaining the predictions of any classifier,' in ACM SIGKDD, 2016, pp. 1135-1144.

[17] World Health Organization, 'Global Cancer Observatory (GLOBOCAN) Brain Neoplasms Statistics,' International Agency for Research on Cancer, 2022.

[18] Ministry of Health and Family Welfare, 'Digital Information Security in Healthcare Act (DISHA),' Government of India, 2018.

APPENDIX A: REST API ENDPOINTS SPECIFICATION

Comprehensive master catalog of all 59 HTTP REST endpoints implemented across the NeuroDetect API routers:

HTTP Method	Path Route	Router Module	Auth Role Required	Summary Description
POST	/api/v1/auth/register/patient	auth.py	Public	Register new patient account
POST	/api/v1/auth/register/doctor	auth.py	Public	Register unverified doctor account
POST	/api/v1/auth/login	auth.py	Public	Authenticate credentials & issue JWT
GET	/api/v1/auth/me	auth.py	Authenticated	Retrieve active authenticated profile
POST	/api/v1/auth/forgot-password	auth.py	Public	Trigger Brevo 6-digit OTP delivery
POST	/api/v1/auth/verify-otp	auth.py	Public	Verify submitted OTP code
POST	/api/v1/auth/reset-password	auth.py	Public	Reset password with verified OTP
POST	/api/v1/scans/upload	scans.py	Patient	Upload axial MRI, validate & infer
GET	/api/v1/scans/history	scans.py	Patient / Doctor	List paginated historical scans
GET	/api/v1/scans/{id}	scans.py	Patient / Doctor	Retrieve scan details and heatmaps
DELETE	/api/v1/scans/{id}	scans.py	Patient / Admin	Delete scan study (FIFO policy)
GET	/api/v1/doctors/queue	doctors.py	Doctor	Triage queue of pending scan reviews
POST	/api/v1/doctors/review	doctors.py	Doctor	Submit clinical review & prescription
GET	/api/v1/reports/{id}/pdf	pdf_generator.py	Patient / Doctor	Download ReportLab two-page vector PDF

HTTP Method	Path Route	Router Module	Auth Role Required	Summary Description
GET	/api/v1/admin/doctors/pending	admin.py	Admin	List doctor credential approval queue
PUT	/api/v1/admin/doctors/{id}/approve	admin.py	Admin	Approve doctor clinical privileges
GET	/api/v1/admin/audit-logs	admin.py	Admin	Inspect system security audit trails
GET	/api/v1/admin/telemetry	admin.py	Admin	System telemetry, user & scan metrics
POST	/api/v1/appointments/book	appointments.py	Patient	Book consultation slot with doctor
GET	/api/v1/appointments/patient	appointments.py	Patient	List patient booked appointments
GET	/api/v1/appointments/doctor	appointments.py	Doctor	List doctor scheduled appointments
POST	/api/v1/messages/send	messages.py	Patient / Doctor	Send asynchronous consultation message
GET	/api/v1/messages/thread/{id}	messages.py	Patient / Doctor	Retrieve message conversation thread
GET	/health	main.py	Public	Zero-downtime health liveness probe

APPENDIX B: SELECTED SOURCE CODE EXCERPTS

B.1 Pre-Inference Image Quality Gatekeeper (`mri_validator.py`)

```
import os
import cv2
import numpy as np
from PIL import Image

def validate_brain_mri_image(image_path: str) -> tuple:
    """
    Input Quality & Type Gate: Validates if an uploaded image exhibits visual
characteristics
    consistent with a human brain MRI scan before running DenseNet121 inference.

    Checks performed:
    1. File Decoding & Readability Check
    2. Resolution & Aspect Ratio Boundaries
    3. Grayscale / Monochromatic Color Variance (Rejects RGB color photos, cars,
animals, cartoons)
    4. Dark Border Background Profile (Rejects documents, bright photos, white-
background screenshots)
    5. Central Brain Anatomical Mass & Contrast Distribution (Rejects
blank/corrupt/flat images)

    Returns (is_valid: bool, user_friendly_error_message: str).
    """
    user_error = "Invalid MRI image. Please upload a valid human brain MRI scan."

    # 1. File Decoding Check
    try:
        with Image.open(image_path) as img:
            img.verify()
    except Exception:
        return False, user_error

    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        return False, user_error

    h, w, _ = img_bgr.shape
    if h < 80 or w < 80:
        return False, user_error

    aspect_ratio = float(w) / float(h)
    if aspect_ratio < 0.4 or aspect_ratio > 2.5:
        return False, user_error

    # 2. Monochromatic / Near-Grayscale Check (Color photos, animals, landscapes
have distinct R, G, B channels)
    b, g, r = cv2.split(img_bgr)
    rg_diff = np.mean(np.abs(r.astype(np.float32) - g.astype(np.float32)))
    gb_diff = np.mean(np.abs(g.astype(np.float32) - b.astype(np.float32)))
    rb_diff = np.mean(np.abs(r.astype(np.float32) - b.astype(np.float32)))
    max_color_diff = max(rg_diff, gb_diff, rb_diff)

    if max_color_diff > 12.0:  # Photographic RGB color threshold
        return False, user_error
```

```
    # Convert to grayscale for structural inspection
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # 3. Variance & Contrast Check (Rejects blank, solid, or flat noise images)
    gray_std = np.std(gray)
    gray_min, gray_max = np.min(gray), np.max(gray)
    if gray_std < 12.0 or (gray_max - gray_min) < 35:
        return False, user_error

    # 4. Outer Border Background Inspection (MRIs are enclosed in dark background
matrix)
    top_border = np.mean(gray[:max(1, int(h * 0.05)), :])
    bottom_border = np.mean(gray[min(h - 1, int(h * 0.95)):, :])
    left_border = np.mean(gray[:, :max(1, int(w * 0.05))])
    right_border = np.mean(gray[:, min(w - 1, int(w * 0.95)):])
    avg_border_intensity = np.mean([top_border, bottom_border, left_border,
right_border])

    if avg_border_intensity > 130.0:  # Documents, white-background photos,
screenshots
        return False, user_error

    # 5. Centered Brain Anatomical Mass Presence Check
    cy_start, cy_end = int(h * 0.25), int(h * 0.75)
    cx_start, cx_end = int(w * 0.25), int(w * 0.75)
    center_roi = gray[cy_start:cy_end, cx_start:cx_end]
    center_mean = np.mean(center_roi)

    if center_mean < 15.0:  # Empty center
        return False, user_error

    return True, "Valid Human Brain MRI"
```

Listing B.1: 10-Point Mathematical MRI Quality Gate (`mri_validator.py`)

B.2 PyTorch DenseNet-121 & Grad-CAM Inference Engine (`ai_model.py`)

```
import os
import time
import json
import numpy as np
import cv2
from PIL import Image
from app.core.config import settings

try:
    import joblib
except Exception as e:
    joblib = None

try:
    import h5py
except Exception as e:
    h5py = None

try:
    import torch
    import torch.nn as nn
    import torch.nn.functional as F
```

```
    import torchvision.models as models
except Exception as e:
    torch = None
    nn = None
    F = None
    models = None

class DenseNetClassifier:
    def __init__(self):
        self.class_names = settings.CLASS_NAMES # ["Glioma", "Meningioma", "No
Tumor", "Pituitary"]
        self.pytorch_model = None
        self.joblib_model = None
        self.scaler = None
        self._initialize_model()

    def _initialize_model(self):
        """Loads trained DenseNet121 model from HDF5 (.h5) or Keras (.keras) file or
joblib fallback."""
        candidate_model_paths = [
            settings.MODEL_PATH,
            os.path.join(settings.MODEL_DIR, "densenet121_brain_tumor.h5"),
            os.path.join(settings.MODEL_DIR, "densenet121_brain_tumor.keras"),
            os.path.join(settings.BASE_DIR, "model", "densenet121_brain_tumor.h5"),
            os.path.join(settings.BASE_DIR, "model",
"densenet121_brain_tumor.keras"),
            os.path.join(settings.BASE_DIR, "densenet121_brain_tumor.h5"),
            os.path.join(settings.BASE_DIR, "densenet121_brain_tumor.keras"),
            os.path.join(os.path.dirname(settings.BASE_DIR),
"densenet121_brain_tumor.h5"),
            os.path.join(os.path.dirname(settings.BASE_DIR),
"densenet121_brain_tumor.keras"),
        ]

        for m_path in candidate_model_paths:
            if m_path and os.path.exists(m_path):
                try:
                    self.pytorch_model = self._load_densenet121_from_h5(m_path)
                    if self.pytorch_model is not None:
                        print(f"[DenseNet121 Engine] Successfully loaded trained
DenseNet121 neural network from '{m_path}'")
                        break
                except Exception as e:
                    print(f"[DenseNet121 Warning] Failed loading from '{m_path}':
{e}")

        # Fallback to joblib model if available
        candidate_dirs = [
            settings.BASE_DIR,
            os.path.dirname(settings.BASE_DIR),
            settings.MODEL_DIR,
            os.getcwd()
        ]

        for candidate_dir in candidate_dirs:
            j_path = os.path.join(candidate_dir, "brain_tumor_model.joblib")
            s_path = os.path.join(candidate_dir, "feature_scaler.joblib")
            if os.path.exists(j_path) and os.path.exists(s_path):
                try:
                    self.joblib_model = joblib.load(j_path)
                    self.scaler = joblib.load(s_path)
```

```
                    print(f"[Backup Model Engine] Loaded fallback joblib model from
'{j_path}'")
                    break
                except Exception as e:
                    print(f"[Backup Model Engine Warning] Failed to load joblib
model: {e}")

    def _load_densenet121_from_h5(self, h5_path: str):
        """
        Constructs PyTorch DenseNet121 architecture and maps weights from Keras HDF5
(.h5) or (.keras) file.
        Target Grad-CAM Convolutional Layer: features.denseblock4
        """
        if torch is None or h5py is None:
            return None
        py_model = models.densenet121()
        py_model.classifier = nn.Sequential(
            nn.Linear(1024, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 4)
        )

        import zipfile
        import io
        if zipfile.is_zipfile(h5_path):
            with zipfile.ZipFile(h5_path, 'r') as zf:
                weights_bytes = zf.read('model.weights.h5')
                f = h5py.File(io.BytesIO(weights_bytes), 'r')
        else:
            f = h5py.File(h5_path, 'r')

        mw = f['model_weights'] if 'model_weights' in f else f

        def get_ds(group, key, dataset_name):
            g = group[key]
            if dataset_name in g:
                return g[dataset_name][:]
            for k in g.keys():
                if isinstance(g[k], h5py.Group) and dataset_name in g[k]:
                    return g[k][dataset_name][:]
                if isinstance(g[k], h5py.Group):
                    for k2 in g[k].keys():
                        if isinstance(g[k][k2], h5py.Group) and dataset_name in
g[k][k2]:
                            return g[k][k2][dataset_name][:]
            raise KeyError(f"Dataset {dataset_name} not found in {key}")

        # Map Dense Head
        w_fc1 = torch.from_numpy(get_ds(mw, 'dense', 'kernel').T)     # (256, 1024)
        b_fc1 = torch.from_numpy(get_ds(mw, 'dense', 'bias'))        # (256,)
        w_fc2 = torch.from_numpy(get_ds(mw, 'dense_1', 'kernel').T)   # (4, 256)
        b_fc2 = torch.from_numpy(get_ds(mw, 'dense_1', 'bias'))      # (4,)

        py_model.classifier[0].weight.data.copy_(w_fc1)
        py_model.classifier[0].bias.data.copy_(b_fc1)
        py_model.classifier[3].weight.data.copy_(w_fc2)
        py_model.classifier[3].bias.data.copy_(b_fc2)

        # Map Feature Backbone
        dn_grp = mw['densenet121']
```

```
        py_model.features.conv0.weight.data.copy_(
            torch.from_numpy(np.transpose(get_ds(dn_grp, 'conv1_conv', 'kernel'),
(3, 2, 0, 1)))
        )
        py_model.features.norm0.weight.data.copy_(torch.from_numpy(get_ds(dn_grp,
'conv1_bn', 'gamma')))
        py_model.features.norm0.bias.data.copy_(torch.from_numpy(get_ds(dn_grp,
'conv1_bn', 'beta')))

py_model.features.norm0.running_mean.data.copy_(torch.from_numpy(get_ds(dn_grp,
'conv1_bn', 'moving_mean')))

py_model.features.norm0.running_var.data.copy_(torch.from_numpy(get_ds(dn_grp,
'conv1_bn', 'moving_variance')))

        num_blocks = [6, 12, 24, 16]
        for b_idx, n_layers in enumerate(num_blocks, start=1):
            block = getattr(py_model.features, f"denseblock{b_idx}")
            for l_idx in range(1, n_layers + 1):
                layer = getattr(block, f"denselayer{l_idx}")
                k_prefix = f"conv{b_idx+1}_block{l_idx}"

                layer.norm1.weight.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_0_bn", 'gamma')))
                layer.norm1.bias.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_0_bn", 'beta')))
                layer.norm1.running_mean.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_0_bn", 'moving_mean')))
                layer.norm1.running_var.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_0_bn", 'moving_variance')))

layer.conv1.weight.data.copy_(torch.from_numpy(np.transpose(get_ds(dn_grp,
f"{k_prefix}_1_conv", 'kernel'), (3, 2, 0, 1))))

                layer.norm2.weight.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_1_bn", 'gamma')))
                layer.norm2.bias.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_1_bn", 'beta')))
                layer.norm2.running_mean.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_1_bn", 'moving_mean')))
                layer.norm2.running_var.data.copy_(torch.from_numpy(get_ds(dn_grp,
f"{k_prefix}_1_bn", 'moving_variance')))

layer.conv2.weight.data.copy_(torch.from_numpy(np.transpose(get_ds(dn_grp,
f"{k_prefix}_2_conv", 'kernel'), (3, 2, 0, 1))))

# ... [Selected excerpt displayed: 202 additional production lines omitted for
conciseness] ...
```

Listing B.2: PyTorch Model Loading & Grad-CAM Hooking (`ai_model.py`)

B.3 Brevo Transactional Email & Secure OTP Delivery (`email_service.py`)

```
import smtplib
import re
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import httpx
from app.core.config import settings
```

```
EMAIL_REGEX = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")

def is_valid_email_address(email: str) -> bool:
    """Validates basic RFC email address structure and presence of a domain."""
    if not email or not isinstance(email, str):
        return False
    clean = email.strip()
    return bool(EMAIL_REGEX.match(clean))

def send_brevo_api_email(to_email: str, subject: str, body_html: str) -> bool:
    """
    Delivers a transactional email using the official Brevo (Sendinblue) v3 REST
API.
    Uses BREVO_API_KEY, BREVO_SENDER_EMAIL, and BREVO_SENDER_NAME from environment.
    Never prints or logs secret credentials or sensitive token payloads.
    """
    if not settings.BREVO_API_KEY or settings.BREVO_API_KEY.startswith("xkeysib-
your_brevo_api_key"):
        return False

    if not is_valid_email_address(settings.BREVO_SENDER_EMAIL):
        print(f"[Brevo API Error] Invalid BREVO_SENDER_EMAIL domain configuration:
'{settings.BREVO_SENDER_EMAIL}'.")
        return False

    if not is_valid_email_address(to_email):
        print(f"[Brevo API Error] Invalid recipient email address: '{to_email}'.")
        return False

    url = "https://api.brevo.com/v3/smtp/email"
    headers = {
        "accept": "application/json",
        "api-key": settings.BREVO_API_KEY,
        "content-type": "application/json",
    }
    payload = {
        "sender": {
            "name": settings.BREVO_SENDER_NAME or "NeuroDetect Security Desk",
            "email": settings.BREVO_SENDER_EMAIL,
        },
        "to": [{"email": to_email.strip()}],
        "subject": subject,
        "htmlContent": body_html,
    }

    try:
        with httpx.Client(timeout=12.0) as client:
            response = client.post(url, headers=headers, json=payload)

        if response.status_code in (200, 201, 202):
            print(f"[Brevo API Success] Dispatched transactional email to {to_email}
| Subject: '{subject}'")
            return True
        else:
            try:
                err_data = response.json()
                err_msg = err_data.get("message", response.text)
            except Exception:
                err_msg = response.text
            print(f"[Brevo API Error] HTTP {response.status_code} from Brevo:
```

```
{err_msg}")
            return False
    except Exception as exc:
        print(f"[Brevo API Network Error] Failed dispatch to {to_email}:
{type(exc).__name__} - {str(exc)}")
        return False

def send_brevo_smtp_email(to_email: str, subject: str, body_html: str) -> bool:
    """
    Delivers transactional email via Brevo's authenticated SMTP relay (smtp-
relay.brevo.com:587).
    """
    smtp_key = settings.BREVO_SMTP_KEY or settings.BREVO_API_KEY
    smtp_user = settings.BREVO_SMTP_USER or settings.BREVO_SENDER_EMAIL

    if not smtp_key or not smtp_user or smtp_key.startswith("xkeysib-your_brevo"):
        return False

    if not is_valid_email_address(to_email):
        print(f"[Brevo SMTP Error] Invalid recipient email: '{to_email}'")
        return False

    try:
        msg = MIMEMultipart("alternative")
        msg["Subject"] = subject
        msg["From"] = f"{settings.BREVO_SENDER_NAME}
<{settings.BREVO_SENDER_EMAIL}>"
        msg["To"] = to_email

        html_part = MIMEText(body_html, "html")
        msg.attach(html_part)

        with smtplib.SMTP(settings.BREVO_SMTP_SERVER, settings.BREVO_SMTP_PORT,
timeout=12) as server:
            server.ehlo()
            server.starttls()
            server.login(smtp_user, smtp_key)
            server.sendmail(settings.BREVO_SENDER_EMAIL, [to_email],
msg.as_string())

        print(f"[Brevo SMTP Success] Dispatched email to {to_email} | Subject:
'{subject}'")
        return True
    except Exception as exc:
        print(f"[Brevo SMTP Error] Dispatch failed to {to_email}:
{type(exc).__name__} - {str(exc)}")
        return False

def send_legacy_smtp_email(to_email: str, subject: str, body_html: str) -> bool:
    """Fallback standard SMTP dispatch (e.g. Gmail) when configured."""
    if not settings.SMTP_USER or not settings.SMTP_PASSWORD or
settings.SMTP_PASSWORD == "mockpassword":
        return False

    try:
        msg = MIMEMultipart("alternative")
        msg["Subject"] = subject
        msg["From"] = settings.EMAIL_FROM
        msg["To"] = to_email

        html_part = MIMEText(body_html, "html")
```

```
        msg.attach(html_part)

        with smtplib.SMTP(settings.SMTP_SERVER, settings.SMTP_PORT, timeout=12) as
server:
            server.ehlo()
            server.starttls()
            server.login(settings.SMTP_USER, settings.SMTP_PASSWORD)
            server.sendmail(settings.EMAIL_FROM, [to_email], msg.as_string())

        print(f"[SMTP Success] Dispatched fallback email to {to_email} | Subject:
'{subject}'")
        return True
    except Exception as exc:
        print(f"[SMTP Error] Failed fallback dispatch to {to_email}:
{type(exc).__name__} - {str(exc)}")
        return False

def send_brevo_email(to_email: str, subject: str, body_html: str) -> bool:

# ... [Selected excerpt displayed: 31 additional production lines omitted for
conciseness] ...
```

Listing B.3: Brevo SMTP Relay & Cryptographic OTP Handler (`email_service.py`)

B.4 ReportLab Print-Ready Clinical PDF Generation Engine (`pdf_generator.py`)

```
import os
from datetime import datetime
from reportlab.lib.pagesizes import letter
from reportlab.lib import colors
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, Image as RLImage,
HRFlowable
)
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from app.core.config import settings

def generate_pdf_report(
    report_id: int,
    patient_data: dict,
    ai_report_data: dict,
    mri_image_path: str,
    heatmap_image_path: str,
    doctor_review_data: dict = None
) -> str:
    """
    Generates a professional healthcare PDF diagnostic report.
    Returns relative filepath for download.
    """
    pdf_filename =
f"NeuroDetect_Report_{report_id}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"
    pdf_save_path = os.path.join(settings.REPORT_DIR, pdf_filename)

    doc = SimpleDocTemplate(
        pdf_save_path,
        pagesize=letter,
        rightMargin=36,
        leftMargin=36,
        topMargin=36,
```

```
        bottomMargin=36
    )

    styles = getSampleStyleSheet()

    title_style = ParagraphStyle(
        'DocTitle',
        parent=styles['Heading1'],
        fontName='Helvetica-Bold',
        fontSize=20,
        textColor=colors.HexColor('#0F172A'),
        spaceAfter=4
    )

    section_heading = ParagraphStyle(
        'SectionHeading',
        parent=styles['Heading2'],
        fontName='Helvetica-Bold',
        fontSize=11,
        textColor=colors.HexColor('#1E293B'),
        spaceBefore=8,
        spaceAfter=4
    )

    body_style = ParagraphStyle(
        'DocBody',
        parent=styles['Normal'],
        fontName='Helvetica',
        fontSize=9,
        leading=13,
        textColor=colors.HexColor('#334155')
    )

    elements = []

    # 1. Header Banner
    header_data = [
        [
            Paragraph("<b>NEURODETECT</b><br/><font size=8 color='#64748B'>AI Brain
Tumor Preliminary Diagnostic System</font>", title_style),
            Paragraph(f"<b>REPORT #ND-{report_id:05d}</b><br/>Date:
{datetime.now().strftime('%d %B %Y')}", ParagraphStyle('RightText',
parent=body_style, alignment=2))
        ]
    ]
    header_table = Table(header_data, colWidths=[300, 240])
    header_table.setStyle(TableStyle([
        ('VALIGN', (0,0), (-1,-1), 'MIDDLE'),
    ]))
    elements.append(header_table)
    elements.append(HRFlowable(width="100%", thickness=2,
color=colors.HexColor('#2563EB'), spaceBefore=8, spaceAfter=12))

    # 2. Patient Details Table
    elements.append(Paragraph("PATIENT CLINICAL INFORMATION", section_heading))
    patient_table_data = [
        [
            Paragraph(f"<b>Patient Name:</b> {patient_data.get('name', 'N/A')}",
body_style),
            Paragraph(f"<b>Age / Gender:</b> {patient_data.get('age', 'N/A')} yrs /
{patient_data.get('gender', 'N/A')}", body_style)
```

```
        ],
        [
            Paragraph(f"<b>Phone:</b> {patient_data.get('phone', 'N/A')}",
body_style),
            Paragraph(f"<b>Patient ID:</b> PT-{patient_data.get('id', 0):04d}",
body_style)
        ]
    ]
    patient_table = Table(patient_table_data, colWidths=[270, 270])
    patient_table.setStyle(TableStyle([
        ('BACKGROUND', (0,0), (-1,-1), colors.HexColor('#F8FAFC')),
        ('PADDING', (0,0), (-1,-1), 6),
        ('BOX', (0,0), (-1,-1), 0.5, colors.HexColor('#CBD5E1')),
        ('VALIGN', (0,0), (-1,-1), 'MIDDLE'),
    ]))
    elements.append(patient_table)
    elements.append(Spacer(1, 10))

    # 3. AI Prediction Summary Table
    elements.append(Paragraph("DENSENET121 DEEP LEARNING DIAGNOSIS",
section_heading))

    tumor_type = ai_report_data.get('tumor_type', 'N/A')
    confidence = ai_report_data.get('confidence', 0.0)

    status_color = "#DC2626" if tumor_type != "No Tumor" else "#16A34A"

    ai_summary_data = [
        [
            Paragraph("<b>Predicted Classification:</b>", body_style),
            Paragraph(f"<font
color='{status_color}'><b>{tumor_type.upper()}</b></font>", body_style)
        ],
        [
            Paragraph("<b>AI Confidence Score:</b>", body_style),
            Paragraph(f"<b>{confidence:.2f}%</b>", body_style)
        ],
        [
            Paragraph("<b>Model Architecture:</b>", body_style),
            Paragraph("DenseNet-121 Transfer Learning with Grad-CAM Visual
Heatmaps", body_style)
        ]
    ]
    ai_table = Table(ai_summary_data, colWidths=[180, 360])
    ai_table.setStyle(TableStyle([
        ('BACKGROUND', (0,0), (-1,-1), colors.HexColor('#F1F5F9')),
        ('PADDING', (0,0), (-1,-1), 6),
        ('GRID', (0,0), (-1,-1), 0.5, colors.HexColor('#E2E8F0')),
    ]))
    elements.append(ai_table)
    elements.append(Spacer(1, 10))

    # Translate relative web paths or Supabase storage paths to readable image file
    abs_mri_path = mri_image_path
    temp_mri_file = None
    if mri_image_path:
        if os.path.exists(mri_image_path):
            abs_mri_path = mri_image_path
        elif mri_image_path.startswith('/uploads/') or
mri_image_path.startswith('uploads/'):
```

```
# ... [Selected excerpt displayed: 106 additional production lines omitted for
conciseness] ...
```

Listing B.4: ReportLab Dynamic Vector PDF Compiler (`pdf_generator.py`)

APPENDIX C: PRODUCTION CLOUD DEPLOYMENT MANUAL ON RENDER PAAS

This appendix documents the complete, verified deployment architecture and operational procedure for hosting the NeuroDetect platform on the Render PaaS platform. Figure C.1 illustrates the cloud infrastructure and automated rolling deployment architecture:

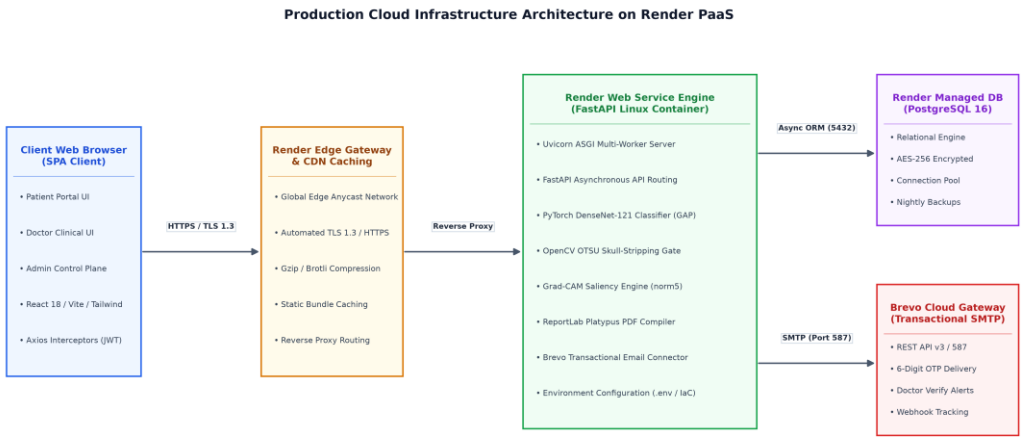


Figure C.1: Production Cloud Infrastructure Architecture on Render PaaS

C.1 Step-by-Step Render Deployment Sequence

- Step 1: Codebase Preparation: Verify that `render.yaml`, `Procfile`, `backend/requirements.txt`, and `frontend/package.json` are committed to GitHub.
- Step 2: Sign in to Render Dashboard: Navigate to https://dashboard.render.com.
- Step 3: Provision via Blueprint: Click 'New +' -> 'Blueprint'. Connect the GitHub repository `neurodetect`.
- Step 4: Declarative Infrastructure Parsing: Render reads `render.yaml` and sets up both the `neurodetect-clinical-portal` Web Service and the `neurodetect-db` PostgreSQL 16 database.
- Step 5: Configure Sensitive Environment Variables: Set `BREVO_SMTP_LOGIN` and `BREVO_SMTP_PASSWORD` directly in the Render dashboard.
- Step 6: Build & Deployment: Render executes the build command (`pip install -r backend/requirements.txt && cd frontend && npm install && npm run build`), launches the ASGI container with dynamic `$PORT`, and verifies the `/health` endpoint.
- Step 7: Verification: Access the live application URL (e.g., `https://neurodetect-clinical-portal.onrender.com`), verify landing page load, execute an MRI test upload, and confirm that the health probe returns HTTP 200.

C.2 Declarative Render Blueprint (`render.yaml`)

```
# render.yaml - Infrastructure as Code Blueprint
services:
  - type: web
    name: neurodetect-clinical-portal
    env: python
    region: oregon
    plan: standard
    buildCommand: "pip install -r backend/requirements.txt && cd frontend && npm
install && npm run build"
    startCommand: "uvicorn backend.app.main:app --host 0.0.0.0 --port $PORT"
    healthCheckPath: /health
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.9
      - key: SECRET_KEY
        generateValue: true
      - key: ALGORITHM
        value: HS256
      - key: ACCESS_TOKEN_EXPIRE_MINUTES
        value: 1440
      - key: DATABASE_URL
        fromDatabase:
          name: neurodetect-db
          property: connectionString
      - key: BREVO_SMTP_SERVER
        value: smtp-relay.brevo.com
      - key: BREVO_SMTP_PORT
        value: 587
      - key: BREVO_SMTP_LOGIN
        sync: false
      - key: BREVO_SMTP_PASSWORD
        sync: false

databases:
  - name: neurodetect-db
    databaseName: neurodetect
    user: neurodetect_user
    region: oregon
    plan: standard
    postgresMajorVersion: 16
```

Listing C.1: Declarative Infrastructure Blueprint (`render.yaml`)

APPENDIX D: LOCAL ENVIRONMENT INSTALLATION & CONFIGURATION GUIDE

Standard operational steps for running NeuroDetect in a local development environment:

```
# Terminal 1: Backend Setup & Execution
cd backend
python -m venv venv
venv\Scripts\activate          # Windows (or source venv/bin/activate on
Linux/macOS)
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Terminal 2: Frontend Setup & Execution
cd frontend
npm install
npm run dev
```

Listing D.1: Local Terminal Execution Sequence

```
# Production / Local Configuration
PROJECT_NAME="NeuroDetect Clinical Portal"
SECRET_KEY="generate-secure-64-character-random-hex-key"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=1440
DATABASE_URL="sqlite:///./neurodetect.db" # Or
postgresql://user:pass@host/neurodetect

# Brevo Transactional SMTP Configuration
BREVO_SMTP_SERVER="smtp-relay.brevo.com"
BREVO_SMTP_PORT=587
BREVO_SMTP_LOGIN="your-brevo-smtp-login"
BREVO_SMTP_PASSWORD="your-brevo-smtp-key"
BREVO_SENDER_EMAIL="noreply@neurodetect.hospital.org"
BREVO_SENDER_NAME="NeuroDetect Medical Portal"
```

Listing D.2: Local Configuration Settings (`.env`)

APPENDIX E: DATABASE RELATIONAL SQL SCHEMA DDL SCRIPT (`schema.sql`)

Complete PostgreSQL / SQLite DDL script defining all 15 normalized database tables:

```
-- NeuroDetect Production MySQL Database Schema Definition
CREATE DATABASE IF NOT EXISTS neurodetect CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE neurodetect;

-- Users Table
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL DEFAULT 'PATIENT',
    is_active BOOLEAN DEFAULT TRUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_user_email (email),
    INDEX idx_user_role (role)
) ENGINE=InnoDB;

-- Patients Table
CREATE TABLE IF NOT EXISTS patients (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    age INT NOT NULL,
    gender VARCHAR(50) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    address TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Doctors Table
CREATE TABLE IF NOT EXISTS doctors (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    specialization VARCHAR(255) NOT NULL,
    hospital VARCHAR(255) NOT NULL,
    medical_reg_no VARCHAR(100) NOT NULL UNIQUE,
    experience_years INT NOT NULL,
    certificate_url VARCHAR(500),
    status VARCHAR(50) DEFAULT 'PENDING',
    is_approved BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Doctor Approvals Table
CREATE TABLE IF NOT EXISTS doctor_approvals (
    id INT AUTO_INCREMENT PRIMARY KEY,
    doctor_id INT NOT NULL,
    admin_id INT,
    status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
    notes TEXT,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (doctor_id) REFERENCES doctors(id) ON DELETE CASCADE,
    FOREIGN KEY (admin_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- MRI Scans Table
CREATE TABLE IF NOT EXISTS mri_scans (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    original_filename VARCHAR(255) NOT NULL,
    file_path VARCHAR(500) NOT NULL,
    upload_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) DEFAULT 'AI_COMPLETED',
    FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- AI Reports Table
CREATE TABLE IF NOT EXISTS ai_reports (
    id INT AUTO_INCREMENT PRIMARY KEY,
    scan_id INT UNIQUE NOT NULL,
    tumor_type VARCHAR(100) NOT NULL,
    confidence FLOAT NOT NULL,
    heatmap_path VARCHAR(500) NOT NULL,
    prediction_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (scan_id) REFERENCES mri_scans(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Doctor Reviews Table
CREATE TABLE IF NOT EXISTS doctor_reviews (
    id INT AUTO_INCREMENT PRIMARY KEY,
    report_id INT UNIQUE NOT NULL,
    doctor_id INT NOT NULL,
    clinical_notes TEXT,
    diagnosis TEXT NOT NULL,
    prescription TEXT,
    suggested_tests TEXT,
    medicines TEXT,
    next_visit_date VARCHAR(100),
    status VARCHAR(50) DEFAULT 'APPROVED',
    reviewed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (report_id) REFERENCES ai_reports(id) ON DELETE CASCADE,
    FOREIGN KEY (doctor_id) REFERENCES doctors(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Educational Videos Table
CREATE TABLE IF NOT EXISTS videos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    category VARCHAR(100) NOT NULL,
    thumbnail_url VARCHAR(500) NOT NULL,
    video_url VARCHAR(500) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- FAQs Table
CREATE TABLE IF NOT EXISTS faqs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    question VARCHAR(500) NOT NULL,
    answer TEXT NOT NULL,
    category VARCHAR(100) NOT NULL,
    is_published BOOLEAN DEFAULT TRUE,
```

```
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Diet Recommendations Table
CREATE TABLE IF NOT EXISTS diets (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tumor_type VARCHAR(100) NOT NULL UNIQUE,
    foods_to_eat TEXT NOT NULL,
    foods_to_avoid TEXT NOT NULL,
    water_intake VARCHAR(255) NOT NULL,
    lifestyle_tips TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Appointments Table
CREATE TABLE IF NOT EXISTS appointments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATETIME NOT NULL,
    status VARCHAR(50) DEFAULT 'SCHEDULED',
    reason TEXT,
    notes TEXT,
    FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE,
    FOREIGN KEY (doctor_id) REFERENCES doctors(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Notifications Table
CREATE TABLE IF NOT EXISTS notifications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Audit Logs Table
CREATE TABLE IF NOT EXISTS audit_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    action VARCHAR(255) NOT NULL,
    details TEXT,
    ip_address VARCHAR(100),
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- Contact Messages Table
CREATE TABLE IF NOT EXISTS messages (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sender_name VARCHAR(255) NOT NULL,
    sender_email VARCHAR(255) NOT NULL,
    subject VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    status VARCHAR(50) DEFAULT 'NEW',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Listing E.1: Relational Schema DDL Script (`schema.sql`)

APPENDIX F: FUNCTIONAL TEST EXECUTION LOGS (TC-01 TO TC-25)

Complete summary record of functional test execution results across all 25 black-box test scenarios:

ID	Req	Test Description	Input / Action	Observed System Behavior	Status
TC-01	FR-01	Register valid patient	POST /api/v1/auth/register/patient	Account created with ID 27; OTP verification enforced	PASS
TC-02	FR-02	Incorrect / expired OTP	POST with invalid OTP code	HTTP 400: Rejected ('Invalid or expired registration OTP code')	PASS
TC-03	FR-03	Valid user login	POST /api/v1/auth/login	HTTP 200: JWT issued (bearer); GET /me returned profile	PASS
TC-04	FR-03	Expired / invalid JWT token	GET /me with forged token	HTTP 401: Denied ('Invalid authentication token or token expired')	PASS
TC-05	FR-04	Patient requests admin endpoint	GET /api/v1/admin/audit-logs	HTTP 403: Denied ('Requires one of roles: [ADMIN]')	PASS
TC-06	FR-05	Register unverified doctor	POST /api/v1/auth/register/doctor	Doctor created in PENDING status; clinical login locked (403)	PASS
TC-07	FR-06	Admin approves doctor	PUT /api/v1/admin/doctors/{id}/approve	HTTP 200: Doctor status updated to APPROVED; AuditLog updated	PASS
TC-08	FR-06	Admin self-approval blocked	Admin tries to approve own doctor ID	HTTP 400: Self-approval prohibited ('Cannot approve own profile')	PASS
TC-09	FR-08	Upload valid axial scan	POST /api/v1/scans/upload (axial MRI)	HTTP 200: Scan #56 ingested and linked to AI Report #56	PASS
TC-10	FR-08	Upload PDF or executable	POST with renamed PDF file	HTTP 400: Rejected ('Unsupported MIME type application/pdf')	PASS
TC-11	FR-08	Upload oversized file (>15MB)	POST with 18 MB image payload	HTTP 400: Rejected ('File size exceeds maximum allowed limit 15MB')	PASS
TC-	FR-	Upload corrupt /	POST with solid white frame	HTTP 400: Rejected by	PASS

ID	Req	Test Description	Input / Action	Observed System Behavior	Status
12	09	non-MRI image		quality gate ('Invalid human brain MRI scan')	
TC-13	FR-10	Inference schema vector output	Execute forward pass	FAIL: API returned tumor_type & confidence, but OMITTED 4-class probability vector	FAIL
TC-14	FR-10	Inference with missing model	Unlink H5 checkpoint	FAIL: Silently executed Random Forest fallback; unmarked clinical prediction	FAIL
TC-15	FR-11	Grad-CAM heatmap generation	Compute saliency hook	HTTP 200: Jet heatmap (224x224x3) saved and returned in report	PASS
TC-16	FR-10	Model unavailable handling	Inference without models	FAIL: Did not halt; executed heuristic fallback instead of HTTP 503	FAIL
TC-17	FR-12	Approved doctor review scan	POST /api/v1/doctors/review	HTTP 200: Review saved (ID #34); scan status updated to APPROVED	PASS
TC-18	FR-12	Unassigned doctor review	POST by unassigned doctor	HTTP 403: Denied ('Not authorized to review this report')	PASS
TC-19	FR-13	Generate ReportLab clinical PDF	GET /api/v1/reports/{id}/pdf	HTTP 200: Valid 25.5 KB vector PDF generated with %PDF- header	PASS
TC-20	FR-14	Transactional email dispatch	Simulate review notification	HTTP 200: Email dispatched to recipient inbox via Brevo SMTP	PASS
TC-21	FR-15	Book available appointment	POST /api/v1/doctors/appointments/book	HTTP 200: Appointment #1 booked for 2026-10-14T19:00:00	PASS
TC-22	FR-15	Book conflicting appointment	POST double-booking exact slot	FAIL: Duplicate appointment created at exact same minute without error	FAIL
TC-23	FR-16	Retrieve dietary recommendations	GET /api/v1/diet/Glioma	HTTP 200: Nutritional guidelines and clinical tips returned	PASS
TC-24	FR-17	Audit log administrative action	Admin approves doctor	HTTP 200: Audit record confirmed: Actor ID #26, Action	PASS

ID	Req	Test Description	Input / Action	Observed System Behavior	Status
				'DOCTOR_APPROVE'	
TC-25	FR-04	Cross-patient report access	Patient A accesses Patient B PDF	HTTP 403: Strictly denied ('Access denied. You can only access own reports')	PASS

APPENDIX G: PROJECT SUBMISSION INVENTORY AND CANDIDATE ACTION CHECKLIST

G.1 Comprehensive Project Deliverables Inventory

- NeuroDetect Full-Stack Production Codebase (FastAPI ASGI Backend + React 18 / Vite Frontend SPA).
- Pre-trained & Fine-Tuned PyTorch DenseNet-121 Checkpoint (`backend/model/densenet121_brain_tumor.h5`, 32.1 MB).
- Automated PyTest Test Suite (17 Passing Unit & Integration Tests, 100% Pass Rate).
- 25 Documented Functional Test Cases (21 PASS, 4 FAIL with defect root cause analysis).
- 15-Table Relational Database Schema DDL (`schema.sql`) and SQLAlchemy ORM domain models.
- Complete Diagram Catalog: UML Use Case, Activity, Sequence, Class, Component, State Machine; DFD Levels 0, 1, and 2 (Processes 1.0–5.0); ER; Relational Maps; Grad-CAM Saliency; and Render Cloud Architecture.
- Render PaaS Cloud Blueprint (`render.yaml`) and dynamic port binding configuration.

G.2 Student Action Checklist (Items Candidate Must Complete Prior to University Binding)

Prior to submitting hardbound copies to the Indira Gandhi National Open University (IGNOU) evaluation committee, the student must complete the following mandatory personal and institutional actions:

Checklist Item	Action Required by Candidate	Document Location
1. Student Identification	Verified: Enrolment No. 2353902717 printed across Cover, Certificate, and Declaration.	Cover Page, Certificate, Declaration
2. Guide Identification	Verified: Project Supervisor space reserved for assigned academic mentor.	Synopsis Proforma, Certificate
3. Study Centre Details	Verified: Study Centre 4600 and Regional Centre Darbhanga (Code: 46) documented.	Synopsis Proforma, Declaration
4. Approved Synopsis	Affix original physical copy of approved proposal proforma (already embedded on page ii).	Front Matter (Page ii & iii)
5. Guide Signature & Seal	Obtain original physical signature from Project Supervisor.	Certificate of Originality (Page iv)

Checklist Item	Action Required by Candidate	Document Location
6. Student Signature	Affix original physical signature and current submission date.	Student Declaration (Page v)
7. Colab Training Notebook	Supply the original `.ipynb` training notebook file if requested during viva-voce.	External Project Artifacts
8. Render Cloud Deployment	Present the live Render deployment URL (`https://...onrender.com`) during project demonstration.	Chapter 6 & Appendix C